System Design: Zero to Hero

A study report covering theory, trade-offs, technical terms, alternatives (including free ones), which language suits which component, and working code.

How to read this report

The report has sixteen parts. Parts 1 to 6 are the core course: theory, trade-offs, tools, languages, code and a roadmap. Parts 7 to 16 are deep dives: database and storage internals, performance math and networking, twelve full case studies, data and ML systems, cloud and DevOps, reliability and security, infrastructure configurations, tested programs, and a reference section (decision sheets, numbers, glossary, Q&A, debugging playbook). Read them in order the first time.

- **Part 1: Foundations and theory.** Vocabulary, estimation, scaling, CAP, consistency, load balancing, caching.
- **Part 2: Data, messaging, networking and reliability.** Databases, sharding, queues, APIs, resilience patterns, architecture styles, distributed-systems theory.
- **Part 3: Tools and alternatives.** A table for every building block with paid, open-source and free options.
- **Part 4: Which language for which component.** Pros and cons per language and a mapping table.
- **Part 5: Code and case studies.** Working code for the key building blocks, worked designs, and a learning roadmap.

The one rule behind everything: **system design is the art of choosing trade-offs on purpose.** Nothing is free. Every choice buys one property and pays with another. Whenever you read a technique below, ask what it buys and what it costs.

Part 1: Foundations and Theory

1.1 What system design is

System design is deciding how the components of a software system (clients, servers, databases, caches, queues, networks) fit together so that the system meets its requirements at the scale and cost you need. It has two levels:

- **High-level design (HLD):** the boxes and arrows. Which services exist, which databases, how data flows.
- **Low-level design (LLD):** the inside of one box. Classes, schemas, algorithms, APIs.

This report is mostly HLD, with LLD code in Part 5.

1.2 A framework you can apply to any design problem

1. **Clarify requirements.** Ask what the system must do (functional) and how well it must do it (non-functional).
2. **Estimate scale.** Users, requests per second, data size, read/write ratio.
3. **Define the API and data model.** What are the operations and the entities?
4. **Draw the simple version first.** One client, one server, one database. It must work before it scales.
5. **Find the bottleneck.** Where does the simple version break first as load grows?
6. **Fix it with the right building block.** Cache, replica, queue, shard, CDN, and so on.
7. **Repeat steps 5 and 6** until the requirements are met.
8. **Discuss trade-offs, failure modes and cost.** What happens when each component dies?

Functional requirements are features: users can post, search, upload. **Non-functional requirements (NFRs)** are qualities: latency, availability, durability, consistency, security, cost, maintainability. NFRs decide the architecture far more than features do.

1.3 Core vocabulary

Term	Meaning	Why it matters
Latency	Time for one request to complete (ms).	What the user feels. Look at p50, p95, p99, not only the average.
Throughput	Work done per unit time (requests/sec, MB/sec).	Capacity. Latency and throughput trade against each other.
Bandwidth	Maximum data a link can carry per second.	The ceiling on throughput over a network.

Term	Meaning	Why it matters
QPS / RPS	Queries or requests per second.	The basic unit of load.
Scalability	Ability to handle growth by adding resources.	Vertical (bigger machine) or horizontal (more machines).
Availability	Fraction of time the system works.	Measured in nines (see below).
Reliability	Works correctly over time, including under faults.	A system can be available but wrong.
Durability	Committed data is not lost.	Disks fail; replication and backups provide durability.
Consistency	All readers see the same data.	The hardest property in distributed systems.
Fault tolerance	Keeps working when parts fail.	Achieved with redundancy.
SPOF	Single point of failure.	Any component whose death kills the system. Remove them.
Idempotent	Doing it twice has the same effect as once.	Makes retries safe.
Stateless	Server keeps no per-user data between requests.	Makes horizontal scaling easy.
SLI / SLO / SLA	Indicator (measured), objective (internal target), agreement (contract with penalty).	How reliability is defined and promised.
RTO / RPO	Recovery time objective, recovery point objective.	How long you can be down, and how much data you can lose.
Hot spot	One node getting disproportionate load.	Breaks sharding and caching if not handled.
Tail latency	The slowest few percent of requests.	At scale, the tail dominates user experience.
Backpressure	A slow consumer signals a fast producer to slow down.	Prevents overload and memory blowups.
Throttling / rate limiting	Capping how much a client may do.	Protects the system from abuse and spikes.

Term	Meaning	Why it matters
Eventual consistency	Replicas converge if updates stop.	Accepts temporary staleness for speed and availability.

Availability in nines

Availability	Downtime per year	Downtime per month
99% (two nines)	3.65 days	7.3 hours
99.9% (three nines)	8.76 hours	43.8 minutes
99.99% (four nines)	52.6 minutes	4.4 minutes
99.999% (five nines)	5.26 minutes	26 seconds

If components are in **series** (all must work), availabilities multiply: two 99.9% components give about 99.8%. If they are in **parallel** (any one is enough), failure probabilities multiply: two 99.9% replicas give 99.9999%. This is why redundancy is the basis of high availability, and why long chains of dependencies hurt.

1.4 Back-of-envelope estimation

Estimation tells you whether you need one server or one thousand, so you do not over- or under-build. Memorize these orders of magnitude.

Latency numbers every engineer should know (approximate):

Operation	Time
CPU L1 cache reference	1 ns
Main memory (RAM) reference	100 ns
Read 1 MB sequentially from RAM	10 microseconds
SSD random read	100 microseconds
Read 1 MB sequentially from SSD	0.5 ms
Round trip within the same data centre	0.5 ms

Operation	Time
Read 1 MB sequentially from HDD	5 ms
Disk seek (HDD)	10 ms
Round trip across continents	100-150 ms

The lessons: memory is about 1,000x faster than SSD, SSD is much faster than a network call across regions, and sequential access beats random access on every medium.

Handy conversions:

- 1 day is about 86,400 seconds, call it 100,000 for quick math.
- 1 million requests per day is about 12 requests per second.
- 1 billion requests per day is about 12,000 requests per second.
- Peak traffic is typically 2x to 5x the average.
- 1 KB = 1,000 bytes, 1 MB = 1,000 KB, and so on. Powers of ten are fine for estimates.

Worked example: a photo-sharing app with 10 million daily users.

- Each user views 50 photos and uploads 0.1 photos per day.
- Reads: 10M x 50 = 500M reads/day, about 5,800 reads/sec average, about 20,000 at peak.
- Writes: 10M x 0.1 = 1M uploads/day, about 12 uploads/sec.
- Read-to-write ratio is about 500:1, so the system is **read-heavy**. That points to caching, CDN and read replicas.
- Storage: 1M photos/day x 2 MB = 2 TB/day, about 730 TB/year. That is too large for one database, so use object storage.
- Bandwidth out: 20,000 reads/sec x 200 KB (thumbnail) = 4 GB/sec at peak. A CDN is mandatory.

Notice how a few multiplications produced real architecture decisions.

1.5 Scalability

Vertical scaling (scale up)

Buy a bigger machine: more CPU, RAM, faster disk.

- **Pros:** simple, no code changes, no distributed-systems problems.

- **Cons:** hardware has a ceiling, cost grows faster than capacity, and it is still a single point of failure.

Horizontal scaling (scale out)

Add more machines and spread the load.

- **Pros:** nearly unlimited growth, cheap commodity hardware, built-in redundancy.
- **Cons:** you now have a distributed system, which means network failures, partial failures, data consistency issues and operational complexity.

Rule of thumb: scale vertically until it hurts, then horizontally. Many real systems stay on a big single database for far longer than people expect.

Making services stateless

Horizontal scaling of application servers only works cleanly if any server can handle any request. So push state out: sessions go to Redis or into a signed token (JWT), files go to object storage, data goes to the database. The app servers become interchangeable and disposable, so you can add, remove or restart them freely.

Trade-off: externalizing state adds a network hop per request and makes the external store a critical dependency.

1.6 The CAP theorem and PACELC

In a distributed data store, you can have at most two of these three at once when a network partition happens:

- **Consistency (C):** every read returns the most recent write (this is linearizability, not the same C as in ACID).
- **Availability (A):** every request to a working node gets a non-error response.
- **Partition tolerance (P):** the system keeps working when messages between nodes are lost.

Networks do fail, so partitions are a fact of life and **P is not optional**. The real choice during a partition is:

- **CP:** refuse some requests to stay correct. Examples: HBase, MongoDB (default config), etcd, ZooKeeper, Redis Cluster in strict setups. Good for money, inventory, coordination.

- **AP:** keep answering, possibly with stale data, and reconcile later. Examples: Cassandra, DynamoDB (default), CouchDB, Riak. Good for feeds, carts, analytics.

CAP only describes behavior during a partition. **PACELC** completes the picture: if there is a **P**artition, choose **A** or **C**; **E**lse (normal operation), choose **L**atency or **C**onsistency. Even without failures, stronger consistency costs latency because replicas must coordinate. DynamoDB and Cassandra are PA/EL (fast and available, weaker consistency); traditional single-leader SQL setups are PC/EC.

1.7 ACID, BASE and consistency models

ACID (traditional transactional databases):

- **Atomicity:** all of a transaction happens or none of it.
- **Consistency:** the transaction moves the database from one valid state to another (constraints hold).
- **Isolation:** concurrent transactions do not see each other's partial work.
- **Durability:** once committed, it survives a crash.

BASE (many distributed NoSQL stores): **B**asically **A**vailable, **S**oft state, **E**ventually consistent.

Isolation levels (weakest to strongest) and what they allow:

Level	Dirty read	Non-repeatable read	Phantom read
Read uncommitted	Possible	Possible	Possible
Read committed	No	Possible	Possible
Repeatable read	No	No	Possible (not in some engines)
Serializable	No	No	No

Stronger isolation is safer and slower. Most databases default to read committed or repeatable read.

Consistency models for distributed data, strongest to weakest:

1. **Linearizable (strong):** behaves as if there is one copy and operations happen instantly in real-time order. Expensive.
2. **Sequential:** everyone sees the same order, which may lag real time.

3. **Causal:** if A caused B, everyone sees A before B. Unrelated events may be seen in any order.
4. **Read-your-writes:** you always see your own updates.
5. **Eventual:** replicas converge if writes stop. Cheapest and fastest.

Choosing: use strong consistency for money, inventory and uniqueness constraints. Use eventual consistency for likes, view counts, timelines and recommendations.

1.8 Load balancing

A **load balancer (LB)** spreads incoming requests across many servers, detects dead servers with **health checks**, and hides the server fleet behind one address.

Layers

- **Layer 4 (transport):** routes on IP and port. Very fast, protocol-agnostic, cannot see HTTP content. Examples: HAProxy in TCP mode, AWS NLB, LVS.
- **Layer 7 (application):** routes on URL, headers, cookies. Can do path routing, TLS termination, caching, compression. Slower per request, much more flexible. Examples: Nginx, HAProxy HTTP mode, Envoy, Traefik, AWS ALB.

Algorithms

Algorithm	How it works	Best for	Weakness
Round robin	Next server in turn	Uniform requests and servers	Ignores actual load
Weighted round robin	Bigger servers get more turns	Mixed hardware	Static weights
Least connections	Send to the server with fewest active connections	Long or uneven requests	Needs connection tracking
Least response time	Fastest responder wins	Latency-sensitive	More measurement overhead
IP hash	Hash the client IP to pick a server	Simple stickiness	Uneven when few clients are huge

Algorithm	How it works	Best for	Weakness
Consistent hashing	Key maps to a ring of servers	Caches, sharded backends	More complex
Power of two choices	Pick two at random, take the less loaded	Large fleets	Probabilistic

Sticky sessions pin a user to one server. They are convenient but defeat statelessness and cause imbalance. Prefer externalized sessions.

The LB itself is a SPOF. Run at least two (active-passive with a floating IP via keepalived/VRRP, or active-active behind DNS or anycast).

Global server load balancing (GSLB) uses DNS or anycast to direct users to the nearest or healthiest region.

1.9 Caching

A cache stores the results of expensive operations in fast storage so repeated requests are cheap. It is the single most effective performance tool.

Where caches live

1. **Browser cache:** controlled by `Cache-Control`, `ETag`, `Expires` headers.
2. **CDN:** edge servers near users cache static (and sometimes dynamic) content.
3. **Reverse proxy / API gateway cache:** Nginx, Varnish.
4. **Application cache:** in-process memory (fastest, but per-instance and not shared).
5. **Distributed cache:** Redis or Memcached shared by all app servers.
6. **Database cache:** buffer pool, query cache.

Caching strategies

Strategy	Flow	Pros	Cons
Cache-aside (lazy loading)	App checks cache; on miss, reads DB and fills cache	Simple, only caches what is used, cache failure is survivable	First request is slow; data can go stale

Strategy	Flow	Pros	Cons
Read-through	Cache itself loads from DB on miss	Cleaner app code	Needs cache library support
Write-through	Write to cache and DB together	Cache always fresh	Slower writes; caches data that may never be read
Write-back (write-behind)	Write to cache, flush to DB later	Very fast writes, batching	Data loss if cache dies before flush
Write-around	Write to DB only, skip the cache	Avoids cache pollution	Next read is a miss

Eviction policies

- **LRU** (least recently used): the default choice, good for most workloads.
- **LFU** (least frequently used): better when popularity is stable.
- **FIFO:** simple, rarely optimal.
- **TTL** (time to live): expire after a fixed time; used together with the above.

The hard problems

- **Cache invalidation.** Keeping cache and source of truth in sync. Options: short TTLs, explicit delete on write, event-driven invalidation. There is no perfect answer, which is why this is a famously hard problem.
- **Cache stampede (thundering herd).** A popular key expires and thousands of requests hit the database at once. Fixes: request coalescing (only one request rebuilds), locking, jittered TTLs, early refresh.
- **Cache penetration.** Requests for keys that do not exist bypass the cache every time. Fixes: cache negative results briefly, Bloom filter in front.
- **Hot keys.** One key gets enormous traffic. Fixes: replicate the key across nodes, add a local in-process layer.
- **Cold start.** An empty cache after restart overloads the database. Fix: warm the cache before taking traffic.

Trade-off summary: caching buys latency and database relief and pays with staleness, complexity and a new failure mode. Never cache without deciding how stale the data is allowed to be.

Free options: Redis (note licensing changes; Valkey is the fully open fork), Memcached, Varnish, Nginx cache, Caffeine (Java in-process), Ristretto (Go in-process).

Part 2: Data, Messaging, Networking and Reliability

2.1 Databases

Choosing the database is usually the most consequential decision in a design, because data is the hardest thing to migrate.

Relational (SQL)

Tables, rows, a fixed schema, joins, ACID transactions.

- **Pros:** strong consistency, powerful queries (joins, aggregates), mature tooling, constraints protect data integrity.
- **Cons:** harder to scale writes horizontally, schema changes can be painful at scale, joins across shards are expensive.
- **Use when:** data is structured and related, correctness matters (payments, orders, accounts), and you want flexible ad-hoc queries.
- **Examples:** PostgreSQL, MySQL, MariaDB, SQLite (all free), Oracle, SQL Server (paid), CockroachDB and YugabyteDB (distributed SQL, with free tiers or open-source editions).

NoSQL families

Family	Data shape	Strengths	Weaknesses	Free examples
Key-value	key to opaque value	Extremely fast, trivial to shard	Query only by key	Redis/Valkey, Memcached, etcd, RocksDB (embedded)

Family	Data shape	Strengths	Weaknesses	Free examples
Document	JSON-like documents	Flexible schema, natural for app objects, good horizontal scaling	Weak joins, duplication of data	MongoDB (SSPL), CouchDB, FerretDB
Wide-column	rows with dynamic columns, grouped by partition key	Massive write throughput, linear scaling, multi-region	Query patterns must be designed up front, no joins	Apache Cassandra, ScyllaDB (limited free), HBase
Graph	nodes and edges	Relationship traversal (friends of friends) is fast	Hard to shard, niche	Neo4j Community, ArangoDB, JanusGraph
Time-series	timestamped measurements	Efficient compression and range queries	Specialized	InfluxDB (OSS), TimescaleDB (Postgres extension), Prometheus
Search	inverted indexes	Full-text search, relevance ranking, faceting	Not a source of truth	Elasticsearch/OpenSearch, Meilisearch, Typesense, Apache Solr
Object storage	blobs by key	Cheap, huge, durable	No queries, higher latency	MinIO, Ceph (self-hosted); S3, GCS, R2 (managed)
Columnar / OLAP	column-oriented tables	Fast analytics over billions of rows	Poor for single-row updates	ClickHouse, DuckDB, Apache Druid, Apache Pinot

Practical advice: start with PostgreSQL. It handles relational data, JSON documents, full-text search, geospatial queries (PostGIS) and time-series (TimescaleDB) acceptably. Add specialized stores only when a measured bottleneck demands it. Using five databases on day one is a common beginner mistake.

SQL vs NoSQL: how to decide

Question	Leans SQL	Leans NoSQL
Is the data highly relational?	Yes	No
Do you need multi-row transactions?	Yes	Rarely
Is the schema stable?	Yes	Changes often
Is write volume beyond one node?	No	Yes
Are the access patterns known and simple (lookup by key)?	No (ad-hoc queries)	Yes
Do you need global multi-region writes?	Harder	Easier (AP stores)

Indexing

An **index** is an auxiliary data structure that makes lookups fast at the cost of extra storage and slower writes.

- **B-tree / B+ tree:** the default. Good for equality and range queries. Used by PostgreSQL, MySQL (InnoDB).
- **Hash index:** equality only, very fast.
- **LSM tree (log-structured merge tree):** writes go to memory then flush to sorted files and are compacted in the background. Excellent write throughput, slower and more complex reads. Used by Cassandra, RocksDB, LevelDB, HBase.
- **Inverted index:** maps words to documents. Used by search engines.
- **Bitmap, geospatial (R-tree, geohash, quadtree) and Bloom filters** serve special cases.

B-tree vs LSM trade-off: B-trees favor reads and in-place updates; LSM trees favor writes and sequential disk use but need compaction and can have read amplification.

Index rules of thumb: index columns used in WHERE, JOIN and ORDER BY; use composite indexes with the most selective column first when it matches queries; do not over-index write-heavy tables; check query plans with `EXPLAIN`.

Replication

Copying data to multiple nodes for availability and read scale.

Model	How it works	Pros	Cons
Single-leader (primary-replica)	All writes go to the leader, copied to followers; reads can go to followers	Simple, no write conflicts	Leader is a write bottleneck; replica lag gives stale reads; failover is tricky
Multi-leader	Several nodes accept writes	Multi-region write latency, tolerates leader loss	Write conflicts must be resolved
Leaderless (Dynamo-style)	Any node accepts writes; quorum reads and writes	High availability, no failover	Conflict resolution, tunable consistency

- **Synchronous replication:** the leader waits for followers before confirming. No data loss on leader crash, but slower and blocks if a follower stalls.
- **Asynchronous replication:** the leader confirms immediately. Fast, but recent writes can be lost on failover and followers lag.
- **Semi-synchronous:** wait for at least one follower. A common compromise.

Quorums: with N replicas, write to W and read from R. If **R + W > N**, reads overlap with the latest write, giving strong-ish consistency. Example: N=3, W=2, R=2. Lowering R or W gives speed and availability at the cost of consistency.

Conflict resolution in multi-leader and leaderless systems: last-write-wins (simple, loses data, depends on clocks), vector clocks (detect concurrency), application-level merge, or **CRDTs** (data types that merge automatically, used in collaborative editing and counters).

Sharding (partitioning)

Splitting a dataset across machines when it no longer fits on one, or one node cannot handle the writes.

Strategies:

1. **Range-based:** shard by key ranges (A-F, G-M). Easy range queries, but hot spots (for example, timestamps all hit the newest shard).
2. **Hash-based:** shard = hash(key) mod N. Even distribution, but range queries scatter and changing N reshuffles almost everything.
3. **Consistent hashing:** servers and keys are placed on a ring; a key belongs to the next server clockwise. Adding or removing a server moves only about 1/N of the keys. Virtual nodes smooth out imbalance. Used by Cassandra, DynamoDB, many caches and CDNs.
4. **Directory-based:** a lookup service maps keys to shards. Most flexible, but the directory is a new SPOF and bottleneck.
5. **Geographic:** shard by region for latency and data-residency laws.

Costs of sharding: cross-shard joins and transactions are hard or impossible, rebalancing is operationally painful, schema changes multiply, and a bad shard key creates hot shards. **Shard only when necessary**, and choose a key with high cardinality, even distribution and alignment with the dominant query pattern.

Alternatives before sharding: better indexes, caching, read replicas, vertical scaling, archiving old data, or a distributed SQL database that shards for you.

Denormalization and the CQRS idea

Normalization removes duplication and keeps data consistent; denormalization duplicates data to make reads fast. Large systems often keep a normalized source of truth and build denormalized, read-optimized copies (materialized views, search indexes, caches). This is the seed of **CQRS** (Part 2.7).

2.2 Messaging, queues and streaming

A **message queue** decouples producers from consumers. The producer drops a message and moves on; consumers process it when ready.

Why use one:

- **Decoupling:** services do not need to know each other or be up at the same time.
- **Buffering (load leveling):** absorb traffic spikes so downstream does not collapse.
- **Async work:** send email, resize images, and so on, outside the request path.
- **Reliability:** retries and dead-letter handling.

- **Fan-out:** one event, many consumers.

Cost: eventual consistency, harder debugging, ordering and duplicate-delivery problems, and another system to operate.

Queue vs log

Property	Traditional queue (RabbitMQ, SQS, ActiveMQ)	Distributed log (Kafka, Redpanda, Pulsar)
Model	Message removed after consumption	Messages retained; consumers track an offset
Replay	Generally no	Yes, re-read history
Ordering	Per queue (with caveats)	Per partition
Throughput	Good	Very high (millions msgs/sec)
Consumer scaling	Competing consumers	Consumer groups, limited by partition count
Best for	Task distribution, job queues	Event streaming, analytics, event sourcing

Delivery guarantees

- **At-most-once:** may lose messages, never duplicates.
- **At-least-once:** never loses, may duplicate. The practical default. **Consumers must be idempotent.**
- **Exactly-once:** achievable only in limited forms (Kafka transactions, idempotent producers plus deduplication). In general, think of it as at-least-once plus idempotent processing.

Patterns

- **Point-to-point (work queue):** each message goes to one consumer.
- **Publish/subscribe:** each message goes to every subscriber.
- **Dead-letter queue (DLQ):** messages that fail repeatedly go aside for inspection.

- **Outbox pattern:** write the event into the same database transaction as the data change, then a relay publishes it. Solves the problem of updating the database and publishing a message atomically.
- **Competing consumers, priority queues, delayed messages, request-reply.**

Free options: RabbitMQ, Apache Kafka, Redpanda (source-available), Apache Pulsar, NATS and NATS JetStream, Redis Streams, ZeroMQ (library), Apache ActiveMQ, Postgres-based queues (pgmq, SKIP LOCKED pattern) for small scale.

2.3 Networking essentials

How a request travels

1. The browser checks its caches, then asks **DNS** to turn a name into an IP address.
2. It opens a **TCP** connection (three-way handshake) and, for HTTPS, a **TLS** handshake.
3. It sends an **HTTP** request, which may hit a **CDN**, a **load balancer**, an **API gateway**, then an app server.
4. The app server talks to caches and databases and returns a response.

DNS

Hierarchical, cached naming system. Record types: A/AAAA (name to IP), CNAME (alias), MX (mail), TXT, NS. Uses TTLs for caching. DNS is also used for load balancing (round-robin, geo-DNS, latency-based routing). **Trade-off:** caching makes failover slow because clients keep old answers until the TTL expires. Free: BIND, CoreDNS, PowerDNS, Unbound, Cloudflare (free tier).

TCP vs UDP

	TCP	UDP
Connection	Yes (handshake)	No
Delivery	Reliable, ordered	Best effort, unordered
Flow and congestion control	Yes	No
Overhead	Higher	Lower

	TCP	UDP
Used for	Web, databases, file transfer	Video/voice, games, DNS, QUIC

QUIC / HTTP/3 runs over UDP and rebuilds reliability in user space, removing head-of-line blocking and speeding up connection setup.

HTTP versions

- **HTTP/1.1:** one request at a time per connection (browsers open several), keep-alive.
- **HTTP/2:** multiplexing many streams on one connection, header compression.
- **HTTP/3:** HTTP over QUIC, better on lossy networks and mobile.

Real-time communication options

Technique	Direction	Notes
Short polling	Client asks repeatedly	Simple, wasteful
Long polling	Client asks, server holds the request until data exists	Works everywhere, uses many connections
Server-Sent Events (SSE)	Server to client	Simple, one-way, auto-reconnect
WebSocket	Both directions, persistent	Chat, games, collaboration; stateful connections complicate load balancing
WebRTC	Peer to peer	Audio, video, low-latency data
Webhooks	Server to server callback	For integrations

CDN (content delivery network)

A geographically distributed fleet of caches. Users fetch static assets (images, JS, video) from a nearby edge server, which cuts latency and offloads your origin. **Pull CDN:** the edge fetches from origin on first miss. **Push CDN:** you upload content to the CDN ahead

of time. Invalidation is the usual pain: use versioned file names (app.v123.js) to avoid it. Free or low-cost: Cloudflare free tier, jsDelivr, BunnyCDN (cheap), or self-hosted Nginx/Varnish edges.

Proxies

- **Forward proxy:** sits in front of clients (privacy, filtering).
- **Reverse proxy:** sits in front of servers (load balancing, TLS termination, caching, security). Nginx, HAProxy, Envoy, Traefik, Caddy are all free.
- **API gateway:** a reverse proxy specialized for APIs: authentication, rate limiting, routing, request transformation. Free: Kong (OSS), APISIX, Envoy Gateway, Traefik, Tyk (OSS).

2.4 API design

Style	Idea	Pros	Cons	Use for
REST	Resources with URLs and HTTP verbs	Simple, cacheable, universal	Over- or under-fetching, many round trips	Public APIs, CRUD
GraphQL	Client specifies exactly the fields needed	No over-fetching, one endpoint, strong schema	Caching is harder, query cost control, complexity	Rich clients, mobile, many frontends
gRPC	Binary RPC with Protocol Buffers over HTTP/2	Fast, compact, streaming, typed contracts, code generation	Not browser-friendly without a proxy, harder to debug	Internal service-to-service
WebSocket / SSE	Persistent streams	Real-time	Stateful	Live updates
Message-based	Events on a broker	Decoupled	Eventual consistency	Async workflows

Good API practices:

- **Idempotency keys** for unsafe operations (payments): the client sends a unique key, the server stores the result and returns the same response on retry.

- **Pagination:** offset-based is simple but slow for deep pages and unstable under inserts; **cursor (keyset)** pagination is stable and fast.
- **Versioning:** in the URL (/v1/) or headers; never break existing clients silently.
- **Consistent error format, rate-limit headers, authentication (OAuth 2.0 / OIDC, API keys, JWT), input validation.**
- **Backward-compatible evolution:** add fields, do not rename or remove them.

2.5 Reliability and resilience patterns

In a distributed system, **everything fails eventually**: networks drop packets, disks die, dependencies slow down. Resilience means designing for failure rather than hoping it will not happen.

Rate limiting

Cap requests per client per time window to protect against abuse, bugs and overload.

Algorithm	How it works	Pros	Cons
Fixed window	Count per calendar window (e.g., per minute)	Trivial	Burst at window edges (2x limit)
Sliding window log	Store every request timestamp	Exact	Memory-heavy
Sliding window counter	Weighted blend of current and previous windows	Cheap and smooth	Approximate
Token bucket	Bucket refills at a steady rate; each request takes a token	Allows bursts, simple, widely used	Needs per-key state
Leaky bucket	Requests queue and drain at a constant rate	Smooth output rate	Bursts get delayed or dropped

In distributed setups, keep counters in Redis (with atomic Lua scripts) or enforce at the gateway. Return HTTP 429 with a `Retry-After` header.

Timeouts, retries, backoff

- **Always set timeouts.** A call without a timeout can hang forever and exhaust your threads.
- **Retry only idempotent operations**, and only on transient errors.
- **Exponential backoff with jitter:** wait 100 ms, 200 ms, 400 ms, and so on, plus randomness. Without jitter, thousands of clients retry in lockstep and create a retry storm.
- **Retry budgets:** cap retries as a percentage of traffic so retries cannot multiply load during an outage.

Circuit breaker

Wraps calls to a dependency. States: **closed** (normal), **open** (the dependency is failing, so fail fast without calling it), **half-open** (let a few trial requests through to test recovery). It prevents cascading failures and gives the dependency room to recover. Libraries: Resilience4j (Java), Polly (.NET), gobreaker (Go), opossum (Node), pybreaker (Python). Service meshes (Istio, Linkerd) can do it without code.

Bulkheads

Isolate resources (thread pools, connection pools, instances) per dependency or per tenant so one failing part cannot sink the whole ship.

Graceful degradation and fallbacks

When a non-critical part fails, serve a reduced experience: show cached recommendations, hide the reviews widget, use default values. A product page without reviews is far better than an error page.

Load shedding and backpressure

When overloaded, deliberately reject low-priority work to keep serving critical work. Bounded queues and flow control let slow consumers push back on fast producers instead of the system running out of memory.

Redundancy and failover

- **Active-passive:** a standby takes over on failure. Simpler, wastes capacity, failover takes time.

- **Active-active:** all nodes serve traffic. Better utilization, more complex state handling.
- **Multi-AZ / multi-region:** survive data-centre or region loss. Multi-region multiplies cost and consistency complexity; do it only if your availability target demands it.

Health checks and self-healing

Liveness says the process is alive (restart it if not); **readiness** says it can accept traffic (remove it from the load balancer if not). Orchestrators like Kubernetes use both.

Disaster recovery levels

Strategy	RTO	RPO	Cost
Backup and restore	Hours to days	Hours	Lowest
Pilot light (minimal copy running)	Tens of minutes	Minutes	Low
Warm standby (scaled-down copy)	Minutes	Seconds to minutes	Medium
Active-active multi-region	Near zero	Near zero	Highest

Backups you have never restored are not backups. Test restores regularly.

2.6 Distributed-systems theory you must know

Why distributed systems are hard

- The network is unreliable: messages are lost, delayed, duplicated or reordered.
- **Partial failure:** some parts fail while others work, and you often cannot tell a dead node from a slow one.
- **No global clock:** machine clocks drift, so you cannot trust timestamps to order events across machines.

Time and ordering

- **Physical clocks** (NTP): drift by milliseconds or more.
- **Lamport clocks:** a counter that gives a consistent partial order of causally related events.
- **Vector clocks:** detect whether two events are causally related or concurrent.

- **Hybrid logical clocks (HLC)** combine physical time with a logical counter (used by CockroachDB). Google Spanner uses **TrueTime** (GPS and atomic clocks with bounded uncertainty).

Consensus

Getting several nodes to agree on a value even if some crash. Needed for leader election, replicated state machines, distributed locks, configuration stores.

- **Paxos:** the original, correct and notoriously hard to understand and implement.
- **Raft:** designed to be understandable. A leader is elected, appends log entries to followers, and commits once a **majority (quorum)** has them. A cluster of 2f+1 nodes tolerates f failures (3 nodes tolerate 1, 5 tolerate 2). Used by etcd, Consul, CockroachDB, TiKV.
- **ZAB** (ZooKeeper), **Viewstamped Replication**, **PBFT** (Byzantine fault tolerance, used in blockchains and adversarial settings).

Trade-off: consensus gives strong consistency but every write needs a majority round trip, so it is slower and unavailable without a quorum. Use it for small, critical coordination data, not bulk data.

Other essentials

- **Leader election:** exactly one node coordinates. Beware **split brain** (two nodes both believe they are leader). Use quorum and **fencing tokens** to protect shared resources.
- **Distributed locks:** hard to do right. A lock with an expiry can be lost while the holder is paused (garbage-collection pause), so combine with fencing tokens. Prefer designing to avoid locks.
- **Gossip protocols:** nodes periodically exchange state with random peers; information spreads like an epidemic. Scalable membership and failure detection (Cassandra, Consul).
- **Failure detection:** heartbeats and timeouts; phi-accrual detectors adapt to network conditions.
- **Two-phase commit (2PC):** a coordinator asks all participants to prepare, then to commit. Atomic across nodes, but blocking if the coordinator dies. **Three-phase commit** reduces blocking but is rarely used.
- **Saga pattern:** a long transaction broken into local steps with compensating actions for rollback (book flight, book hotel, if hotel fails then cancel flight). Eventually consistent, no global lock. Two styles: **choreography** (services react to events) and **orchestration**

(a central coordinator directs steps).

- **Idempotency and deduplication:** the foundation of safe retries.
- **Merkle trees:** hash trees that let replicas find differences cheaply (anti-entropy repair in Cassandra, Git, blockchains).
- **Bloom filters:** space-efficient probabilistic set that says definitely not present or possibly present. Used to avoid useless disk or database lookups.
- **HyperLogLog, Count-Min Sketch:** approximate counting of unique items and frequencies using tiny memory.
- **Fallacies of distributed computing:** the network is reliable; latency is zero; bandwidth is infinite; the network is secure; topology does not change; there is one administrator; transport cost is zero; the network is homogeneous. Every one is false.

2.7 Architecture styles

Style	Idea	Pros	Cons	When
Monolith	One deployable unit	Simple to build, test, deploy, debug; fast in-process calls; easy transactions	Scales as a whole, big codebase gets tangled, one bug can take down everything	Most startups and early products
Modular monolith	Monolith with strict internal module boundaries	Simplicity plus clean seams for later extraction	Needs discipline	The best default for a growing team
Microservices	Many small independently deployable services	Independent scaling and deployment, team autonomy, technology freedom, fault isolation	Network calls, distributed transactions, operational and monitoring overhead, versioning pain	Large orgs, clearly separable domains

Style	Idea	Pros	Cons	When
SOA	Coarser services with shared middleware (ESB)	Reuse	Heavy governance, central bottleneck	Legacy enterprise
Event-driven	Components react to events through a broker	Loose coupling, easy fan-out, scalable	Hard to trace and reason about, eventual consistency	Workflows, integrations, analytics
Serverless / FaaS	Functions run on demand	No server management, scales to zero, pay per use	Cold starts, vendor lock-in, limits, hard local testing	Spiky or low-volume workloads
Layered (n-tier)	Presentation, business, data layers	Familiar	Can become rigid	Traditional apps
Hexagonal / clean architecture	Core domain isolated from frameworks via ports and adapters	Testable, swappable infrastructure	More boilerplate	Long-lived domain-heavy software

Honest advice: do not start with microservices. Start with a modular monolith, extract a service only when a specific module needs independent scaling, a different technology, or a separate team. Many companies have gone back from microservices to monoliths after paying the complexity tax without needing the benefits.

Patterns worth knowing

- **API gateway:** single entry point for clients.
- **Backend for Frontend (BFF):** a tailored backend per client type (web, mobile).
- **Service discovery:** services find each other dynamically (Consul, etcd, Kubernetes DNS, Eureka).
- **Service mesh:** a sidecar proxy layer handling mTLS, retries, routing and telemetry (Istio, Linkerd, Cilium).
- **Sidecar, ambassador, adapter:** helper containers beside the main service.

- **Strangler fig:** migrate a legacy system piece by piece behind a facade.
- **CQRS (command query responsibility segregation):** separate models for writes and reads, so each can be optimized and scaled independently. Cost: eventual consistency between them and more moving parts.
- **Event sourcing:** store the sequence of events as the source of truth instead of current state. Gives a full audit trail, time travel and rebuildable views; costs schema evolution headaches, replay time and conceptual complexity.
- **Cell-based architecture:** split the fleet into isolated cells so a failure affects only a fraction of users.
- **Lambda and Kappa architectures:** batch plus streaming layers (Lambda) or streaming only (Kappa) for data processing.
- **Twelve-factor app:** config in the environment, stateless processes, logs as streams, disposable processes, dev/prod parity.

2.8 Observability

You cannot operate what you cannot see.

- **Metrics:** numeric time series (request rate, errors, latency, CPU). Use the **RED** method for services (Rate, Errors, Duration) and **USE** for resources (Utilization, Saturation, Errors). Google's **four golden signals:** latency, traffic, errors, saturation.
- **Logs:** structured (JSON), with correlation IDs so one request can be followed across services.
- **Traces:** the path of one request through many services, with timings (distributed tracing). Standard: **OpenTelemetry**.
- **Alerting:** alert on symptoms users feel (error rate, latency SLO burn), not on every internal cause. Too many alerts cause alert fatigue.

Free stack: Prometheus (metrics), Grafana (dashboards), Loki (logs), Tempo or Jaeger (traces), OpenTelemetry (instrumentation), Alertmanager, the ELK/EFK stack (Elasticsearch/OpenSearch, Logstash/Fluentd, Kibana), Sentry (self-hosted) for errors, Uptime Kuma for uptime checks.

2.9 Security basics in design

- **Authentication (who are you)** vs **authorization (what may you do):** OAuth 2.0, OpenID Connect, SAML, JWT, sessions, API keys. Free identity servers: Keycloak, Authentik, Ory, Zitadel.
- **TLS everywhere**, including between internal services (mTLS in a mesh). Free certificates: Let's Encrypt.
- **Least privilege, defense in depth, zero trust:** never trust a request just because it comes from the internal network.
- **Secrets management:** never in code or images. HashiCorp Vault (BSL) or OpenBao, SOPS, Kubernetes secrets with encryption, cloud secret managers.
- **Input validation, parameterized queries, output encoding** (stop injection and XSS), **CSRF protection**, **CORS** configured narrowly.
- **Encryption at rest and in transit**, key rotation, password hashing with bcrypt, scrypt or Argon2 (never plain SHA).
- **DDoS and abuse protection:** CDN or WAF, rate limiting, bot detection.
- **Audit logging** and data-retention policies; know your legal obligations (GDPR, India's DPDP Act, PCI-DSS for cards).

Part 3: Tools and Alternatives

For every building block: the common choice, alternatives, and free or open-source options. "Managed" means a cloud provider runs it for you (easy, costs money, some lock-in). "Self-hosted" means you run it (cheaper in cash, expensive in time).

Building block	Popular choices	Open-source / free alternatives	Notes
Reverse proxy / LB	AWS ALB/NLB, GCP LB, Azure LB	Nginx, HAProxy, Envoy, Traefik, Caddy, Apache httpd	Nginx or HAProxy cover most needs
API gateway	AWS API Gateway, Apigee, Azure APIM	Kong OSS, Apache APISIX, Tyk OSS, Envoy Gateway, KrakenD	Gateway adds a hop; do not stuff business logic in it

Building block	Popular choices	Open-source / free alternatives	Notes
CDN	CloudFront, Akamai, Fastly	Cloudflare free tier, jsDelivr, self-hosted Varnish/Nginx	Cloudflare free is generous for small projects
Relational DB	Oracle, SQL Server, Aurora, Cloud SQL	PostgreSQL, MySQL, MariaDB, SQLite	Postgres is the safe default
Distributed SQL	Spanner, Aurora DSQL	CockroachDB (free core limits), YugabyteDB, TiDB, Vitess (MySQL sharding)	For horizontal scale with SQL
Document DB	DynamoDB, Cosmos DB, Atlas	MongoDB Community, CouchDB, FerretDB	Mongo license is SSPL, not OSI-open
Wide-column	Bigtable, Keyspaces	Cassandra, ScyllaDB, HBase	Design tables per query
Cache / KV	ElastiCache, Memorystore	Redis (source-available now), Valkey, Memcached, KeyDB, Dragonfly	Valkey is the community fork under the Linux Foundation
Search	Algolia, Elastic Cloud	OpenSearch, Elasticsearch (AGPL option), Meilisearch, Typesense, Solr, Zinc	Meilisearch and Typesense are easy for small apps
Object storage	S3, GCS, Azure Blob	MinIO, Ceph, SeaweedFS, Garage	S3 API is the de facto standard
Message queue	SQS, Azure Service Bus, Google Pub/Sub	RabbitMQ, NATS, ActiveMQ, Redis Streams	Queues for tasks
Event streaming	Kinesis, Confluent Cloud, Event Hubs	Apache Kafka, Redpanda, Apache Pulsar, NATS JetStream	Logs for events and replay

Building block	Popular choices	Open-source / free alternatives	Notes
Stream processing	Kinesis Analytics, Dataflow	Apache Flink, Kafka Streams, Spark Structured Streaming, Benthos/Redpanda Connect	Flink for stateful streaming
Batch / big data	EMR, Databricks, BigQuery	Apache Spark, Hadoop, Trino, DuckDB	DuckDB for single-machine analytics
Orchestration	EKS, GKE, AKS	Kubernetes (k3s, k0s lightweight), Docker Swarm, Nomad	Kubernetes is powerful and complex; do not adopt it for a single app
Containers	Docker Desktop (paid at scale)	Docker Engine, Podman, containerd	
Service mesh	AWS App Mesh	Istio, Linkerd, Cilium, Consul	Only at real microservice scale
Service discovery / config	AWS Cloud Map	Consul, etcd, ZooKeeper, Kubernetes DNS	
Workflow engines	Step Functions	Temporal, Apache Airflow, Argo Workflows, Camunda, Conductor	Temporal excellent for sagas and long-running flows
CI/CD	GitHub Actions, CircleCI	GitHub Actions free tier, GitLab CI, Jenkins, Drone, Woodpecker, ArgoCD (GitOps)	
Infrastructure as code	CloudFormation, Pulumi	Terraform (BSL) / OpenTofu, Ansible, Pulumi OSS	OpenTofu is the open fork
Monitoring	Datadog, New Relic, CloudWatch	Prometheus, Grafana, VictoriaMetrics, Zabbix, Netdata	

Building block	Popular choices	Open-source / free alternatives	Notes
Logging	Splunk, Datadog Logs	Loki, ELK/OpenSearch, Graylog, Vector	
Tracing	Datadog APM, X-Ray	Jaeger, Tempo, Zipkin, SigNoz, OpenTelemetry	
Identity	Auth0, Okta, Cognito	Keycloak, Authentik, Ory, Zitadel, Supertokens	
Secrets	AWS Secrets Manager	Vault/OpenBao, SOPS, Infisical	
Feature flags	LaunchDarkly	Unleash, GrowthBook, Flagsmith, OpenFeature	
Hosting	AWS, GCP, Azure	Oracle Cloud free tier, Hetzner (cheap), self-hosted bare metal, Fly.io/Render free tiers	Cheap VPS goes a long way

Managed vs self-hosted trade-off: managed services buy you time and reliability and cost money and portability. Self-hosting buys control and lower cash cost and costs your attention (upgrades, backups, 3 a.m. incidents). A reasonable approach for small teams: self-host stateless things freely, but think hard before self-hosting your primary database.

Part 4: Which Language for Which Component

There is no best language, only the best fit for a component's constraints: latency, throughput, memory, concurrency model, safety, team skill, and ecosystem. Most real systems are **polyglot**: a few languages, each used where it shines. A small team should still limit itself to two or three, because every extra language multiplies hiring, tooling and operational cost.

4.1 The languages and where they win

Language	Strengths	Weaknesses	Best for
Go	Simple, fast compile, excellent concurrency (goroutines, channels), single static binary, great networking standard library, easy to operate	Garbage collector (small pauses), less expressive generics, verbose error handling	Network services, API servers, proxies, load balancers, CLI and infra tools, tunnelling, orchestration (Docker, Kubernetes, etcd, Prometheus are Go)
Rust	C-like speed with no GC, memory safety without a runtime, fearless concurrency, tiny footprint	Steep learning curve, slow compile, slower iteration	Proxies and gateways (Pingora, Linkerd), databases and storage engines (TiKV, SurrealDB), game and real-time engines, performance-critical parsers, WebAssembly, security-sensitive code
C / C++	Maximum control and performance, huge ecosystem, deterministic memory	Manual memory management, memory-safety bugs, slower development	OS kernels, database engines (MySQL, ClickHouse, ScyllaDB), game engines, high-frequency trading, embedded, codecs
Java / Kotlin (JVM)	Mature, enormous ecosystem, strong tooling, great JIT performance when warm, stable for large teams	Higher memory use, slower startup, verbose (Java)	Enterprise backends, big-data systems (Kafka, Cassandra, Elasticsearch, Spark, Flink are JVM), Android (Kotlin)

Language	Strengths	Weaknesses	Best for
C# / .NET	Excellent performance, great tooling, mature async model, cross-platform	Smaller open-source infra ecosystem than JVM or Go	Enterprise backends, Windows shops, game servers (Unity), FiveM/CitizenFX resources in C#
Python	Fastest to write, vast libraries, dominant in data and ML	Slow CPU-bound execution, GIL limits threads, packaging pain	Data pipelines, ML serving, automation, glue code, prototypes, admin tools, ETL
JavaScript / TypeScript (Node, Deno, Bun)	One language front to back, huge npm ecosystem, great for I/O-bound work, event loop	Single-threaded CPU limits, callback and dependency sprawl	API backends, real-time apps (WebSockets), BFF layers, serverless functions, tooling
Elixir / Erlang	Lightweight processes, built-in fault tolerance (let it crash), hot code reload, massive concurrency	Smaller talent pool, niche ecosystem	Chat, messaging, presence, telecom, WebSocket-heavy systems (WhatsApp, Discord used Erlang/Elixir)
Scala	Powerful type system, functional style on the JVM	Complexity, compile time	Data engineering (Spark), streaming
Lua	Tiny, embeddable, fast scripting	Small standard library	Scripting inside other programs: Nginx/OpenResty, Redis scripts, game modding, FiveM resources
SQL	Declarative, optimized by the engine	Not a general-purpose language	Anything involving relational data; push set-based work into the database
Bash / shell	Glue for operations	Fragile at scale	Deployment scripts, small automation

4.2 Mapping components to languages

Component	Recommended language(s)	Reasoning
Generic CRUD / business API	Go, Java/Kotlin, C#, TypeScript, Python (FastAPI/Django)	Choose by team skill; all are fine. Productivity matters more than raw speed here
High-throughput API or edge service	Go, Rust, Java (virtual threads), C#	Predictable latency, great concurrency
Reverse proxy, load balancer, API gateway	Go, Rust, C (Nginx, HAProxy), Lua (OpenResty plugins)	Network I/O, low latency, safety; use existing proxies before writing your own
Tunnelling / port-forwarding service	Go, Rust	Many long-lived TCP/UDP connections, cheap concurrency, static binaries on any OS
Real-time chat / presence / WebSocket fanout	Elixir, Go, Node.js, Rust	Hundreds of thousands of concurrent connections
Game server logic (real-time simulation)	C++, Rust, C#, Go; Lua/JS for scripting layers	Frame-time budgets; GC pauses matter. Scripting languages for modders and content
Game server management panel / daemon	Go for the node daemon and agents; TypeScript, Python or PHP for the web panel	Daemon manages processes and sockets; panel is CRUD plus UI
Database or storage engine	C, C++, Rust, Java (Cassandra-style), Go (CockroachDB, TiDB, etcd)	Memory layout and I/O control
Cache server	C, Rust, Go	Microsecond latencies, memory efficiency
Message broker	Java/Scala (Kafka), Erlang (RabbitMQ), Go (NATS), C++ (Redpanda)	Throughput and reliability, or concurrency model
Stream and batch data processing	Java/Scala (Flink, Spark), Python (PySpark), SQL	Mature distributed engines and ecosystem

Component	Recommended language(s)	Reasoning
ML training and inference	Python (PyTorch, TensorFlow, scikit-learn) with C++/CUDA underneath; Rust or C++ for edge inference	Ecosystem dominance
Search ranking / NLP / data science (e.g., resume and ATS ranking)	Python, with Rust or C++ extensions for hot paths	Libraries (spaCy, sentence-transformers, scikit-learn, pandas)
Background workers and cron jobs	Same language as the main app; Go or Python common	Reuse code and models
Frontend web	TypeScript with React, Vue, Svelte, or Solid	The browser speaks JavaScript (and WebAssembly)
Mobile	Kotlin (Android), Swift (iOS), or Flutter/Dart, React Native	Native for best experience, cross-platform to save effort
Infrastructure automation, CLIs, Kubernetes operators	Go, Python, Bash, Rust	Ecosystem and single binaries
Observability agents and collectors	Go, Rust	Low overhead, easy deployment
Embedded or IoT firmware	C, C++, Rust	Constrained memory
Security-critical parsing, crypto, sandboxing	Rust, Go	Memory safety
Serverless functions	TypeScript, Python, Go	Fast cold start (Go, Node) or ecosystem (Python)
Analytics and reporting queries	SQL (ClickHouse, DuckDB, Postgres)	Engines are optimized for it

4.3 How to decide

1. **Is the workload I/O-bound or CPU-bound?** I/O-bound (waiting on network and disk) is happy in Node, Python (async), Go, Java. CPU-bound (parsing, encoding, simulation) wants Go, Rust, C++, Java, C#.
2. **Is latency consistency critical?** Garbage-collected languages can pause. Go and modern JVM collectors keep pauses tiny, but Rust and C++ give hard guarantees.
3. **How many concurrent connections?** Tens of thousands per box is easy in Go, Elixir, Rust, Node. Thread-per-connection models do not scale there.
4. **What does the team know?** A team fluent in Python will ship a correct system faster than one learning Rust, and correctness beats micro-optimization.
5. **What is the ecosystem?** The library you would otherwise have to write (an ML library, a payment SDK) is often the deciding factor.
6. **Where is the hot path?** Write most of the system in a productive language and rewrite only the measured 5% in Go or Rust. Python plus a Rust extension is a common pattern.
7. **Deployment:** static single binaries (Go, Rust) make containers tiny and operations easy; JVM and Python need runtimes.

A sensible polyglot stack for a new product: TypeScript (frontend, and possibly the main API), Go or Python (backend services and workers), PostgreSQL plus Redis, SQL for analytics, Rust only if a measured hot spot demands it.

Part 5: Code

Every snippet below is a small, runnable version of a real building block. The Python building blocks in 5.1, 5.2, 5.4, 5.5 and 5.6 were executed and tested; the Go programs and the URL-shortener service are written to the same standard but were not run here. The Go snippets use only the standard library. They are teaching-sized: production versions would add persistence, metrics, configuration and more tests.

5.1 Token bucket rate limiter (Python)

Allows bursts up to `capacity`, then a steady `rate` per second.

```
import time, threading

class TokenBucket:
    def __init__(self, rate, capacity):
        self.rate, self.capacity = rate, capacity
        self.tokens, self.last = capacity, time.monotonic()
        self.lock = threading.Lock()

    def allow(self, cost=1):
        with self.lock:
            now = time.monotonic()
            self.tokens = min(self.capacity, self.tokens + (now - self.last)
* self.rate)
            self.last = now
            if self.tokens >= cost:
                self.tokens -= cost
                return True
            return False

# 5 requests/second steady, bursts up to 10
bucket = TokenBucket(rate=5, capacity=10)
print(sum(bucket.allow() for _ in range(20)))   # 10 allowed instantly
```

In production: one bucket per client key, stored in Redis with a Lua script so that read-modify-write is atomic across all app servers.

5.2 LRU cache with TTL (Python)

```
import time, threading
from collections import OrderedDict

class LRUCache:
    def __init__(self, capacity, ttl=None):
        self.capacity, self.ttl = capacity, ttl
        self.data = OrderedDict()
        self.lock = threading.Lock()

    def get(self, key, default=None):
        with self.lock:
            item = self.data.get(key)
            if item is None:
```

```
                return default
            value, expires = item
            if expires is not None and expires < time.monotonic():
                del self.data[key]            # expired
                return default
            self.data.move_to_end(key)        # mark as recently used
            return value

    def put(self, key, value):
        with self.lock:
            expires = time.monotonic() + self.ttl if self.ttl else None
            self.data[key] = (value, expires)
            self.data.move_to_end(key)
            if len(self.data) > self.capacity:
                self.data.popitem(last=False)  # evict least recently used
```

Both `get` and `put` are O(1): the ordered dictionary acts as a hash map plus a doubly linked list, which is the classic LRU design.

5.3 Cache-aside pattern with stampede protection (Python)

Only one caller rebuilds a missing key while the others wait for the result.

```
import threading

class CacheAside:
    def __init__(self, cache, loader):
        self.cache, self.loader = cache, loader       # loader(key) reads the
database
        self.locks, self.guard = {}, threading.Lock()

    def get(self, key):
        value = self.cache.get(key)
        if value is not None:
            return value                               # hit
        with self.guard:                               # one lock object per
key
            lock = self.locks.setdefault(key, threading.Lock())
        with lock:
            value = self.cache.get(key)                # re-check: someone
may have filled it
            if value is None:
                value = self.loader(key)               # only one thread
reaches the database
                self.cache.put(key, value)
        return value
```

5.4 Consistent hashing ring (Python)

```
import hashlib, bisect

class ConsistentHashRing:
    def __init__(self, nodes=(), vnodes=100):
        self.vnodes, self.ring, self.keys = vnodes, {}, []
        for n in nodes:
            self.add_node(n)

    @staticmethod
    def _hash(s):
        return int(hashlib.md5(s.encode()).hexdigest(), 16)

    def add_node(self, node):
        for i in range(self.vnodes):               # virtual nodes even out
the load
```

```
            h = self._hash(f"{node}#{i}")
            self.ring[h] = node
            bisect.insort(self.keys, h)

    def remove_node(self, node):
        for i in range(self.vnodes):
            h = self._hash(f"{node}#{i}")
            del self.ring[h]
            self.keys.remove(h)

    def get_node(self, key):
        i = bisect.bisect(self.keys, self._hash(key)) % len(self.keys)
        return self.ring[self.keys[i]]

ring = ConsistentHashRing(["n1", "n2", "n3"])
before = {f"k{i}": ring.get_node(f"k{i}") for i in range(10000)}
ring.add_node("n4")
moved = sum(before[k] != ring.get_node(k) for k in before) / 10000
print(moved)   # about 0.25 (ideal 1/4), versus about 0.75 with hash(key) % N
```

A test run moved 27.6% of the keys when a fourth node joined, close to the theoretical 1/N.

5.5 Bloom filter (Python)

A test run with 1,000 items and a 1% target gave a measured false-positive rate of 0.93%.

```
import math, hashlib

class BloomFilter:
    def __init__(self, expected_items, fp_rate=0.01):
        self.m = max(8, int(-expected_items * math.log(fp_rate) /
(math.log(2) ** 2)))
        self.k = max(1, round(self.m / expected_items * math.log(2)))
        self.bits = bytearray((self.m + 7) // 8)

    def _positions(self, item):
        h1 = int(hashlib.sha256(item.encode()).hexdigest(), 16)
        h2 = int(hashlib.md5(item.encode()).hexdigest(), 16) | 1
        return [(h1 + i * h2) % self.m for i in range(self.k)]   # double
hashing

    def add(self, item):
        for p in self._positions(item):
            self.bits[p // 8] |= 1 << (p % 8)

    def might_contain(self, item):
        return all(self.bits[p // 8] & (1 << (p % 8)) for p in
self._positions(item))
```

Use: put it in front of the database. If `might_contain(id)` is False, answer "not found" without touching storage (this defeats cache penetration).

5.6 Retry with backoff and a circuit breaker (Python)

```
import time, random, threading

class CircuitOpenError(Exception):
    pass

class CircuitBreaker:
    def __init__(self, failure_threshold=5, reset_timeout=30.0):
        self.threshold, self.reset_timeout = failure_threshold, reset_timeout
        self.failures, self.opened_at, self.state = 0, None, "closed"
        self.lock = threading.Lock()

    def call(self, fn, *args, **kwargs):
        with self.lock:
```

```
            if self.state == "open":
                if time.monotonic() - self.opened_at >= self.reset_timeout:
                    self.state = "half_open"           # allow a trial call
                else:
                    raise CircuitOpenError("failing fast")
        try:
            result = fn(*args, **kwargs)
        except Exception:
            with self.lock:
                self.failures += 1
                if self.state == "half_open" or self.failures >=
self.threshold:
                    self.state, self.opened_at = "open", time.monotonic()
            raise
        with self.lock:
            self.failures, self.state = 0, "closed"
        return result

def retry(fn, attempts=4, base=0.1, cap=2.0):
    for attempt in range(attempts):
        try:
            return fn()
        except CircuitOpenError:
            raise                                       # do not hammer an
open circuit
        except Exception:
            if attempt == attempts - 1:
                raise
            time.sleep(random.uniform(0, min(cap, base * 2 ** attempt)))  #
full jitter

breaker = CircuitBreaker(failure_threshold=3, reset_timeout=10)
# result = retry(lambda: breaker.call(requests.get,
"https://api.example.com", timeout=2))
```

5.7 Round-robin reverse proxy with health checks (Go)

A minimal L7 load balancer. Save as `lb.go` and run `go run lb.go`.

```
package main

import (
        "log"
        "net/http"
        "net/http/httputil"
        "net/url"
        "sync"
        "sync/atomic"
        "time"
)

type backend struct {
        proxy *httputil.ReverseProxy
        url   *url.URL
        alive atomic.Bool
}

type pool struct {
        backends []*backend
        next     atomic.Uint64
}

// pick returns the next alive backend, or nil when all are down.
func (p *pool) pick() *backend {
        n := uint64(len(p.backends))
        for i := uint64(0); i < n; i++ {
                b := p.backends[p.next.Add(1)%n]
                if b.alive.Load() {
                        return b
                }
        }
        return nil
}

func (p *pool) healthLoop(interval time.Duration) {
        client := http.Client{Timeout: 2 * time.Second}
        for {
                var wg sync.WaitGroup
                for _, b := range p.backends {
                        wg.Add(1)
                        go func(b *backend) {
                                defer wg.Done()
```

```
                                resp, err := client.Get(b.url.String() +
"/health")
                                ok := err == nil && resp.StatusCode == 200
                                if resp != nil {
                                        resp.Body.Close()
                                }
                                b.alive.Store(ok)
                        }(b)
                }
                wg.Wait()
                time.Sleep(interval)
        }
}

func main() {
        p := &pool{}
        for _, raw := range []string{"http://localhost:9001",
"http://localhost:9002", "http://localhost:9003"} {
                u, _ := url.Parse(raw)
                b := &backend{url: u, proxy:
httputil.NewSingleHostReverseProxy(u)}
                b.alive.Store(true)
                p.backends = append(p.backends, b)
        }
        go p.healthLoop(5 * time.Second)

        http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
                b := p.pick()
                if b == nil {
                        http.Error(w, "no healthy backend",
http.StatusServiceUnavailable)
                        return
                }
                b.proxy.ServeHTTP(w, r)
        })
        log.Println("load balancer on :8080")
        log.Fatal(http.ListenAndServe(":8080", nil))
}
```

This shows the three ideas in a load balancer: an algorithm (round robin via an atomic counter), health checks (a background loop), and failing safe (503 when nothing is healthy). Swapping `pick` for least-connections or consistent hashing changes the

strategy without touching the rest.

5.8 Worker pool with a bounded queue and backpressure (Go)

```
package main

import (
        "errors"
        "fmt"
        "sync"
)

var ErrBusy = errors.New("queue full, try again later")

type Job func()

type Pool struct {
        jobs chan Job
        wg   sync.WaitGroup
}

func NewPool(workers, queueSize int) *Pool {
        p := &Pool{jobs: make(chan Job, queueSize)}   // bounded buffer =
backpressure
        for i := 0; i < workers; i++ {
                p.wg.Add(1)
                go func() {
                        defer p.wg.Done()
                        for j := range p.jobs {
                                j()
                        }
                }()
        }
        return p
}

// Submit never blocks: when the queue is full it sheds load instead.
func (p *Pool) Submit(j Job) error {
        select {
        case p.jobs <- j:
                return nil
        default:
```

```
                return ErrBusy
        }
}

func (p *Pool) Shutdown() { close(p.jobs); p.wg.Wait() }

func main() {
        p := NewPool(4, 100)
        var mu sync.Mutex
        done := 0
        for i := 0; i < 50; i++ {
                if err := p.Submit(func() { mu.Lock(); done++; mu.Unlock()
}); err != nil {
                        fmt.Println("rejected:", err)   // in an API, return
HTTP 429 or 503 here
                }
        }
        p.Shutdown()
        fmt.Println("completed:", done)
}
```

5.9 A URL shortener: the classic end-to-end design

A good first complete design because it touches every concept: ID generation, storage, caching, redirects, scale.

Requirements. Create a short link, redirect fast, optional expiry and custom alias, basic click analytics. Assume 100 million new links per month and a 100:1 read-to-write ratio.

Estimates. Writes: about 40 per second. Reads: about 4,000 per second average, 10-20k at peak. Storage: 100M links x 12 months x 5 years x ~500 bytes = about 3 TB. Small enough for one well-indexed database, and with a cache hit rate above 90% the database barely notices the reads.

Short-code generation options:

Approach	Pros	Cons
Hash the URL (MD5/SHA, take 7 chars)	Same URL gives same code, stateless	Collisions need handling; codes are guessable

Approach	Pros	Cons
Random 7-char base62 string	Not guessable, simple	Collision check needed on insert
Auto-increment ID encoded in base62	No collisions, compact	Sequential and guessable (can scramble with a bijective permutation); single counter is a bottleneck
Pre-allocated ID ranges per app server (Snowflake-style or a range service)	No contention, scales out	Slightly more infrastructure

62^7 is about 3.5 trillion codes, which is plenty.

```
import secrets, string
ALPHABET = string.digits + string.ascii_letters      # 62 symbols

def base62(n: int) -> str:
    if n == 0:
        return ALPHABET[0]
    out = []
    while n:
        n, r = divmod(n, 62)
        out.append(ALPHABET[r])
    return "".join(reversed(out))

def random_code(length=7) -> str:
    return "".join(secrets.choice(ALPHABET) for _ in range(length))
```

A minimal service (FastAPI with SQLite for clarity; swap in PostgreSQL and Redis for real use):

```
# pip install fastapi uvicorn
import sqlite3, time
from fastapi import FastAPI, HTTPException
from fastapi.responses import RedirectResponse
from pydantic import BaseModel, HttpUrl

app = FastAPI()
db = sqlite3.connect("links.db", check_same_thread=False)
db.execute("CREATE TABLE IF NOT EXISTS links (code TEXT PRIMARY KEY, url TEXT
```

```
NOT NULL, expires REAL)")
cache = LRUCache(capacity=10_000, ttl=300)           # from section 5.2

class NewLink(BaseModel):
    url: HttpUrl
    ttl_seconds: int | None = None

@app.post("/links")
def create(body: NewLink):
    expires = time.time() + body.ttl_seconds if body.ttl_seconds else None
    for _ in range(5):                                # retry on the rare
collision
        code = random_code()
        try:
            db.execute("INSERT INTO links VALUES (?,?,?)", (code,
str(body.url), expires))
            db.commit()
            return {"short": f"https://sho.rt/{code}"}
        except sqlite3.IntegrityError:
            continue
    raise HTTPException(500, "could not allocate code")

@app.get("/{code}")
def redirect(code: str):
    hit = cache.get(code)                             # cache-aside read path
    if hit is None:
        row = db.execute("SELECT url, expires FROM links WHERE code=?",
(code,)).fetchone()
        if not row or (row[1] and row[1] < time.time()):
            raise HTTPException(404)
        hit = row[0]
        cache.put(code, hit)
    return RedirectResponse(hit, status_code=302)     # 302 lets you count
clicks; 301 gets cached by browsers
```

Scaling path: add Redis as a shared cache and put it behind the app servers; run several stateless app instances behind a load balancer; add read replicas; move click counting off the request path into a queue and batch it into an analytics store; shard by code only if one database truly runs out of room; put a CDN or edge cache in front for hot links.

5.10 Worked design: a tunnelling service (ngrok-style)

A second full design, because it exercises long-lived connections rather than request/response traffic.

Goal. A user runs a small client on a laptop behind a NAT. The service gives them a public address (for example `abc.tunnel.example.com`) and forwards incoming traffic to the laptop, with no port forwarding.

Core idea. The client opens an **outbound** connection to the server (outbound is allowed through NATs and firewalls) and keeps it open. The server, receiving a public request, sends it back down that existing connection.

Components and trade-offs:

Component	Choice	Why and what it costs
Edge / ingress	Go or Rust TCP/HTTP server on a public IP, wildcard DNS (`*.tunnel.example.com`) and wildcard TLS certificate	Routes by the Host header (HTTP) or SNI (TLS) to the right tunnel. Wildcard certs need DNS-based validation
Control channel	One persistent TCP or WebSocket connection per client, with heartbeats	Cheap to hold open with goroutines or async tasks; must detect dead clients quickly
Multiplexing	Many logical streams over one connection (yamux, smux, HTTP/2, or QUIC)	One connection serves many visitors; head-of-line blocking is the trade-off (QUIC avoids it)
Registry	Map from subdomain to the edge node holding that tunnel	In memory on one node; Redis or etcd when several edge nodes exist
Multi-node routing	Consistent hashing on subdomain, or a shared registry plus forwarding between edges	Without it, a visitor can reach an edge that does not hold the tunnel
Auth and quotas	Tokens in Postgres, rate limits and bandwidth caps enforced at the edge	Freemium tiers become quota differences

Component	Choice	Why and what it costs
Raw TCP/UDP tunnels	Allocate a public port per tunnel from a pool; UDP needs framing over the stream or datagrams over QUIC	Port exhaustion and abuse are the main problems
Abuse protection	Rate limits, bandwidth caps, blocklists, phishing detection on public HTTP tunnels	Public tunnels are routinely abused; plan for it from day one
Observability	Per-tunnel byte and connection counters in Prometheus, structured logs	Also feeds billing

Language choice: Go is the pragmatic default (excellent networking library, cheap concurrency, static binaries for every client OS). Rust is a strong alternative if you need to squeeze memory and latency at very large scale.

Free building blocks: frp, rathole, chisel, bore and sish are open-source tunnels you can study or deploy; Cloudflare Tunnel and Tailscale Funnel are free hosted options; WireGuard is the standard for VPN-style overlays.

Part 6: Mistakes, Checklist and Roadmap

6.1 Common mistakes

- **Designing for scale you do not have.** Ten users do not need Kafka, Kubernetes and five databases. Build the simplest thing that meets the requirements and scale when measurements say so.
- **Skipping requirements and estimation.** Without numbers, you cannot justify any choice.
- **Ignoring failure.** Every dependency call needs a timeout, a retry policy and a fallback.
- **Treating a cache as a source of truth.**
- **Choosing a poor shard key** (monotonic timestamps, low-cardinality fields).
- **Distributed transactions by default.** Redesign boundaries so that most operations stay inside one service or database.
- **No idempotency**, so retries double-charge or double-send.

- **Shared database between microservices**, which silently turns them back into a distributed monolith.
- **No observability**, so you find out about outages from users.
- **Untested backups and untested failover.**
- **Optimizing before measuring.** Profile, then fix the actual bottleneck.
- **Adopting a tool because it is fashionable.** Operational cost is real.

6.2 Design checklist

- [ ] Functional and non-functional requirements written down
- [ ] Traffic, storage and bandwidth estimated; read/write ratio known
- [ ] API and data model defined; access patterns listed
- [ ] Consistency requirement chosen per piece of data (strong or eventual)
- [ ] Single points of failure identified and removed or accepted
- [ ] Caching, CDN and read replicas considered for read-heavy paths
- [ ] Queues used for slow or spiky work; consumers idempotent
- [ ] Rate limiting, timeouts, retries with backoff, circuit breakers in place
- [ ] Authentication, authorization, TLS, secrets handled
- [ ] Metrics, logs, traces, alerts and SLOs defined
- [ ] Backup, restore and disaster-recovery plan tested
- [ ] Deployment: CI/CD, rollbacks, staged rollouts, feature flags
- [ ] Cost estimated; managed vs self-hosted decided deliberately
- [ ] Trade-offs and rejected alternatives written down

6.3 Learning roadmap: zero to hero

Stage	What to study	What to build	Time (rough)
1. Foundations	Networking basics (TCP, HTTP, DNS, TLS), operating systems, SQL, one backend language	A CRUD API with PostgreSQL, deployed on a cheap VPS behind Nginx	4-6 weeks

Stage	What to study	What to build	Time (rough)
2. Single-node mastery	Indexing, transactions, query plans, caching, concurrency, profiling	Add Redis caching and load-test it (k6, wrk, Locust); read the query plans	3-4 weeks
3. Scaling basics	Load balancing, replication, queues, CDNs, statelessness	Run 3 app instances behind HAProxy or Nginx with a replica database and a job queue	4 weeks
4. Distributed theory	CAP/PACELC, consistency models, consensus (Raft), partitioning, clocks	Implement consistent hashing and a toy Raft (the MIT 6.5840 labs are the best guide)	6-8 weeks
5. Resilience and operations	Observability, SLOs, chaos testing, Kubernetes, IaC	Add Prometheus and Grafana; kill nodes on purpose; write a runbook	4-6 weeks
6. Architecture and case studies	Microservices trade-offs, event sourcing, CQRS, sagas, data pipelines	Design and build 3 systems end to end (URL shortener, chat, feed or tunnelling service)	6-8 weeks
7. Hero level	Read real postmortems and papers; operate something real with users	Run a real service, measure it, break it, and fix it	Ongoing

6.4 Where to learn more

- **Books:** *Designing Data-Intensive Applications* (Martin Kleppmann), the single most important book on this subject; *System Design Interview* vols. 1 and 2 (Alex Xu); *Site Reliability Engineering* (Google, free online); *Database Internals* (Alex Petrov); *Building Microservices* (Sam Newman); *Release It!* (Michael Nygard) for resilience patterns.
- **Papers (all free):** Google File System, MapReduce, Bigtable, Spanner, Dynamo (Amazon), Kafka, Raft ("In Search of an Understandable Consensus Algorithm"), Paxos Made Simple, the Chubby lock service, Borg, Facebook's TAO and Memcache papers.

- **Free courses and repositories:** MIT 6.5840 Distributed Systems (lectures and labs online), the `system-design-primer` repository on GitHub, ByteByteGo and High Scalability blogs, engineering blogs of Netflix, Uber, Discord, Cloudflare, Stripe and Shopify.
- **Practice:** load-test your own projects, read public postmortems, and design a system a week on paper, estimating numbers and listing trade-offs each time.

6.5 The one-paragraph summary

Start simple: one app, one database. Measure. When a bottleneck appears, apply the smallest tool that removes it: a cache for hot reads, replicas for read scale, a queue for slow or spiky work, a CDN for static content, partitioning only when data or writes outgrow a node. Decide consistency per piece of data, assume every dependency will fail, make operations idempotent, and watch everything. Choose boring, proven, preferably open-source tools, a productive language for most code, a fast language for measured hot paths, and write down the trade-offs every time. That discipline is what the word *architect* really means.

Part 7: Deep Dive: Storage and Database Internals

Parts 1 to 6 told you what each building block does. This part opens the box. Knowing how a database actually stores, logs, locks and plans queries is what separates someone who picks tools from someone who can predict how they behave under load and fix them when they do not.

7.1 The storage hierarchy and why layout matters

Databases are designed around one fact: **reading a byte from disk costs almost the same as reading a whole block**, so data is grouped into fixed-size **pages** (PostgreSQL uses 8 KB, InnoDB uses 16 KB, most OS file systems 4 KB). Everything else follows:

- Sequential access is far faster than random access, on SSD as well as on HDD, because of read-ahead, page-cache hits and fewer I/O operations.
- A **buffer pool** (page cache) keeps hot pages in RAM. The single most valuable tuning lever is making the working set fit in memory.

- Writes go first to a sequential log (the WAL), and the slow page writes happen later in the background.

Row store vs column store

	Row-oriented (PostgreSQL, MySQL, Oracle)	Column-oriented (ClickHouse, Redshift, BigQuery, Parquet files, DuckDB)
Layout	All columns of a row stored together	All values of a column stored together
Good at	Fetch or update a whole row (OLTP)	Scan a few columns over billions of rows (OLAP)
Compression	Moderate	Excellent (similar values sit together, run-length and dictionary encoding)
Weak at	Wide scans of one column	Single-row inserts and updates

OLTP (online transaction processing: many small reads and writes) and **OLAP** (online analytical processing: few huge scans and aggregations) need different engines. Running heavy analytics on your production OLTP database is a classic outage cause. The standard fix is to replicate data into an analytical store (ClickHouse, DuckDB on Parquet, BigQuery) and query there. Systems that do both in one engine are called **HTAP** (TiDB, SingleStore).

7.2 B-tree indexes in depth

A B+ tree keeps keys sorted in pages. Each internal page holds hundreds of keys pointing to child pages, so the tree is very shallow: with a fan-out of about 500, three levels address 125 million rows and four levels address 62 billion. A lookup costs 3 or 4 page reads, and the top levels are always cached, so usually only the leaf read touches disk.

Things that follow from this design:

- **Clustered vs heap storage.** In InnoDB the table *is* a B+ tree ordered by primary key (a clustered index); secondary indexes store the primary key as their pointer, so a secondary lookup is two tree searches. In PostgreSQL the table is an unordered **heap** and every index points to the physical row location, so updates that move a row can force index updates (mitigated by HOT updates when no indexed column changes).

- **Primary key choice matters.** Random UUIDv4 keys insert at random places in the tree, causing page splits, fragmentation, poor cache locality and higher write amplification. Time-ordered IDs (UUIDv7, ULID, Snowflake IDs, auto-increment) append at the right edge of the index and are much friendlier. Auto-increment leaks business volume and is guessable; time-ordered IDs are a good compromise.
- **Fill factor and page splits.** A full page that receives an insert splits in two, costing extra writes. Leaving free space (fill factor below 100) helps update-heavy tables.
- **Composite indexes** follow the **leftmost-prefix rule**: an index on (a, b, c) serves queries on a, on (a, b) and on (a, b, c), but not on b alone. Put equality columns first, then the range column, and the sort column last when you can.
- **Covering indexes** include every column a query needs (INCLUDE columns in PostgreSQL and SQL Server) so the engine answers from the index alone (index-only scan) without touching the table.
- **Partial indexes** index only some rows (`WHERE status = 'pending'`), keeping them small and fast.
- **Expression indexes** index a computed value (`lower(email)`), required when queries filter on that expression.
- **Selectivity.** An index on a column with 2 distinct values (a boolean) rarely helps; the planner will scan the table instead. Indexes pay off when they cut the result to a small fraction of rows.
- **Cost of indexes.** Every index slows every insert, update and delete and consumes memory. Remove unused ones (check `pg_stat_user_indexes`).
- **Other index types.** PostgreSQL has GIN (arrays, JSONB, full-text), GiST and SP-GiST (geometry, ranges), BRIN (huge naturally ordered tables, tiny index), hash. Vector indexes (HNSW, IVF, as in pgvector) support similarity search for embeddings.

7.3 Write-ahead logging and crash recovery

The core durability trick: **never modify the data files before the change is safely recorded in a sequential log.**

1. A transaction changes pages in memory in the buffer pool.
2. It appends a description of the change to the **write-ahead log (WAL)** and flushes (fsync) the log to disk before commit returns.
3. The dirty data pages are written to their files later, in the background, at **checkpoints**.

4. After a crash, the database replays the WAL from the last checkpoint (redo) and rolls back unfinished transactions (undo). This family of algorithms is called ARIES.

Why it works: appending to a log is one sequential write; the scattered page writes can be deferred and batched.

Trade-offs you control:

- **fsync on every commit** is the safest and the slowest setting. **Group commit** lets many concurrent transactions share one fsync.
- PostgreSQL's `synchronous_commit = off` and MySQL's `innodb_flush_log_at_trx_commit = 2` trade up to a fraction of a second of committed transactions (in a crash) for much higher throughput. The database stays consistent; you merely may lose the most recent commits.
- **The same log powers replication and point-in-time recovery (PITR).** Streaming replicas replay the primary's WAL; backups plus archived WAL let you restore to any second. This is why "log-based" designs recur everywhere (Kafka, event sourcing, Raft).

7.4 MVCC: how readers avoid blocking writers

Multi-version concurrency control keeps several versions of a row so that readers see a consistent **snapshot** without taking locks that block writers.

- **PostgreSQL:** an update writes a *new* row version and marks the old one as ended; each row version carries the transaction IDs that created and removed it, and a snapshot decides which versions are visible. Dead versions pile up until **VACUUM** reclaims them. Neglected vacuuming causes **table bloat** and slow queries; long-running transactions prevent cleanup of everything newer than their snapshot. Monitor `idle in transaction` sessions and the age of the oldest transaction.
- **MySQL InnoDB and Oracle:** the latest version stays in the table and older versions are rebuilt from an **undo log**. Long transactions make the undo log huge.

Isolation anomalies beyond the textbook:

Anomaly	Example	Prevented by
Lost update	Two clients read balance 100, both add 10, both write 110	Atomic update (`SET balance = balance + 10`), `SELECT ... FOR UPDATE` , optimistic version column

Anomaly	Example	Prevented by
Write skew	Two on-call doctors each check that another doctor is on call, then both go off call	Serializable isolation, or explicit locks on the checked rows
Phantom	A range query returns different rows when repeated because another transaction inserted a matching row	Serializable, or range locks

Repeatable read in PostgreSQL is actually **snapshot isolation**, which allows write skew. **Serializable** in PostgreSQL uses SSI (serializable snapshot isolation) and aborts conflicting transactions with a serialization error, so your application must be ready to **retry**.

7.5 Locking, deadlocks and concurrency control patterns

- **Pessimistic locking:** lock first, then work. `SELECT ... FOR UPDATE` locks rows until commit. Safe under high contention, but holds locks and can deadlock.
- **Optimistic locking:** read a version number, do the work, then update `WHERE id = ? AND version = ?`; zero rows updated means someone else won, so retry. Best when conflicts are rare and transactions are long (user edits).
- **Deadlock:** transaction A holds row 1 and wants row 2 while B holds row 2 and wants row 1. Databases detect this and kill one transaction. **Prevention:** always lock rows in a consistent order, keep transactions short, and never wait for a human or a network call inside a transaction.
- **Job-queue pattern in SQL:** many workers can safely pull jobs without blocking each other:

```
BEGIN;
SELECT id, payload FROM jobs
WHERE status = 'pending'
ORDER BY id
LIMIT 10
FOR UPDATE SKIP LOCKED;       -- skip rows other workers already hold
-- process them, then:
UPDATE jobs SET status = 'done' WHERE id = ANY(:ids);
COMMIT;
```

- **Advisory locks** (`pg_advisory_lock`) give you application-defined named locks inside the database, handy for singletons such as one cron runner.
- **Unique constraints are your friend.** The cheapest correct way to make an operation idempotent is a unique constraint on an idempotency key plus `INSERT ... ON CONFLICT DO NOTHING`.

7.6 How queries actually run

A query goes through **parse, rewrite, plan, execute**. The **planner** chooses among many possible plans using **statistics** about table sizes and value distributions. Stale statistics (run `ANALYZE`) are a top cause of sudden slow queries.

Scan methods: sequential scan (read all pages; best when you need a large fraction of the table), index scan (follow the index, then fetch rows), index-only scan, bitmap scan (combine several indexes).

Join algorithms:

Algorithm	How	Best when
Nested loop	For each row of A, look up matching rows in B (via index)	Small outer set, indexed inner side
Hash join	Build a hash table of the smaller side, probe with the other	Large unsorted inputs, equality joins
Merge join	Walk two sorted inputs together	Inputs already sorted (by an index)

Reading `EXPLAIN (ANALYZE, BUFFERS)` : compare estimated rows with actual rows; a large mismatch points to bad statistics. Look for sequential scans on big tables, sorts spilling to disk, and nested loops with huge outer sets.

Frequent causes of slow queries:

- Missing or unusable index (function on the column, type mismatch, leading wildcard `LIKE '%x'`).
- **N+1 queries:** one query for a list, then one per item. Fix with a join, `IN (...)` batching or eager loading in your ORM.
- `SELECT *` pulling wide rows or defeating index-only scans.

- **Deep offset pagination** (`OFFSET 1000000` reads and discards a million rows). Use keyset pagination: `WHERE (created_at, id) < (:last_ts, :last_id) ORDER BY created_at DESC, id DESC LIMIT 50`.
- `COUNT(*)` on huge tables (use approximate counts or maintained counters).
- Unbounded `IN` lists, `OR` conditions that defeat indexes, `DISTINCT` and `ORDER BY` on large sets.
- Lock waits that look like slowness (check `pg_stat_activity`).

7.7 LSM trees in depth

LSM engines (RocksDB, LevelDB, Cassandra, ScyllaDB, HBase, ClickHouse parts, many embedded stores) optimize for **write throughput**.

1. A write appends to the **WAL** and inserts into an in-memory sorted **memtable**.
2. A full memtable is flushed as an immutable sorted file, an **SSTable**.
3. Background **compaction** merges SSTables, discarding overwritten and deleted entries.
4. A read checks the memtable, then SSTables from newest to oldest, using **Bloom filters** to skip files that cannot contain the key and sparse indexes to jump within files.

Deletes write a **tombstone** marker; the data disappears only when compaction reaches it. Heavy deletes and TTL-based expiry can therefore make reads slow, a well-known Cassandra pitfall.

The three amplifications you trade between:

- **Write amplification:** bytes written to disk per byte of user data (compaction rewrites data repeatedly).
- **Read amplification:** files and pages touched per read.
- **Space amplification:** disk used per byte of live data.

Compaction strategy	Write amp	Read amp	Space amp	Good for
Size-tiered	Low	High	High (temporary doubling)	Write-heavy, few reads
Leveled	High	Low	Low	Read-heavy, point lookups

Compaction strategy	Write amp	Read amp	Space amp	Good for
Time-window (TWCS) / FIFO	Low	Low for recent data	Low	Time-series with TTL

B-trees do small random in-place writes; LSM trees do large sequential writes plus background merging. SSDs are kinder to LSM patterns, but compaction still burns I/O and can cause latency spikes if not provisioned for.

7.8 Connection management and pooling

Each PostgreSQL connection is an operating-system process using several MB of memory, and many hundreds of them thrash the scheduler and lock structures. MySQL uses threads and tolerates more, but the principle holds: **a database does its best work with a modest number of busy connections.**

- **Connection pooling** keeps a small set of persistent connections and lends them to requests. Application-side pools (HikariCP, pgx pool, SQLAlchemy pool) and server-side poolers (**PgBouncer**, PgCat, ProxySQL, RDS Proxy, Odyssey) both exist. With hundreds of app instances, a central pooler is essential.
- **Pool sizing rule of thumb:** connections about (CPU cores x 2) + number of disk spindles on the database server. Small pools (10 to 50 per database) usually outperform large ones. Excess requests should queue in the pool, not pile onto the database.
- **PgBouncer modes:** session (one server connection per client session), transaction (connection borrowed per transaction; the best for scale, but session features such as prepared statements and advisory locks need care), statement (rarely used).
- **Serverless and many short-lived processes** cause connection storms; use an external pooler or a data API.

7.9 Online schema migrations

Changing a schema on a live system without downtime follows the **expand and contract** pattern:

1. **Expand:** add the new column, table or index in a backward-compatible way (nullable column, new table, `CREATE INDEX CONCURRENTLY`).

2. **Dual write:** the application writes to both old and new.
3. **Backfill:** copy old data to the new structure in small batches (never one giant transaction).
4. **Switch reads** to the new structure, verify.
5. **Contract:** stop writing the old structure, then drop it in a later release.

Dangers: an `ALTER TABLE` that rewrites the table or takes an exclusive lock blocks everything (always check the specific operation for your database version); adding a column with a default was a table rewrite in old PostgreSQL versions and is cheap in current ones; renaming a column breaks old application versions during a rolling deploy, so add-copy-drop instead. Tools: gh-ost and pt-online-schema-change (MySQL), pg_repack, pgroll, Flyway, Liquibase, Alembic, Atlas, Sqitch.

7.10 PostgreSQL operational essentials

- **Declarative partitioning** (range, list, hash) splits a huge table into child tables. Queries that filter on the partition key touch only relevant partitions (**partition pruning**), and dropping an old partition is instant, which makes retention cheap. Time-series and logs are the classic fit.
- **Replication:** physical **streaming replication** (byte-for-byte WAL shipping; replicas are read-only copies) and **logical replication** (row changes by table; allows cross-version upgrades and selective copying). Replicas can serve reads, with replication lag as the staleness cost.
- **High availability:** Patroni (with etcd or Consul), repmgr, pg_auto_failover, or managed offerings. Automated failover needs a quorum-backed decision to avoid split brain.
- **Backups:** base backup plus continuous WAL archiving for PITR using pgBackRest or WAL-G; logical dumps (`pg_dump`) are portable but slow at scale. Test restores.
- **Useful extensions:** PostGIS, pgvector, TimescaleDB, pg_stat_statements (find your slowest queries), pg_partman, Citus (distributed Postgres), pg_cron.
- **Autovacuum tuning** and monitoring bloat prevent the slow decay of busy tables.

7.11 MySQL / MariaDB operational essentials

- **InnoDB** is the engine to use: clustered primary key, row-level locks, MVCC with undo logs, redo log for durability, doublewrite buffer against torn pages.

- **Replication** is based on the **binlog**; row-based format is the safest. **Group Replication / InnoDB Cluster** and Galera (MariaDB) provide multi-primary or consensus-style clusters. Semi-sync replication limits data loss.
- **Scaling out:** ProxySQL for routing and pooling; **Vitess** (born at YouTube) for horizontal sharding of MySQL, used by large companies; PlanetScale is a managed Vitess offering.

7.12 Redis in depth

- **Execution model:** commands run on a **single thread**, which makes each command atomic and the model simple; network I/O can use extra threads in newer versions. One slow command (for example `KEYS *` or a huge `SMEMBERS`) blocks everyone, so use `SCAN` and keep values small.
- **Data structures:** strings, hashes, lists, sets, **sorted sets** (leaderboards, rate-limit windows, delayed queues), bitmaps, **HyperLogLog** (approximate unique counts), geospatial indexes, **streams** (log with consumer groups), pub/sub, plus modules.
- **Persistence: RDB** snapshots (compact, can lose minutes), **AOF** append-only log (fsync every second or every write; larger, slower), or both. Redis is a cache first; if it is your only copy of data, understand what a crash loses.
- **Eviction:** `maxmemory` plus a policy (`allkeys-lru`, `allkeys-lfu`, `volatile-ttl`, `noeviction`, and so on). A cache should evict; a data store should not.
- **High availability:** Replication plus **Sentinel** (monitoring and automatic failover for a single shard), or **Redis Cluster**, which splits keys over **16,384 hash slots** across nodes. Multi-key operations only work within one slot (use hash tags `{user42}` to colocate keys).
- **Patterns:** cache-aside, session store, rate limiter (INCR with TTL, or sorted-set sliding window, or Lua token bucket), leaderboard (ZADD, ZREVRANGE), distributed lock (`SET key value NX PX 30000`; for correctness-critical locking add fencing tokens, because lock expiry and clock pauses can break mutual exclusion), job queues (lists or streams), real-time counters.
- **Problems: hot keys** (one key hammered; replicate or add a local cache), **big keys** (megabyte-sized values block the event loop and network), memory fragmentation, thundering herds after mass expiry (jitter TTLs).
- **Alternatives:** Valkey (open fork), KeyDB, Dragonfly (multi-threaded, Redis-compatible), Memcached (simpler, multi-threaded, strings only).

7.13 Kafka in depth

- **Topics are split into partitions**, and each partition is an append-only, ordered log stored as segment files. **Ordering is guaranteed only within a partition**, so choose the message key to keep related events (all events of one user or order) in the same partition.
- **Replication:** each partition has a leader and followers; the **in-sync replica (ISR)** set holds replicas that are caught up. With `acks=all` and `min.insync.replicas=2` (replication factor 3), a write is acknowledged only when at least two replicas have it, surviving one broker failure without loss. `acks=1` is faster and can lose data if the leader dies; `acks=0` is fire and forget.
- **Consumers** in a **consumer group** divide partitions among themselves: parallelism is capped by the partition count, and one partition is read by one consumer in the group. Each group tracks its own **offsets**, so independent systems can read the same data. Commit offsets *after* processing for at-least-once behavior.
- **Rebalances** (when consumers join or leave) pause consumption; use static membership and cooperative rebalancing to reduce pain.
- **Retention:** by time or size, or **log compaction** (keep only the latest value per key, ideal for changelogs and tables).
- **Exactly-once semantics:** idempotent producers (dedupe retries), transactions (atomic writes across partitions and offset commits), and consumers that read committed data only. Valid inside Kafka-to-Kafka pipelines; for external side effects you still need idempotent consumers.
- **Throughput:** sequential disk I/O, the OS page cache, batching and zero-copy sendfile let one cluster handle millions of messages per second.
- **Operations:** modern Kafka runs with **KRaft** (built-in Raft metadata quorum) instead of ZooKeeper. Watch consumer lag, under-replicated partitions, disk usage and request latencies. Overpartitioning increases overhead and recovery time; plan partition counts for peak parallelism, and remember that increasing partitions later breaks key-to-partition mapping for ordering.
- **Ecosystem:** Kafka Connect (source and sink connectors), Kafka Streams and ksqlDB (stream processing), Schema Registry with Avro or Protobuf (compatible schema evolution), Debezium (change data capture).

7.14 Cassandra and Dynamo-style databases

- **Data model:** a table has a **partition key** (decides which nodes own the row) and **clustering columns** (sort order within a partition). Queries must supply the full partition key. You design **one table per query pattern**, duplicating data on purpose; there are no joins.
- **Ring and replication:** rows are placed on a token ring by hashing the partition key; each row is stored on RF (replication factor, typically 3) nodes. Any node can coordinate a request.
- **Tunable consistency:** per request choose ONE, QUORUM, LOCAL_QUORUM, ALL, and so on. With RF=3, QUORUM reads plus QUORUM writes (2+2 > 3) give read-your-writes behavior. LOCAL_QUORUM keeps latency inside one data centre.
- **Repair mechanisms: hinted handoff** (a node holds writes for a temporarily down peer), **read repair** (fix stale replicas found during reads), **anti-entropy repair** (Merkle-tree comparison, run regularly).
- **Pitfalls:** unbounded or very wide partitions (keep them under roughly 100 MB or 100k rows), hot partitions, tombstone accumulation, secondary indexes and `ALLOW FILTERING` (avoid), lightweight transactions (slow Paxos), large batches across partitions.
- **Fits:** time-series, messaging, IoT, activity feeds, anything needing massive write volume, multi-data-centre replication and high availability with well-known access patterns. **DynamoDB** is the managed relative: partition key plus sort key, single-digit-millisecond latency, on-demand or provisioned capacity, global tables. Its hot-partition and access-pattern-first design rules are the same.

7.15 Search engine internals

- An **inverted index** maps each term to the list of documents (and positions) containing it. A query looks up terms and intersects or unions their posting lists.
- **Analysis:** text is tokenized, lowercased, stemmed, stripped of stop words, and synonyms may be applied. Index-time and query-time analysis must match.
- **Ranking: TF-IDF** and its successor **BM25** score documents by term frequency (with saturation), inverse document frequency (rare terms matter more) and length normalization. Modern search adds **vector (semantic) search** with embeddings and **hybrid** blends of keyword and vector scores, plus learning-to-rank models.

- **Distribution:** an index is split into **shards**, each a self-contained Lucene index, with replica copies for availability and read scale. A query fans out to all shards and the coordinator merges the top results (scatter-gather). The number of primary shards is fixed at index creation, so plan it.
- **Near-real-time:** new documents become searchable after a **refresh** (about one second by default); durability comes from the translog and periodic **flush**.
- **Pitfalls:** deep pagination (use `search_after`), too many small shards, mapping explosions from dynamic fields, using it as the primary database (it is a search index, not a source of truth), and heavy aggregation memory use.
- **Typical architecture:** the primary database is the source of truth; a change-data-capture stream or an outbox feeds the search index; the index can always be rebuilt.

7.16 Object storage and erasure coding

Object stores (S3, GCS, Azure Blob, MinIO, Ceph) hold immutable blobs addressed by key, with metadata. They reach very high durability (S3 advertises eleven nines) by replicating or, more cheaply, by **erasure coding**: data is cut into k pieces plus m parity pieces (for example 10+4) spread over failure domains, and any k pieces rebuild the data. This costs about 1.4x storage instead of 3x for replication, at the price of more CPU and network work on reads and repairs.

Practical rules: store files in object storage and only their keys and metadata in the database; use **multipart upload** for big files and **pre-signed URLs** so clients upload and download directly without passing through your servers; use lifecycle rules to move cold data to cheaper tiers; use versioning for protection against accidental deletes. S3 provides strong read-after-write consistency for objects since late 2020; other implementations vary, so check.

7.17 Choosing a data store: workload cheat sheet

Workload	Start with	Consider when scale or shape demands
Transactions, orders, accounts	PostgreSQL	CockroachDB, Spanner, YugabyteDB, Vitess
Flexible nested documents, rapid iteration	PostgreSQL JSONB, MongoDB	DynamoDB, Couchbase

Workload	Start with	Consider when scale or shape demands
Sessions, caches, counters, leaderboards	Redis / Valkey	Dragonfly, Memcached
Huge write volume, known queries	Cassandra / ScyllaDB / DynamoDB	HBase, Bigtable
Full-text and faceted search	OpenSearch / Elasticsearch, Meilisearch, Typesense	Postgres full-text (small scale), Vespa
Analytics over big tables	ClickHouse, DuckDB, BigQuery	Druid, Pinot, Snowflake, Trino over Parquet
Metrics and time-series	Prometheus, VictoriaMetrics, TimescaleDB	InfluxDB, ClickHouse, M3
Relationship-heavy queries	PostgreSQL recursive queries, Neo4j	JanusGraph, Dgraph, TigerGraph
Files and media	S3-compatible object storage	CDN in front
Event log and streaming	Kafka / Redpanda	Pulsar, NATS JetStream
Vector similarity	pgvector	Qdrant, Milvus, Weaviate, FAISS, Vespa
Coordination, leader election, config	etcd, ZooKeeper, Consul	cloud-native lock services
Embedded, single-process	SQLite, RocksDB, DuckDB	LMDB, LevelDB

SQLite deserves respect. One file, no server, ACID, and fast enough for a surprising number of applications (with WAL mode it handles many concurrent readers and a single writer). Litestream and LiteFS add replication. For read-heavy, single-region products it can be the simplest correct choice.

Part 8: Deep Dive: Concurrency, Performance Math and Networking Internals

8.1 Processes, threads, coroutines

Unit	What it is	Cost	Notes
Process	Own memory space, own file descriptors	Heavy (MBs, slow to create, costly context switch)	Strong isolation; used by PostgreSQL, Chrome, Gunicorn workers, PHP-FPM
Thread	Shares memory with siblings; scheduled by the OS	About 1 MB of stack by default, kernel context switch cost (microseconds)	Shared state means locks and races
Green thread / coroutine / goroutine / virtual thread	Scheduled by the language runtime on a few OS threads	A few KB, nanosecond-scale switches	Hundreds of thousands are practical (Go, Java 21 virtual threads, Kotlin coroutines, Erlang processes, Python asyncio tasks)

Concurrency is dealing with many things at once (structure); **parallelism** is doing many things at once (needs multiple cores). An I/O-bound server benefits mostly from concurrency; a CPU-bound job needs parallelism.

8.2 Server concurrency models

Model	How it works	Strengths	Weaknesses	Examples
Process per request / pre-fork	Worker processes, each handling one request at a time	Isolation, simple code, crash containment	Memory heavy, limited concurrency	Apache prefork, Gunicorn sync workers, PHP-FPM
Thread per request (pool)	A pool of OS threads, one request per thread	Simple blocking code	Thousands of threads thrash; a slow dependency exhausts the pool	Tomcat, classic Spring MVC, Django with threads
Event loop (reactor)	One thread multiplexes thousands of sockets with epoll/kqueue and callbacks	Very high connection counts, low memory	One CPU-heavy or blocking call stalls everything; harder control flow	Nginx, Node.js, Redis, Netty, Python asyncio
Event loop per core	One event loop per CPU core, no sharing	Scales across cores, no lock contention	Sharding state by core	Seastar (ScyllaDB), Envoy, Nginx workers
Goroutines / virtual threads	Blocking-style code on cheap runtime threads	Simple code with event-loop scalability	Runtime complexity, GC	Go, Java 21+, Kotlin, Erlang
Actor model	Isolated actors with mailboxes, communicate only by messages	No shared state, supervision trees, distribution-friendly	Message overhead, debugging flows	Erlang/Elixir OTP, Akka, Orleans

Rule of thumb: a CPU-bound workload wants about as many worker threads as cores; an I/O-bound workload wants many lightweight tasks. **Never block an event loop**: push CPU-heavy or blocking work to a thread pool or a separate service.

8.3 I/O multiplexing: how one thread serves 100,000 sockets

The old `select` and `poll` calls scan every socket each time (O(n)). **epoll** (Linux), **kqueue** (BSD and macOS) and IOCP (Windows) let the kernel report only the sockets that are ready, so the cost scales with activity, not with connections. This is what solved the **C10K problem** (10,000 concurrent connections) and made C100K and beyond routine. **io_uring** (Linux) goes further with shared ring buffers and batched asynchronous system calls, reducing syscall overhead; it is used by modern runtimes and storage engines.

Resources you hit first at high connection counts: file descriptor limits (`ulimit -n` , `fs.file-max`), memory per connection (buffers), ephemeral ports on the client side (about 28,000 by default), connection-tracking table size in NAT or firewalls, and kernel accept-queue backlog (`somaxconn`).

8.4 Synchronization primitives and hazards

- **Mutex / lock:** one holder at a time. **RW lock:** many readers or one writer. **Semaphore:** a counted permit (limits concurrency). **Condition variable / channel:** wait for an event.
- **Atomics and compare-and-swap (CAS):** hardware-level lock-free updates of a single word; the basis of counters and lock-free queues. The **ABA problem** and memory ordering make hand-written lock-free code risky; use proven libraries.
- **Data race:** two threads access the same memory, at least one writes, with no synchronization. Result: undefined behavior. Go's `-race` detector, Rust's ownership system, Java's `synchronized` / `java.util.concurrent` and ThreadSanitizer all target this.
- **Deadlock** needs four conditions (Coffman): mutual exclusion, hold and wait, no preemption, circular wait. Break one: for example impose a global lock order.
- **Livelock** (threads keep reacting to each other without progress) and **starvation** (some thread never gets the resource).
- **False sharing:** two threads write different variables on the same CPU cache line (64 bytes) and keep invalidating each other's caches. Fix with padding.
- **Design strategies that avoid locks:** immutable data, message passing, the **single-writer principle** (one thread owns each piece of mutable state), sharding state by key so no two workers touch the same data, and copy-on-write.

8.5 Performance math every designer should know

Little's Law

L = lambda x W: the average number of items in a system (L) equals the arrival rate (lambda) times the average time each spends inside (W). Example: a service receiving 2,000 requests/sec with 50 ms average latency holds 100 requests in flight on average, so it needs capacity for at least 100 concurrent requests (threads, connections or coroutines). It also sizes connection pools and queues.

Utilization and queueing

For a simple single-server queue (M/M/1) the average time in the system is W = S / (1 - rho), where S is service time and rho is utilization. Latency does not rise linearly:

Utilization	Average time in system (multiple of service time)
50%	2x
70%	3.3x
80%	5x
90%	10x
95%	20x
99%	100x

Lesson: a system running hot has terrible latency and no headroom for bursts. Target 50 to 70% utilization at peak for latency-sensitive services, and use autoscaling and load shedding to keep it there.

Tail latency amplification

If one request fans out to N backend calls and waits for all of them, the chance of hitting at least one slow call grows fast. If each call exceeds its p99 latency 1% of the time, then with N = 100 the probability that a user request sees at least one slow call is 1 - 0.99^100, about **63%**. At scale the tail becomes the typical experience. Mitigations: fewer fan-out hops, **hedged requests** (send a duplicate after a delay and use the first reply), timeouts, load-aware routing, and smaller blast radius per call.

Amdahl's law and the Universal Scalability Law

Amdahl: if a fraction p of the work can be parallelized over n workers, speedup = 1 / ((1 - p) + p / n). With p = 95%, the maximum speedup is 20x no matter how many cores you add. The serial portion (a lock, a single database, a coordinator) caps scale. The **Universal Scalability Law** adds a coherency penalty (cross-talk between nodes), explaining why throughput can actually *fall* after a point when you add more nodes.

Capacity planning recipe

1. Measure the per-instance throughput at your latency target (load test, not guesswork).
2. Forecast peak requests per second (including growth and seasonal spikes, with a safety factor).
3. Instances needed = peak RPS / (per-instance RPS at 60 to 70% target utilization), plus N+1 or N+2 for failures and deploys.
4. Repeat for every tier (app, cache, database, queue) and every resource (CPU, memory, disk I/O, network, connections).

8.6 Memory, garbage collection and the OS

- **Stack vs heap:** stack allocation is free and automatic; heap allocation costs and requires reclaiming.
- **Garbage collection** families: reference counting (Python, Swift; cycles need extra handling), tracing mark-and-sweep, **generational** (most objects die young; JVM, .NET, V8), **concurrent and low-pause** collectors (Java ZGC and Shenandoah with sub-millisecond pauses, Go's concurrent collector). GC pauses create tail latency; remedies: reduce allocation rate, reuse buffers (pools), tune heap size, or choose a language with no GC.
- **Memory leaks** in long-running servers: unbounded caches, listeners never removed, growing maps, retained references. Always bound every cache and queue.
- **Page cache:** the OS caches file data in free RAM. Databases like PostgreSQL rely on it heavily, and Kafka's speed depends on it. "Free memory" on a server is not wasted.
- **mmap, zero-copy and sendfile:** moving file data to a socket without copying through user space (Nginx, Kafka). **Kernel-bypass** (DPDK, XDP/eBPF) is used for the most extreme packet rates.

- **NUMA:** on multi-socket machines, memory attached to another socket is slower; pin processes and memory for latency-critical software.
- **Swap** on a latency-sensitive database is usually a bad sign; set limits and avoid it.
- **Containers:** CPU limits cause throttling and latency spikes; memory limits cause OOM kills. Know the difference between requests (scheduling guarantee) and limits (hard cap).

8.7 TCP in depth

- **Handshake:** SYN, SYN-ACK, ACK adds one round trip before data. Opening a new connection per request wastes this, hence **keep-alive** and **connection pooling**.
- **Slow start and congestion control:** a new connection starts with a small window and grows it; loss signals congestion. Classic loss-based control (Reno, **CUBIC**, the Linux default) differs from model-based **BBR**, which can perform much better on long, lossy paths. Short-lived connections never reach full speed, another reason to reuse connections.
- **Nagle's algorithm and delayed ACK** can add up to 40 ms of latency to small request/response exchanges. Latency-sensitive software sets `TCP_NODELAY`.
- **Head-of-line blocking:** one lost packet stalls all the data behind it on that connection (including every multiplexed HTTP/2 stream). QUIC fixes this with independent streams.
- **TIME_WAIT:** after closing, the side that closed first keeps the connection state for about 60 seconds. Many short outbound connections to one destination can exhaust ephemeral ports; fix with connection reuse, more address or port range, or letting the server close.
- **Listen backlog:** if the accept queue overflows under a spike, clients see timeouts. Tune `somaxconn` and the application's backlog.
- **SYN flood:** attackers send SYNs without completing handshakes. **SYN cookies** let the server stay stateless until the handshake completes.
- **Keepalive and dead peers:** a TCP connection can look open for hours after the other side vanished (power loss, NAT timeout). Use TCP keepalive and application-level heartbeats.
- **MTU and fragmentation:** typical Ethernet MTU is 1500 bytes; tunnels and VPNs reduce it, and a blocked path-MTU discovery produces mysterious hangs for large packets. Clamp MSS when needed.

8.8 TLS essentials

- **TLS 1.3** needs one round trip for a new connection (TLS 1.2 needs two), and supports **session resumption** and optional **0-RTT** data (replayable, so only for idempotent requests).
- **Termination:** decrypt at the load balancer or proxy (simplifies certificates, allows L7 routing and inspection) or **pass through** to the backend (end-to-end encryption, no L7 features). Many internal systems re-encrypt to backends or use **mTLS** where both sides present certificates.
- **Certificates:** automate issuance and renewal (Let's Encrypt with ACME, cert-manager in Kubernetes, Caddy does it automatically). Expired certificates are one of the most common outage causes. Use **OCSP stapling**, modern ciphers, HSTS.
- **SNI** lets one IP serve many TLS hostnames; it is also how proxies route encrypted traffic by name without decrypting it.

8.9 DNS in depth

1. The application asks the OS stub resolver, which asks a **recursive resolver** (ISP, 8.8.8.8, 1.1.1.1).
2. The recursive resolver, if it has no cached answer, walks from the **root servers** to the **TLD servers** (.com) to the domain's **authoritative servers**.
3. The answer is cached everywhere along the path for its **TTL**.

Design implications: low TTLs speed failover but increase query load and latency; **negative caching** remembers "does not exist" answers; some resolvers and JVMs cache longer than the TTL; DNS-based load balancing cannot react faster than the slowest cache; **anycast** announces one IP from many locations so each client reaches the nearest one (used by root servers, CDNs and DDoS-resistant services); GeoDNS and latency-based routing choose answers by client location. **Split-horizon DNS** serves different answers inside and outside a network. For service discovery inside a cluster, DNS (CoreDNS) is convenient but its caching quirks require care.

8.10 Load balancer internals

- **Layer 4 modes:** NAT (rewrite addresses), **DSR** (direct server return: the backend replies straight to the client, bypassing the balancer, giving huge throughput), IP tunnelling. Large fleets spread traffic across many L4 balancers with **ECMP** routing,

and use **Maglev**-style or consistent hashing so a given connection keeps hitting the same backend even when balancers change.

- **Layer 7:** terminates the client connection, opens or reuses a backend connection, and can route on host, path, header, cookie or gRPC method, retry, rate limit, rewrite, compress and cache.
- **Connection draining** finishes in-flight requests before a backend is removed (essential for zero-downtime deploys).
- **Health checks:** TCP connect (cheap, shallow), HTTP status (checks the app), deep checks that touch dependencies (risky: a database blip can take every backend out at once, so many teams keep liveness shallow and use separate readiness checks). **Outlier detection** ejects hosts that return errors.
- **Client IP preservation:** behind a proxy, the backend sees the proxy's IP. Use `X-Forwarded-For` / `Forwarded` headers (and trust them only from your proxies) or the **PROXY protocol** for raw TCP.
- **Slow start and warm-up** gradually shift traffic to a freshly started instance whose JIT and caches are cold.
- **Client-side load balancing** (gRPC, service meshes, Finagle) removes the middle hop: the client knows the backend list from service discovery and picks one itself (often power-of-two-choices).

8.11 NAT, firewalls, hole punching and tunnels

Most home and mobile devices sit behind **NAT**, which allows outbound connections and drops unsolicited inbound packets. That is the whole reason tunnelling and relay services exist.

- **Outbound-initiated tunnels** (ngrok, Cloudflare Tunnel, SSH reverse tunnels `ssh -R`) work everywhere because the client connects out and the server sends traffic back over that connection.
- **Peer-to-peer connections** (video calls, games, WireGuard between laptops) use **STUN** (discover your public address), **UDP hole punching** (both sides send to each other so NATs create mappings) and **TURN** relays as a fallback when direct paths fail (symmetric NATs, strict firewalls). The whole negotiation is standardized as **ICE** (WebRTC).
- **WireGuard** is a small, fast, modern VPN protocol over UDP; Tailscale, Headscale and NetBird build mesh networks on it.

- **Design lessons:** keep heartbeats shorter than NAT idle timeouts (often 30 to 120 seconds for UDP, longer for TCP); expect connections to die silently and reconnect with exponential backoff and jitter; multiplex streams over a single connection; authenticate and rate-limit every tunnel to resist abuse.

8.12 HTTP details that decide performance and correctness

- **Caching headers:** `Cache-Control: max-age, s-maxage, no-store, no-cache, immutable, stale-while-revalidate, stale-if-error`; validators `ETag` and `Last-Modified` with conditional requests returning 304; `Vary` tells caches which request headers affect the response (misuse fragments or poisons caches). Fingerprinted asset names plus `immutable` and a year-long max-age is the standard recipe.
- **Compression:** gzip and Brotli (best for text); do not compress already-compressed media.
- **Range requests** enable resumable downloads and video seeking; **chunked transfer** and streaming allow responses before the full body is known.
- **Cookies and CORS:** `Secure`, `HttpOnly`, `SameSite` flags; CORS preflight requests add a round trip, so cache them with `Access-Control-Max-Age`.
- **WebSocket scaling:** connections are long-lived and stateful. Put a pub/sub backplane (Redis, NATS, Kafka) behind the socket servers so a message to a user reaches whichever node holds their connection; send heartbeats; reconnect with backoff and resume from the last event ID; set load balancer idle timeouts above heartbeat intervals; drain gracefully on deploy.
- **Server-Sent Events** are simpler for one-way streams and reconnect automatically with `Last-Event-ID`.
- **gRPC** streams (unary, server, client, bidirectional) run over HTTP/2; use deadlines on every call and propagate them downstream.
- **Large uploads:** accept direct-to-object-storage with pre-signed URLs; use resumable protocols (tus, multipart).

8.13 Latency budgets

Split your end-to-end target across the path so every tier knows its share. Example for a 300 ms page API at p99: DNS and TLS (cached or resumed) 20 ms, CDN or load balancer 5 ms, gateway and auth 10 ms, application logic 40 ms, two cache lookups 4 ms, one or two database queries 40 ms, downstream service calls 100 ms, serialization and network back

30 ms, leaving about 50 ms slack. Each tier gets a **timeout shorter than its caller's** (a downstream timeout of 2 s inside a caller with a 1 s timeout just wastes work). Propagate deadlines across services so abandoned work stops early.

Part 9: Case Studies, Set One

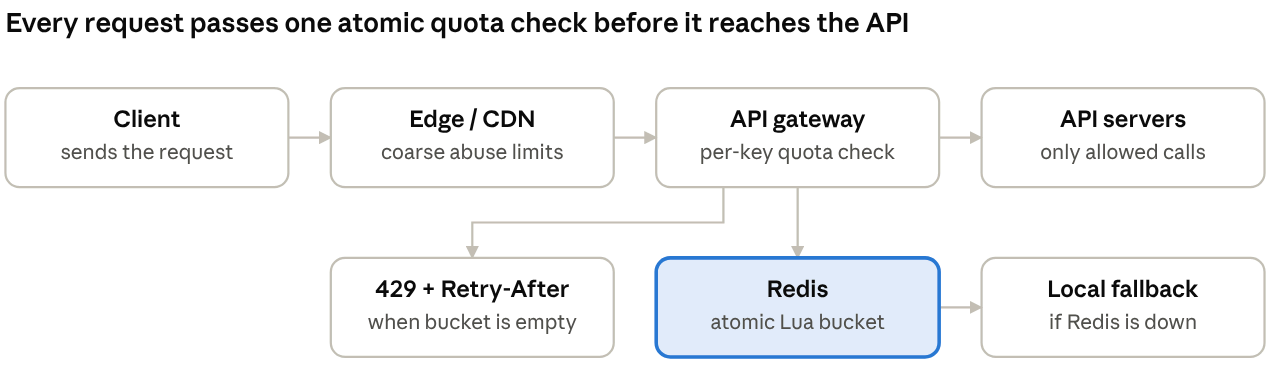
Each case study follows the same shape so you can reuse it on any new problem: **requirements, estimates, API and data model, high-level design, deep dives, trade-offs, failure modes, scaling path.** Read each one, then close the document and redesign it from memory. That is the real exercise.

9.1 Distributed rate limiter

Requirements. Limit each client (API key, user, IP) to a quota, for example 100 requests per minute, across a fleet of API servers. Add under 2 ms of latency, keep working if the limiter store is down, return HTTP 429 with `Retry-After` and `X-RateLimit-*` headers, support different limits per plan and per endpoint.

Estimates. 50,000 requests/sec at peak. Each check is one small read-modify-write, so the limiter store must sustain about 50,000 operations/sec with sub-millisecond latency. One Redis node handles 100,000 or more simple operations per second; a small cluster gives headroom.

Where to enforce. Options: in the API gateway (central, simple, one place), in a sidecar or middleware inside each service (no extra hop, needs a shared store), or at the CDN or edge (cheapest place to stop abuse and DDoS). Large systems layer them: coarse limits at the edge, per-customer limits at the gateway, per-resource limits in the service.



rate limiter · layered limits, Redis in the middle

Layered limits: the edge stops obvious abuse, the gateway enforces per-key quotas in Redis, and a local limit takes over if Redis is unreachable.

Algorithm choice. Token bucket is the default: it allows controlled bursts and needs two numbers per key (tokens, last refill time). Sliding-window counter gives smoother enforcement for strict quotas. A fixed window is the cheapest and permits double bursts at window boundaries.

Atomic check in Redis (Lua script, token bucket):

```
-- KEYS[1] = bucket key, ARGV = rate (tokens/sec), capacity, now (ms), cost
local data = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tokens = tonumber(data[1]) or tonumber(ARGV[2])
local ts = tonumber(data[2]) or tonumber(ARGV[3])
local elapsed = math.max(0, tonumber(ARGV[3]) - ts) / 1000
tokens = math.min(tonumber(ARGV[2]), tokens + elapsed * tonumber(ARGV[1]))
local allowed = 0
if tokens >= tonumber(ARGV[4]) then
  tokens = tokens - tonumber(ARGV[4])
  allowed = 1
end
redis.call('HSET', KEYS[1], 'tokens', tokens, 'ts', ARGV[3])
redis.call('PEXPIRE', KEYS[1], math.ceil(tonumber(ARGV[2]) /
tonumber(ARGV[1]) * 1000) * 2)
return allowed
```

The script runs atomically inside Redis, so concurrent API servers cannot race. Passing the time from the app server introduces clock-skew risk; on Redis 5+ you can read the Redis clock inside the script instead.

Deep dives and trade-offs.

- **Race conditions:** separate GET then SET loses updates; use the Lua script, or `INCR` plus `EXPIRE` for fixed windows.
- **Latency vs accuracy:** to avoid a network hop per request, each server can keep a local token allowance and sync with Redis in batches. This is faster but allows some overshoot (accuracy traded for speed).
- **Fail open vs fail closed:** if Redis is unreachable, allowing traffic (fail open) protects availability; blocking (fail closed) protects the backend. Most public APIs fail open with a conservative local fallback limit; security-sensitive endpoints (login attempts, OTP sends) fail closed.

- **Hot keys and sharding:** shard buckets across Redis nodes by key hash. A single abusive client hammers one shard, which is acceptable because it is exactly the traffic you are limiting.
- **Multi-region:** exact global limits need cross-region coordination and add latency. Usually give each region a share of the global quota and rebalance periodically.
- **Distinguish rate limiting (per time), quotas (per billing period) and concurrency limits (in-flight requests).** Different counters, different storage.
- **Dynamic rules** come from a config store and are cached locally.

Failure modes. Redis failover loses a few seconds of counters (harmless); a bug in the key function can lump all users into one bucket (alert on high rejection ratios); clients behind one NAT share an IP (prefer API keys and user IDs over IPs).

9.2 Distributed key-value store (Dynamo-style)

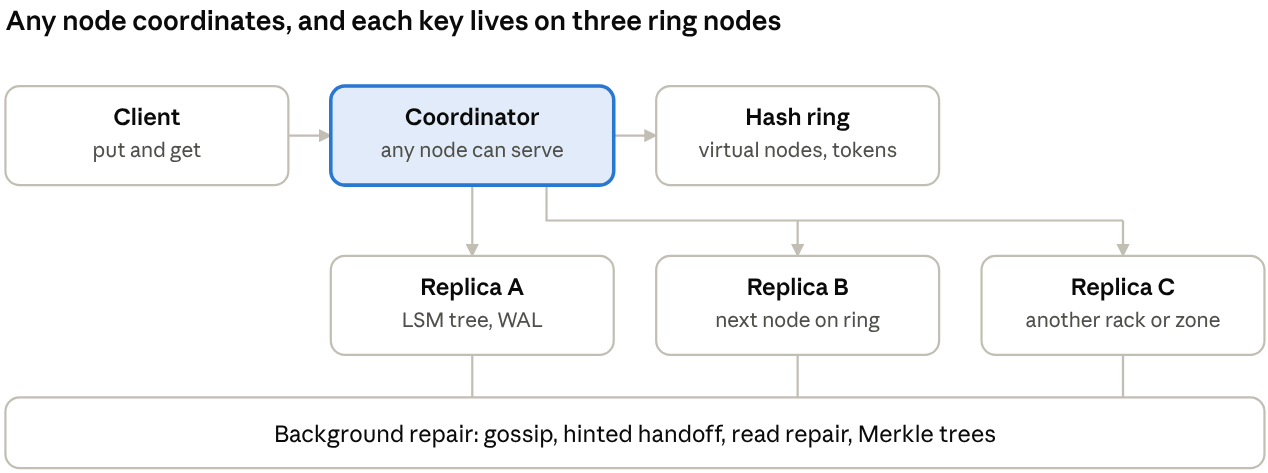
Requirements. `put(key, value)` and `get(key)` at very high volume, always writable, horizontally scalable, tolerant of node and data-centre failure, tunable consistency.

Design choices, in order:

1. **Partitioning:** consistent hashing with virtual nodes spreads keys and makes node joins cheap.
2. **Replication:** each key is stored on N nodes (typically 3), the next N distinct physical nodes clockwise on the ring (the **preference list**), ideally in different racks or zones.
3. **Consistency:** quorum with R + W > N for strong-ish reads (N=3, R=2, W=2); W=1 and R=1 for speed and availability. Choose per request.
4. **Coordinator:** any node can receive a request, find the preference list and coordinate the quorum. Clients can be ring-aware to skip a hop.
5. **Conflicts:** concurrent writes to different replicas produce divergent versions. Detect with **vector clocks** and return all siblings to the application to merge (Dynamo, Riak), or resolve by last-write-wins using timestamps (Cassandra; simple but can silently drop writes).
6. **Failure handling: sloppy quorum and hinted handoff** (if a preference-list node is down, write to a stand-in that hands the data back later), **read repair** (fix stale replicas when a read notices a mismatch), **anti-entropy with Merkle trees** (background comparison of replicas to repair silent divergence).
7. **Membership and failure detection:** gossip protocol spreads membership and health; a phi-accrual detector decides suspicion.

8. **Local storage engine:** an LSM tree (memtable, WAL, SSTables, compaction) per node for write speed.

The write path: client to coordinator; coordinator forwards to the N replicas; replicas append to the commit log and memtable and acknowledge; coordinator replies after W acknowledgements. **The read path:** coordinator asks R replicas (or one full read plus digest reads from others), compares versions, returns the newest, and triggers read repair if they differ.



Dynamo-style key-value store · request path and repair

A client reaches any node; that coordinator finds the three replicas on the ring, and background repair keeps them converging.

Trade-offs. This is an AP/EL design: always writable and fast, but you accept eventual consistency and the burden of conflict resolution. If you need linearizable operations (compare-and-set, unique constraints), use a consensus-backed store (etcd, ZooKeeper, Spanner, CockroachDB) for that data instead.

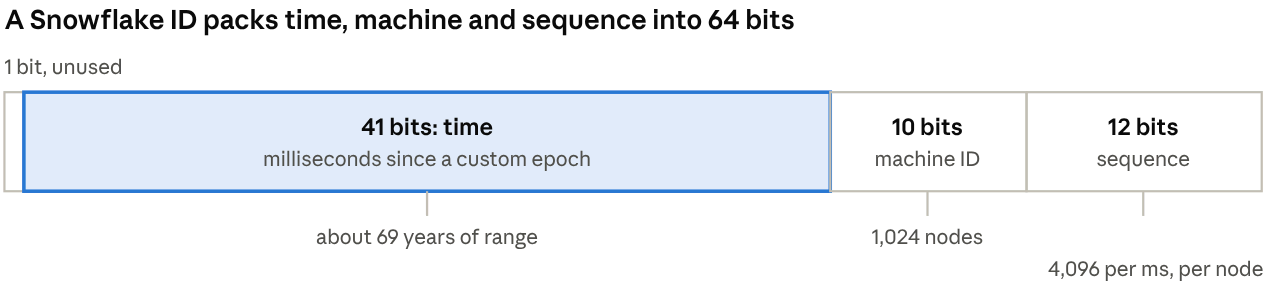
Scaling and operations. Add nodes in batches with streaming of token ranges; throttle repair and compaction so they do not starve foreground traffic; watch hot partitions and tombstone counts; plan capacity so that losing a zone still leaves enough nodes to serve traffic.

9.3 Unique ID generator

Requirements. Generate globally unique 64-bit IDs at tens of thousands per second per service, roughly time-ordered (good for index locality and sorting), no central bottleneck, survive node restarts.

Option	Pros	Cons
Database auto-increment	Simple, dense, ordered	Single writer bottleneck, leaks volume, hard across shards
Auto-increment with ranges per node (ticket servers, hi/lo allocation)	Fast, mostly ordered	Needs a coordinator for ranges; gaps on restart
UUID v4 (random 128-bit)	No coordination, collisions negligible	Large, random so bad index locality, not sortable
UUID v7 / ULID (time prefix plus random)	No coordination, sortable, index-friendly	128-bit
Snowflake-style 64-bit	Compact, sortable, no per-ID coordination	Needs unique machine IDs, depends on clock behavior

Snowflake layout (Twitter's design): 1 unused sign bit, 41 bits of milliseconds since a custom epoch (about 69 years), 10 bits of machine ID (1,024 nodes), 12 bits of sequence (4,096 IDs per millisecond per node). A single node can mint about 4 million IDs per second.



Time comes first, so new IDs sort by creation time and append to the index edge

Snowflake ID · 64-bit layout

Read left to right: the timestamp dominates ordering, the machine ID separates nodes, and the sequence separates IDs created in the same millisecond.

```
import time, threading

class Snowflake:
    EPOCH = 1700000000000          # custom epoch in ms
    def __init__(self, machine_id):
```

```
        assert 0 <= machine_id < 1024
        self.machine_id = machine_id
        self.last_ms, self.seq = -1, 0
        self.lock = threading.Lock()

    def next_id(self):
        with self.lock:
            now = int(time.time() * 1000)
            if now < self.last_ms:                      # clock moved
backwards
                raise RuntimeError('clock went backwards, refusing to issue
IDs')
            if now == self.last_ms:
                self.seq = (self.seq + 1) & 0xFFF
                if self.seq == 0:                       # 4096 used this ms:
wait for next ms
                    while now <= self.last_ms:
                        now = int(time.time() * 1000)
            else:
                self.seq = 0
            self.last_ms = now
            return ((now - self.EPOCH) << 22) | (self.machine_id << 12) |
self.seq
```

Hard parts. (1) **Machine ID assignment:** use a registry (etcd or ZooKeeper lease, or a database table) so two live nodes never share an ID. (2) **Clock going backwards** (NTP corrections, VM migrations): refuse or wait; never reuse timestamps. (3) **Ordering is only roughly global**: IDs from different nodes in the same millisecond are not causally ordered. If you need strict global order, you need a single sequencer or consensus, with the throughput cost that implies.

9.4 Notification system (push, SMS, email, in-app)

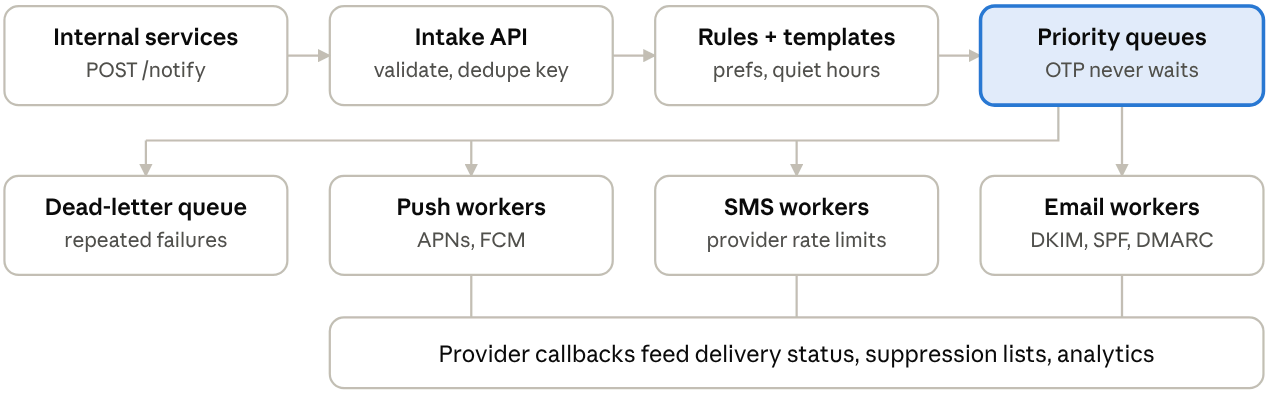
Requirements. Send notifications through several channels, at a scale of for example 10 million per day with bursts of 100x during campaigns; honor user preferences and quiet hours; avoid duplicates; retry failures; track delivery; rate limit per user and per provider.

Estimates. 10 million per day is only about 120 per second on average, but a marketing blast of 5 million within minutes means 10,000 to 20,000 per second for a short while. That burst is the real design driver, and the answer is a queue.

Architecture.

1. **Intake API** (internal services call `POST /notify` with user, template, data, priority, idempotency key). Validate and enqueue; return quickly.
2. **Preference and rules service:** is this user subscribed, which channels, time zone, quiet hours, frequency caps, locale. Cached heavily.
3. **Template service:** renders content per channel and language.
4. **Queues per channel and priority** (Kafka topics or SQS queues): high priority (OTP, security alerts) must never wait behind a marketing blast, so they get separate queues and dedicated workers.
5. **Channel workers** call providers: APNs (iOS push), FCM (Android push), SMS gateways (Twilio, Vonage, regional providers), email (SES, SendGrid, Postfix with proper DKIM, SPF, DMARC), WebSocket or in-app inbox. Each worker pool has its own rate limit matching the provider's limits, with retries and backoff.
6. **Delivery tracking:** provider callbacks and webhooks (delivered, bounced, opened, clicked, unsubscribed) land in an event stream and update a status store used for analytics and suppression lists.
7. **Dead-letter queues** for repeated failures, with alerting.

A queue per priority keeps one-time codes ahead of campaign bursts



notification system · queues per priority, workers per channel

High-priority messages get their own queue and workers, so a marketing blast can never delay a login code.

Deep dives.

- **Exactly-once illusion:** at-least-once delivery plus an **idempotency key** (store `notification_id` with TTL; skip repeats) prevents double sends on retries.

- **Fallback across channels:** if push fails or the token is invalid, try email or SMS only for high-value notification types (SMS costs money).
- **Fan-out for campaigns:** a segment job streams user IDs in batches into the intake path rather than loading millions in memory; pace it to provider limits.
- **Device tokens:** store per device, clean up tokens that providers report invalid.
- **Scheduling:** delayed and scheduled sends use a time-bucketed store (sorted set, or a scheduler table polled by shard) feeding the queues.
- **Compliance:** unsubscribe and consent records, DND registries and sender-ID rules for SMS, data retention, audit trail.
- **Cost control:** SMS and some channels are expensive, so batch, deduplicate and cap per user per day.

Failure modes. Provider outage (multi-provider failover per channel), queue backlog after a burst (autoscale workers on queue lag), poison messages (DLQ), thundering herd at the top of the hour (jitter the schedules).

9.5 Chat system (one-to-one and group)

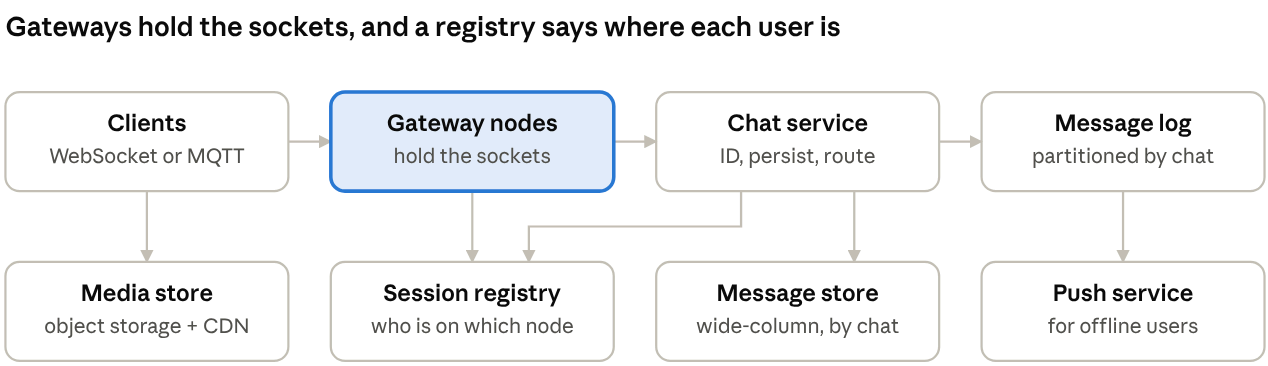
Requirements. Real-time messaging, group chats up to a few thousand members, online presence, message history, delivery and read receipts, multi-device sync, media sharing, low latency (under 200 ms), 50 million daily users.

Estimates. If each user sends 40 messages a day: 2 billion messages/day, about 23,000 per second average and maybe 70,000 at peak. At 100 bytes per message that is 200 GB/day of text, about 73 TB/year before replication. Connections: if 10 million users are online at once, you must hold 10 million WebSockets, around 100 to 200 chat servers at 50,000 to 100,000 connections each.

Architecture.

- **Connection layer:** clients keep a WebSocket (or MQTT) connection to a **gateway node** through a load balancer. Gateways are stateful only in that they know which sockets they hold.
- **Session registry:** a fast store (Redis) maps `user_id` to the gateway node(s) holding their connections.
- **Chat service:** receives a message, assigns an ID, persists it, then routes: look up recipients' gateways in the registry and push through them; if a recipient is offline, store for later and trigger a push notification.

- **Message broker:** a log (Kafka or similar) between the chat service and delivery workers decouples ingestion from fan-out and gives ordering per conversation (partition by `conversation_id`).
- **Storage:** messages are write-heavy, append-mostly and read by conversation in time order, a perfect match for a wide-column or LSM store (Cassandra, ScyllaDB, HBase, DynamoDB). Key: partition by `conversation_id` (and a time bucket for huge conversations), cluster by `message_id` (time-ordered ID). Metadata (users, groups, memberships) lives in a relational database.
- **Media:** uploaded to object storage via pre-signed URLs; messages carry only the reference. Thumbnails generated asynchronously; delivered via CDN.
- **Presence:** gateways send heartbeats; a TTL key in Redis marks a user online (`SET presence:user EX 30`). Fan-out of presence changes to friends is expensive, so only push to active viewers, batch updates, and tolerate staleness.
- **Receipts and sync:** each device tracks a per-conversation cursor (last seen message ID). On reconnect the client asks for everything after its cursor, which makes delivery reliable over flaky mobile networks. Delivered and read receipts are small events routed like messages.



chat system · sockets, registry, log, stores

A message enters through a gateway, the chat service stores it and looks up the recipients' gateways in the registry; offline users get a push notification instead.

Group chat. Small groups: fan out on write (write one copy to each member's inbox or push to each member's connection). Very large groups: store the message once per group and let members pull (fan-out on read), with a push signal of "new messages available".

Ordering. Within one conversation order by a server-assigned monotonic ID or timestamp plus sequence; across conversations order does not matter. Clients render by server order and reconcile optimistic local messages.

Reliability. The server acknowledges receipt; the client retries with a client-generated message ID for idempotency; undelivered messages are retained until acknowledged.

End-to-end encryption (Signal protocol, as in Signal and WhatsApp) means servers relay ciphertext only: no server-side search, spam detection or content moderation, and multi-device key management becomes the hard part. Decide early whether you need it.

Trade-offs. WebSocket (bidirectional, stateful, harder to scale and load balance) vs long polling or SSE plus POST (simpler, less efficient). Erlang/Elixir, Go, Rust and Node.js are natural gateway languages because of cheap concurrency. WhatsApp famously served millions of connections per server on Erlang.

9.6 News feed / timeline

Requirements. Users follow others; the home feed shows recent posts from people they follow, ranked or chronological, loading in under 300 ms. 300 million users, 100 million daily active, average user follows 200 accounts, some accounts have tens of millions of followers.

The central question: fan-out on write or on read?

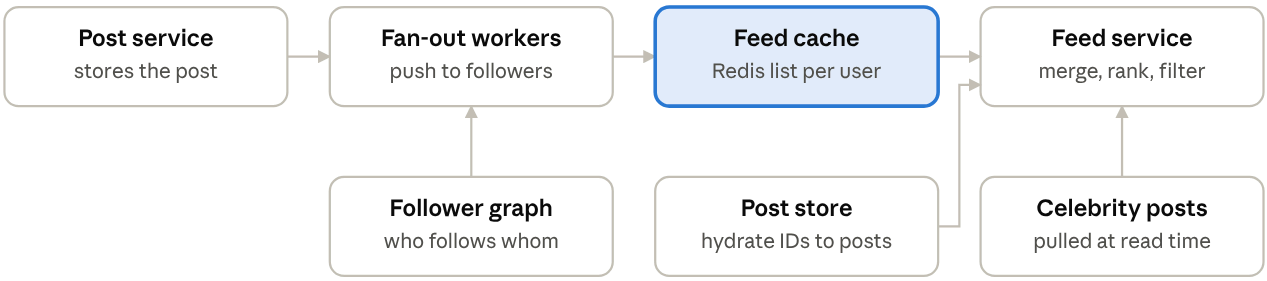
	Fan-out on write (push)	Fan-out on read (pull)	Hybrid
On post	Copy the post ID into every follower's precomputed feed (a cached list per user)	Store only the post	Push for normal users; pull for celebrities
On feed load	Read one list: very fast	Gather recent posts from all followed accounts and merge: slow	Read the list, merge in celebrity posts at read time
Cost	Huge write amplification for accounts with millions of followers, wasted work for inactive users	Expensive reads, hurts latency	Balanced
Freshness	Slight delay for large accounts	Immediate	Good

The hybrid design is the standard answer: push to followers' feed caches for accounts below a follower threshold (and only for recently active followers), and treat celebrity accounts as pulled at read time.

Components.

- **Post service** writes posts to a sharded database (partition by user ID) and publishes an event.
- **Fan-out workers** consume events, look up followers from the **graph service** (follower lists stored in a sharded store), and push the post ID into each follower's **feed cache** (a Redis list or sorted set holding the latest 500 to 1,000 IDs, trimmed on write).
- **Feed service:** reads the list of IDs, merges celebrity posts, hydrates IDs into full posts (batch cache lookups of posts, authors, like counts), applies **ranking**, filters (blocks, mutes, privacy), and returns a page with a cursor.
- **Ranking:** from simple reverse-chronological to ML-based scoring on recency, relationship strength, engagement predictions and diversity. A retrieval stage picks several hundred candidates cheaply, a ranking model scores them, and a re-ranking stage applies business rules.
- **Media and counters:** media from CDN; like and view counters are high-write, tolerate approximation, and are aggregated in batches (a sharded counter, or Redis with periodic flush to the database).
- **Pagination:** cursor-based (last seen post ID or score), never offset-based, because the feed changes while the user scrolls.

Push to normal followers, pull celebrity posts at read time



news feed · hybrid fan-out

Normal posts are pushed into followers' feed caches; celebrity posts are fetched when a feed is read and merged in.

Estimates and sizing. Feed cache: 100 million active users x 800 IDs x 8 bytes is about 640 GB of raw IDs, comfortably held in a sharded Redis cluster. Fan-out writes: if each post reaches 200 followers on average and there are 100 million posts/day, that is 20 billion cache writes/day, about 230,000 per second, which is why queues, batching and the celebrity exception matter.

Failure modes and fixes. A celebrity posting causes a write storm (hybrid avoids it); feed cache loss (rebuild lazily from the post store using the pull path); hot posts (cache aggressively, replicate hot keys); stale feeds for inactive users (rebuild on login rather than maintaining all feeds); privacy changes and deletions must be applied at read time because cached feeds can contain removed content.

Part 10: Case Studies, Set Two

10.1 Video streaming platform (YouTube / Netflix style)

Requirements. Users upload videos; others watch with fast start and no buffering on any device and network; search and recommendations; 100 million daily viewers.

Estimates. If 5 million videos are uploaded per day at an average of 5 minutes and 100 MB raw, that is 500 TB/day of raw uploads, plus every video is stored in several resolutions (often 5 to 8 renditions), so storage multiplies by 2 to 3x. Viewing dominates: 100 million viewers x 60 minutes at an average 3 Mbit/s is about 1.35 petabytes of egress per hour of average load. **Bandwidth cost, not compute, is the main constraint**, which is why CDNs are the core of the design.

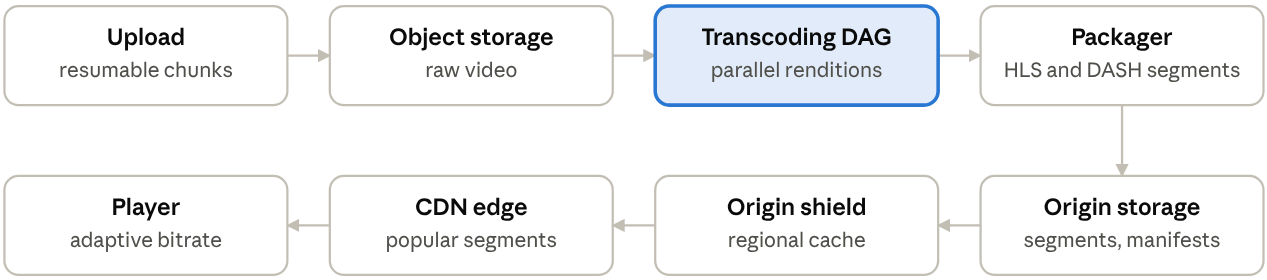
Upload and processing pipeline.

1. Client uploads in chunks (resumable, multipart) directly to object storage via pre-signed URLs.
2. An event triggers the **transcoding pipeline**: split the video into segments, transcode each segment in parallel into multiple resolutions and bitrates (H.264 for compatibility, VP9, AV1 or HEVC for efficiency), generate thumbnails, extract audio, and create captions. This is a **DAG of tasks** run by a workflow engine with workers that scale on a queue (Temporal, Airflow, Argo, or a custom scheduler) over cheap or spot compute.
3. Package into **adaptive bitrate streaming** formats: **HLS** and **MPEG-DASH** cut the video into 2 to 10 second segments with a manifest listing every rendition. The player measures its bandwidth and switches renditions between segments, which is how streams adapt to changing networks.

4. Run safety checks (copyright fingerprinting, content moderation) and publish metadata to the database and search index.

Playback path. The player requests the manifest, then segments, from the **CDN**. Popular content is cached at edge locations; **Netflix** goes further and places its own **Open Connect** appliances inside ISP networks, pre-loading likely content during off-peak hours (push-style CDN). Long-tail content is fetched from regional caches or the origin on demand (pull-style).

Videos are cut into segments, cached at the edge, fetched at the right bitrate



video platform · upload pipeline (top) and playback path (bottom)

The top row is write-once processing; the bottom row is the read path, which almost never touches the origin for popular videos.

Data stores. Video metadata (title, owner, status) in a sharded relational database or document store; watch history and view counts in a write-heavy store (Cassandra, Bigtable) with counters aggregated through a stream pipeline; search index (OpenSearch or Elasticsearch) fed by change events; object storage for all media; Redis for hot metadata.

Deep dives and trade-offs.

- **Start-up latency vs quality:** start with a lower rendition for the first segment, then ramp up.
- **Codec choice:** newer codecs save 30 to 50% bandwidth but cost much more to encode and need device support, so transcode them only for videos that become popular (a cost-aware, demand-driven strategy).
- **Long-tail storage:** rarely watched renditions move to colder, cheaper storage classes and are re-created on demand.
- **Live streaming:** ingest (RTMP, SRT, WebRTC), low-latency transcoding, LL-HLS or WebRTC delivery; far tighter latency budgets and no time to pre-cache.

- **DRM and security:** encrypted segments (Widevine, FairPlay, PlayReady), signed URLs with expiry to prevent hotlinking.
- **Recommendations** are a separate large system (see Part 11).

Failure modes. Transcode worker loss (retry per segment, idempotent outputs), CDN cache stampede on a viral video (request coalescing at the edge, origin shield tier), origin overload (shield cache, rate limits), regional outages (multi-CDN).

10.2 File storage and sync (Dropbox / Google Drive style)

Requirements. Upload and download files, automatic sync across devices, sharing and permissions, version history, offline editing, handle large files, resolve conflicts, 500 million users and exabyte-scale storage over time.

Key insights.

- **Chunking:** split files into blocks (for example 4 MB). Only changed blocks transfer after an edit; uploads resume after failures; blocks upload in parallel.
- **Content-addressed storage and deduplication:** each block is identified by the hash of its content (SHA-256). If a block already exists (from any user, within privacy limits), do not store it again. Savings are large for common files and versions; this also makes blocks immutable and cache-friendly. (Cross-user deduplication has privacy side channels and is often limited to within an account or organization.)
- **Separation of metadata and data:** a **metadata service** (relational database, sharded by user or namespace) stores the file tree, versions, block lists and permissions; a **block store** (object storage) holds the blocks.

Sync protocol.

1. A desktop client watches the file system (inotify, FSEvents, ReadDirectoryChangesW), detects a change, splits it into blocks, computes hashes.
2. It asks the server which block hashes are missing, uploads only those, then commits a new file version (list of block hashes) to the metadata service.
3. Other devices learn of the change via a **notification channel** (long polling, WebSocket, or push) carrying a changed-namespace cursor, then fetch the new metadata and any missing blocks.
4. Each namespace has a **journal** with monotonically increasing sequence numbers; each client remembers its cursor and asks for changes after it. This is the same cursor pattern as in the chat design.

Conflict handling. Two devices edit the same file while offline. Simple systems keep both versions ("file (conflicted copy)"); collaborative document editors use **operational transformation (OT)** or **CRDTs** to merge edits automatically (Google Docs historically used OT; many newer tools, such as Figma-style editors and Yjs-based apps, use CRDT variants).

Data model sketch.

```
CREATE TABLE file_version (
  file_id     BIGINT,
  version     INT,
  namespace_id BIGINT NOT NULL,
  size_bytes  BIGINT,
  block_hashes BYTEA[] NOT NULL,   -- ordered list of block content hashes
  modified_at TIMESTAMPTZ,
  PRIMARY KEY (file_id, version)
);
CREATE TABLE block (
  hash      BYTEA PRIMARY KEY,     -- SHA-256 of contents
  size      INT,
  storage_key TEXT,                -- location in object storage
  refcount  BIGINT                 -- garbage-collect when it reaches zero
);
```

Trade-offs. Block size: small blocks give better deduplication and smaller diffs but more metadata and requests; large blocks are efficient for big media. Strong consistency on metadata (users must not see a half-committed version) vs eventual consistency on block replication. Versions and deleted files keep blocks alive, so storage grows; use retention policies and reference-counted garbage collection that runs carefully (never delete a block that a concurrent upload is about to reference).

Security. Encryption at rest and in transit; per-user or per-organization keys; optional client-side encryption (the server cannot deduplicate across users then); signed, expiring share links; audit logs; malware scanning.

10.3 Ride-hailing (Uber / Ola style): geospatial matching

Requirements. Riders request a ride; the system finds nearby available drivers, matches one, tracks the trip live, computes ETA and price. Millions of drivers send location updates every few seconds.

Estimates. 1 million active drivers sending a location every 4 seconds gives 250,000 writes per second, a high-volume, latency-sensitive, mostly ephemeral stream. Only the latest position per driver matters for matching, so keep it in memory with a TTL instead of in a durable database, and write a sampled history to a stream or time-series store for analytics.

The core problem: finding nearby drivers fast. A plain latitude and longitude index does not support efficient radius queries. Use a **spatial index**:

Index	Idea	Notes
Geohash	Interleaves lat/lng bits into a string; nearby places share prefixes	Easy to store in any key-value store or Redis; edge cases near cell boundaries (query neighboring cells)
Quadtree	Recursively split the map into four cells until each has few points	Adapts to density (dense cities get smaller cells)
S2 (Google) / H3 (Uber)	Hierarchical cell systems on a sphere (S2 squares, H3 hexagons)	Uniform neighbor distances (H3), excellent for aggregation and surge pricing
R-tree / PostGIS	Database spatial indexes	Great for static and moderately dynamic data

Driver positions live in a sharded, in-memory structure partitioned by city or cell ID (Redis GEO commands, or a custom service). A ride request queries the rider's cell and its neighbors, filters by availability and vehicle type, ranks by ETA (using road network routing, not straight-line distance), and offers the ride to the best candidate. If the driver declines or times out, offer to the next one.

Matching. Greedy matching per request is simple and fast. Batched global matching over a few seconds (an assignment problem solved with algorithms like Hungarian or approximations) improves overall wait times, at the cost of a short delay.

Other services.

- **Trip service:** a state machine (requested, matched, driver en route, in progress, completed, cancelled) with durable, strongly consistent storage, since money depends on it. Use idempotent transitions and an event log.

- **ETA and routing:** map graph, live traffic, precomputed contraction hierarchies or similar speed-ups; ML-corrected travel times.
- **Dynamic (surge) pricing:** aggregate supply and demand per H3 or S2 cell over sliding windows in a stream processor (Flink, Kafka Streams) and publish multipliers.
- **Real-time tracking:** rider app subscribes over WebSocket to the driver's location for that trip only.
- **Payments:** pre-authorize, charge at trip end, retries and reconciliation (see 10.6).
- **Fraud, safety and trust:** anomaly detection on GPS, accounts and payments.

Trade-offs. In-memory location store (fast, volatile; rebuild in seconds from the next updates) vs durable (slow, unnecessary). Partition by geography: cities are natural shards, but hot cells (airports, stadiums) need extra capacity. Accuracy vs battery: how often devices send updates.

10.4 Search autocomplete (typeahead)

Requirements. As the user types, show the top 5 to 10 completions within 100 ms; suggestions reflect popular and trending queries; support multiple languages; handle 100,000 requests/sec at peak (a request per keystroke).

Data structure: the trie. A prefix tree where each node represents a prefix. For fast retrieval, each node stores the **top K completions** for that prefix precomputed, so a lookup walks the prefix and returns the stored list in O(length of prefix), independent of the number of queries.

Pipeline (offline plus online):

1. **Log collection:** every completed search goes to a stream (Kafka).
2. **Aggregation:** a batch job (hourly or daily) or stream job (for trending) counts query frequency with time decay, filters bad content (abuse, personal data, adult terms), and normalizes text.
3. **Trie builder:** builds the new trie or prefix map with top-K lists and publishes a versioned snapshot to storage.
4. **Serving:** suggestion servers load the snapshot in memory and serve lookups; roll out new snapshots gradually. A distributed cache and the CDN or browser cache hold common prefixes (short prefixes like "a" or "ho" are extremely hot and can be cached for minutes).

An alternative to a trie is a sharded key-value store mapping `prefix -> top-K list`, or an inverted index with edge n-grams in Elasticsearch/OpenSearch (flexible but slower and heavier).

Details that matter.

- **Client behavior:** debounce keystrokes (wait about 100 to 150 ms), cancel stale requests, cache previous results, and start suggesting after one or two characters.
- **Ranking:** frequency, recency (trending boost), personalization (user history and location), and context (language, device). Keep personalization in a lightweight layer that re-ranks the global top candidates.
- **Typo tolerance:** edit-distance or phonetic matching on top of the prefix results.
- **Sharding:** by first one or two characters is unbalanced (many queries start with common letters); shard by prefix ranges chosen from the measured distribution, or by hash of a longer prefix.
- **Freshness vs cost:** rebuild daily for the long tail; update a small trending layer every few minutes; merge both at query time.
- **Safety:** blocklists, legal removal requests, and avoiding suggestions that expose private information.

10.5 Web crawler

Requirements. Crawl 1 billion pages per month for a search index, respect `robots.txt` and politeness limits, avoid duplicates and spider traps, and keep content reasonably fresh.

Estimates. 1 billion pages per month is about 400 pages/sec. At 100 KB per page that is about 100 TB per month of raw content and about 40 MB/sec of bandwidth. This is a modest rate for a distributed system but demanding on DNS, politeness scheduling and storage.

Architecture.

1. **URL frontier:** the prioritized queue of URLs to fetch. It has two jobs: **priority** (important or frequently changing pages first, using PageRank-like signals and change history) and **politeness** (never hammer one host). Implement with many per-host queues plus a scheduler that releases at most one request per host per time interval (the host-to-queue mapping keeps a host on a single crawler for rate control).

2. **Fetchers:** a large pool of asynchronous workers that resolve DNS (with a local DNS cache, since DNS becomes a bottleneck), fetch pages with timeouts and size limits, follow redirects carefully, and honor `robots.txt` (cached per host) and crawl-delay hints.
3. **Parser and extractor:** extract text, links, metadata; normalize and canonicalize URLs (lowercase host, remove fragments, resolve relative links, sort query parameters where safe).
4. **Deduplication:** a URL-seen check using a Bloom filter in front of a persistent set (to avoid refetching), and **content dedup** using hashes (exact) and SimHash or MinHash (near duplicates).
5. **Storage:** raw pages in object storage (compressed), extracted content and link graph in a distributed store, which feed the indexer and ranking pipeline.
6. **Scheduler for recrawls:** estimate each page's change rate and recrawl proportionally; news home pages every few minutes, static pages monthly.

Problems and solutions.

- **Spider traps** (infinite calendars, session-ID URLs, generated pages): depth and per-host page limits, URL pattern heuristics, content-similarity checks.
- **Dynamic pages** rendered by JavaScript need a headless browser fleet (expensive, so used selectively).
- **Politeness and ethics:** identify your crawler with a user agent and contact URL, obey robots.txt, back off on errors (429 and 5xx), respect the site's terms and legal constraints.
- **Failure handling:** crawlers are disposable; the frontier state is checkpointed; fetches are idempotent.
- **Scaling:** shard the frontier by host hash across crawler nodes so each host is owned by one node (politeness stays local), with consistent hashing to rebalance.

10.6 Payment system

Requirements. Accept payments for merchants and users, never lose or duplicate money, support multiple methods and currencies, reconcile with banks and card networks, comply with regulation. **Correctness dominates every other property**: it is acceptable to be slower or temporarily unavailable, never to be wrong.

Core concepts.

- **Never build card handling yourself if you can avoid it.** Card data falls under **PCI-DSS**. Use a payment service provider (Stripe, Adyen, Razorpay, PayU, Braintree, PayPal) with hosted fields or tokenization so raw card numbers never touch your servers. In India, UPI, net banking and wallets have their own flows and regulator rules (RBI guidelines, tokenization of cards).
- **Payment flow (card):** client gets a token from the PSP; your server creates a payment intent (idempotently); the PSP authorizes with the issuer; you capture (immediately or later); settlement happens days later; refunds and chargebacks follow. Asynchronous **webhooks** report the final status, and your system must treat webhooks as the source of truth, verifying signatures and handling duplicates and out-of-order delivery.
- **Idempotency everywhere:** every state-changing call carries an idempotency key; the server stores the key and the response and replays it on retries. This is the single most important pattern, since networks time out between your request and the answer.
- **Double-entry ledger:** record every movement of money as balanced entries (debit one account, credit another, sum is zero). The ledger is **append-only and immutable**; balances are derived. This gives auditability, and bugs show up as imbalances instead of silent drift.

```
CREATE TABLE ledger_entry (
  entry_id    BIGINT PRIMARY KEY,
  txn_id      UUID NOT NULL,
  account_id  BIGINT NOT NULL,
  amount      BIGINT NOT NULL,        -- minor units (paise, cents); never
floating point
  currency    CHAR(3) NOT NULL,
  direction   CHAR(1) CHECK (direction IN ('D','C')),
  created_at  TIMESTAMPTZ DEFAULT now()
);
-- invariant per txn_id: SUM(debits) = SUM(credits); enforce in code and with
periodic checks
```

- **Money as integers** in minor units plus a currency code. Never use floating point for money.
- **State machine** for each payment (created, authorized, captured, failed, refunded, disputed) with explicit allowed transitions stored durably.
- **Consistency:** use a relational database with ACID transactions (PostgreSQL, MySQL, or a distributed SQL database) for the ledger and payment state. Where you must coordinate across services, use **sagas** with compensating actions plus the **transactional outbox** so events are published reliably.

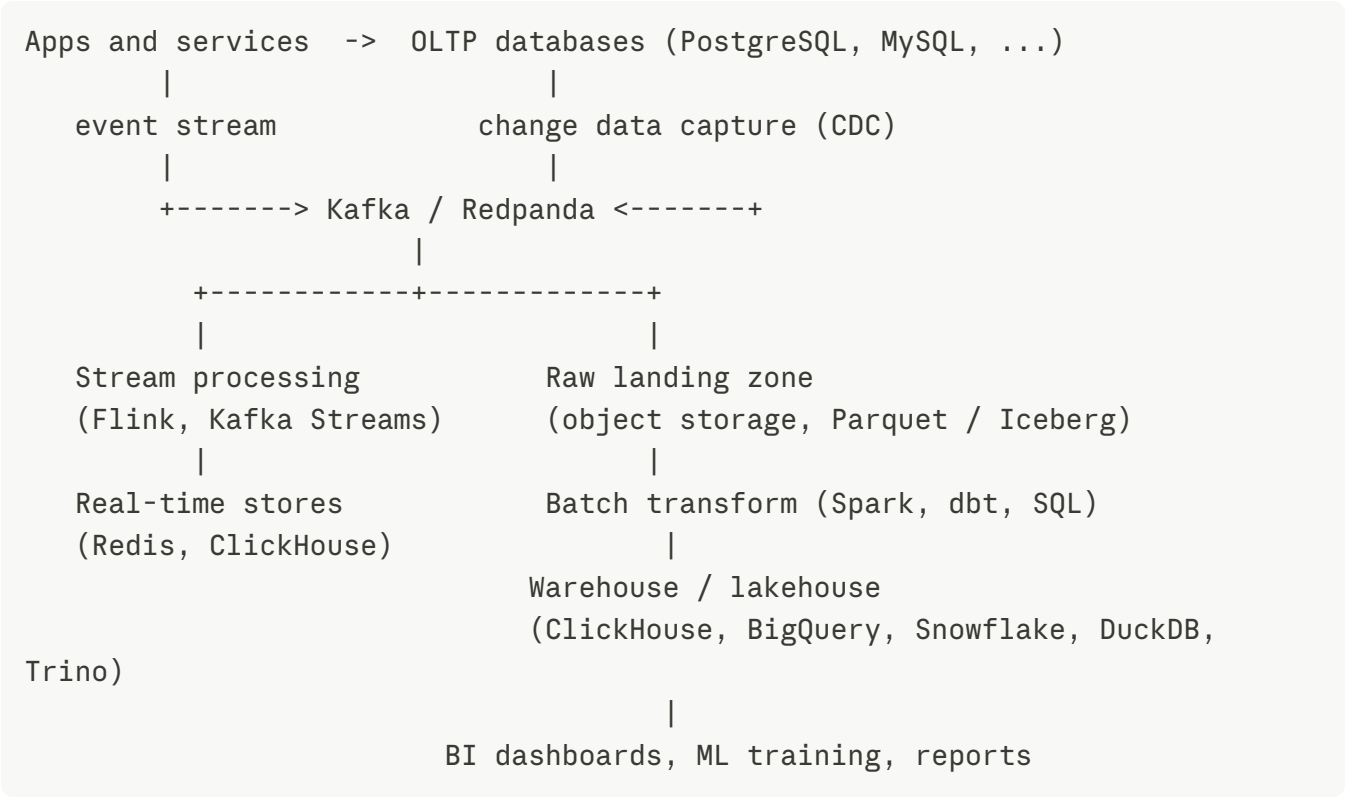
- **Reconciliation:** compare your records with PSP settlement files and bank statements daily, flag mismatches, and investigate. Reconciliation is how you find the bugs that tests missed.
- **Fraud and risk:** velocity checks, device fingerprinting, rules plus ML scoring, 3-D Secure and OTP challenges; keep a manual review queue.
- **Security and compliance:** tokenization, encryption, HSMs for keys, strict access control and audit trails, segregated cardholder-data environment, data localization rules where required.

Failure modes to design for. Timeout after sending a charge (query the PSP by idempotency key; never blindly retry with a new key), duplicate webhooks, webhook arriving before the API response, partial failure between debiting one account and crediting another (single database transaction or saga), clock-related ordering errors, and PSP outages (secondary provider with routing rules; this adds reconciliation complexity).

Part 11: Data Engineering, Analytics and Machine-Learning Systems

11.1 The data platform picture

Operational databases serve users; **analytical systems** answer questions about the business and feed machine learning. Keeping the two apart protects production.



ETL vs ELT. ETL transforms data before loading (older, constrained warehouses). **ELT** loads raw data first and transforms inside the warehouse with SQL (dbt), which is the modern default because storage and compute are cheap and raw data stays available for reprocessing.

Batch vs streaming. Batch processes bounded data on a schedule: simpler, cheaper, easy to rerun and correct. Streaming processes unbounded data continuously: low latency, harder state management and correctness. Use streaming only when the business needs results in seconds or minutes. **Lambda architecture** runs both a batch and a speed layer and merges results (two code paths, hard to keep consistent). **Kappa** uses streaming only and replays the log for reprocessing.

File and table formats.

- **Parquet** and **ORC** are columnar file formats with compression and statistics, which make analytical scans fast and cheap. CSV and JSON are fine for exchange, poor for analytics at scale.
- **Table formats (Apache Iceberg, Delta Lake, Apache Hudi)** add ACID transactions, schema evolution, time travel and efficient updates on top of files in object storage. Together with query engines (Spark, Trino, Flink, DuckDB, ClickHouse) they form the **lakehouse**, which unifies data lake flexibility and warehouse reliability.

- **Partitioning** by date (and sometimes another low-cardinality key) lets queries skip data; avoid too many tiny files (compact them) and over-partitioning.

Orchestration and transformation: Airflow, Dagster, Prefect, Argo Workflows and Temporal schedule and retry pipelines; **dbt** manages SQL transformations with tests and documentation. Make every job **idempotent** and **partition-scoped** (rerunning a day overwrites that day), so backfills and retries are safe.

Data quality and governance. Schema contracts between producers and consumers, automatic tests (nulls, uniqueness, ranges, freshness, volume anomalies), a data catalog and lineage (DataHub, OpenMetadata, Amundsen), access control and PII handling (masking, tokenization, retention, right-to-erasure workflows). Most data incidents are upstream schema or semantic changes, so treat events and tables as APIs with versioning.

11.2 Stream processing concepts

- **Event time vs processing time:** events carry the time they happened; they arrive late and out of order. Correct results use event time.
- **Watermarks** estimate how far event time has progressed so the system knows when a window can close; **allowed lateness** and side outputs handle stragglers.
- **Windows:** tumbling (fixed, non-overlapping: counts per minute), sliding (overlapping: last 5 minutes updated every 30 seconds), session (gaps of inactivity define a user session).
- **State and checkpointing:** aggregations, joins and deduplication need state stored locally and **checkpointed** to durable storage so a failed job resumes without loss. Flink checkpoints with consistent snapshots and can provide end-to-end exactly-once with transactional sinks.
- **Joins:** stream-stream joins within a time window, and stream-table joins enriching events with reference data (change-captured dimension tables).
- **Backpressure** propagates slowness upstream instead of dropping data; monitor consumer lag.
- **Reprocessing:** keep the raw log (Kafka retention or the lake) so fixes can be replayed.

11.3 Change data capture and the outbox

CDC reads a database's transaction log (PostgreSQL logical decoding, MySQL binlog) and emits every row change as an event, without extra load on queries and without changing application code. **Debezium** is the standard open-source tool, usually paired with Kafka Connect. Uses: replicate to the warehouse, keep search indexes and caches in sync, feed microservices, build audit trails. Trade-offs: events mirror table structure (leaking your schema to consumers), schema changes need care, and initial snapshots of big tables take time. The **outbox pattern** complements CDC: the application writes a deliberately designed event row into an outbox table in the same transaction as its business change, and CDC publishes outbox rows. This yields reliable, well-shaped domain events with no dual-write problem.

11.4 Warehouse modeling

- **Star schema:** a central **fact table** (events or measurements: orders, page views) surrounded by **dimension tables** (customer, product, date). Simple queries, fast aggregation.
- **Wide denormalized tables** are common with columnar engines, which compress repeated values well and avoid joins.
- **Slowly changing dimensions (SCD type 2)** keep history by versioning dimension rows with validity ranges.
- **Pre-aggregation:** materialized views, rollup tables and OLAP cubes trade storage and freshness for query speed.
- **Engines:** ClickHouse (extremely fast open-source columnar database), DuckDB (embedded analytical database, brilliant for single-machine work on Parquet), Trino and Presto (federated SQL over many sources), Apache Druid and Pinot (real-time analytics with high concurrency), BigQuery, Snowflake and Redshift (managed warehouses).

11.5 Event analytics pipeline (design walkthrough)

Problem. Collect 50,000 events per second from web and mobile apps; dashboards update within a minute; keep raw data for two years.

1. **SDK and collector:** a lightweight client library batches events, retries with backoff and sends to a stateless HTTP collector (behind a load balancer and CDN edge) that validates against a schema and writes to Kafka. Return 202 quickly.

2. **Schema management:** events have a name, version, timestamp (client and server), user and device IDs and a typed payload, registered in a schema registry; reject or quarantine invalid events.
3. **Stream layer:** a Flink or Kafka Streams job deduplicates (event ID within a window), enriches (geo-IP, user attributes), sessionizes, and writes to ClickHouse or Druid for real-time dashboards.
4. **Batch layer:** raw events land in object storage as Parquet partitioned by date; dbt models build curated tables for analysts.
5. **Serving:** dashboards query ClickHouse; scheduled reports query the warehouse.

Concerns. Clock skew (trust server receive time as well as client time), bots and duplicates, privacy consent and opt-outs, cost of high-cardinality dimensions, and late mobile events arriving days after being generated (design backfill-friendly aggregates).

11.6 Machine-learning system design

A model is a small part of a production ML system; the surrounding data, serving and monitoring dominate the effort.

The lifecycle.

1. **Problem framing:** what decision does the model support, what is the business metric, what is the cost of each kind of error, and is machine learning even needed (a rule or a heuristic often wins first)?
2. **Data:** collect, label, version, split carefully (by time or by user to avoid leakage), check quality and bias.
3. **Features:** engineered inputs. The **training-serving skew** problem (features computed differently offline and online) is a classic failure; a **feature store** (Feast, Tecton, Hopsworks) gives one definition for both paths.
4. **Training:** reproducible pipelines, experiment tracking and a **model registry** (MLflow, Weights & Biases), distributed training when needed.
5. **Evaluation:** offline metrics (precision, recall, AUC, NDCG, calibration), slice analysis (performance by segment), then **online A/B tests** against real business metrics.
6. **Serving: batch prediction** (precompute scores nightly; cheapest and simplest) vs **online inference** (an API with latency budgets, autoscaling, caching, batching of requests on GPUs) vs **edge/on-device** (privacy, offline use). Use model servers such as TorchServe, Triton, vLLM (for LLMs), BentoML, KServe, or ONNX Runtime.

7. **Release safely:** shadow mode (predict but do not act), canary and gradual rollout, instant rollback.
8. **Monitor:** system metrics (latency, errors) and model metrics (input **data drift**, **concept drift**, prediction distribution, delayed ground-truth performance), with retraining triggers and human review for high-stakes decisions.

Trade-offs. Complexity vs gain (a simple model that ships beats a perfect one that does not); latency vs accuracy (distill or quantize models; cascade a cheap model before an expensive one); freshness vs cost of retraining; interpretability vs raw accuracy (regulated domains need explanations); and feedback loops (a recommender trained on its own clicks narrows what users see).

11.7 Retrieval-augmented generation (LLM application architecture)

A typical LLM application grounds a model in private or fresh data:

1. **Ingestion:** parse documents (PDF, HTML, office files), clean, **chunk** (a few hundred tokens with overlap, respecting structure such as headings), attach metadata (source, date, permissions).
2. **Embedding and indexing:** convert chunks to vectors with an embedding model; store in a vector index (pgvector, Qdrant, Milvus, Weaviate, FAISS, OpenSearch k-NN) using approximate nearest-neighbor algorithms such as **HNSW** or IVF, plus a keyword index.
3. **Retrieval:** embed the question, fetch candidates by vector similarity and by BM25 keywords (**hybrid search**), filter by permissions and metadata, then **rerank** with a cross-encoder for precision.
4. **Generation:** build a prompt with the top passages and instructions, call the model, stream the answer, and attach citations.
5. **Quality and safety:** evaluation sets of questions with expected sources, metrics for retrieval (recall@k) and for answers (faithfulness, relevance), guardrails for prompt injection (retrieved text is *data*, never instructions), PII filtering, access control enforced at retrieval time.
6. **Cost and latency:** cache embeddings and frequent answers, use smaller models where adequate, limit context, batch offline work, route requests by difficulty, and set timeouts with graceful fallbacks.

Free and open options: sentence-transformers and BGE or E5 embedding models, Qdrant, Milvus, pgvector, FAISS, vLLM or llama.cpp or Ollama for serving open-weight models, LangChain or LlamaIndex or plain code for orchestration.

11.8 Recommender system architecture

Recommending from millions of items needs a **funnel**, because scoring everything with a heavy model is impossible within 100 ms.

1. **Candidate generation (retrieval):** thousands of candidates from several cheap sources: collaborative filtering, **two-tower embedding models** (user tower and item tower trained so that relevant pairs are close) served with an **approximate nearest-neighbor index** (FAISS, ScaNN, HNSW), popular and trending items, items similar to recent ones, followed-account content.
2. **Ranking:** a heavier model (gradient-boosted trees or a deep network) scores a few hundred candidates using rich features: user, item, context, cross features, predicted click, watch time, purchase probability.
3. **Re-ranking:** business rules and diversity, freshness, de-duplication, fairness constraints, ads insertion, safety filters.
4. **Exploration:** a small share of traffic tests new items and uncertain predictions (bandit methods) so the system does not trap itself in past behavior.

Hard problems: cold start (new users and items; use content features and popularity priors), position bias in click data, feedback loops, metric choice (clicks reward clickbait; long-term satisfaction is harder to measure), and latency at scale. **Offline vs online parts:** embeddings and candidate indexes are rebuilt periodically; features and the final ranking run online.

11.9 Ranking and matching systems (for example, resumes to job descriptions)

A document-matching or ranking system follows the same pipeline shape as search:

1. **Ingestion and normalization:** parse files into structured fields (sections, skills, titles, dates, education), normalize terms and synonyms (an ontology or skills taxonomy), handle languages and messy layouts. Parsing quality often limits the whole system.
2. **Indexing:** an inverted index for keyword matching plus embeddings for semantic matching.

3. **Retrieval:** BM25 and vector search to get a few hundred plausible matches quickly.
4. **Scoring and re-ranking:** a feature-based model (skill overlap, experience match, recency, title similarity, location) or a learned model (learning-to-rank such as LambdaMART, or a cross-encoder) produces the final order.
5. **Evaluation:** a human-judged relevance set, offline metrics (NDCG@k, MAP, MRR, recall@k), error analysis by slice, and online tests where feasible. Test with **synthetic and adversarial data** (keyword stuffing, formatting tricks) as well as real data.
6. **Explainability and fairness:** show which factors drove a score, audit for disparate impact across protected groups, avoid proxies for sensitive attributes, keep humans in the loop for consequential decisions, and document data and model limitations. These concerns are legal requirements in some jurisdictions for hiring-related systems.

11.10 Experimentation (A/B testing) basics

- **Randomize** by a stable unit (user ID) so a person always sees one variant; use a hash of user ID plus experiment name to assign buckets.
- Decide the **primary metric, guardrail metrics** (latency, errors, revenue, retention) and the **minimum detectable effect** before starting; compute the required **sample size** and run for whole weekly cycles.
- Do not **peek** and stop when results look good (inflates false positives); use sequential testing methods if you must monitor.
- Watch for **sample ratio mismatch** (assignment bugs), novelty effects, network effects (users interacting across variants), and many simultaneous tests interfering.
- Roll out with **feature flags** (Unleash, GrowthBook, Flagsmith, OpenFeature) so any change can be switched off instantly.

Part 12: Cloud, Containers, Kubernetes, DevOps and Cost

12.1 Cloud fundamentals

Service models. IaaS (virtual machines, networks, disks: EC2, Compute Engine), PaaS (you deploy code, the platform runs it: Heroku-style, App Engine, Cloud Run, Render, Fly.io), FaaS (functions: Lambda, Cloud Functions, Cloudflare Workers) and SaaS (finished

software). Moving up the stack buys speed and removes operations work, at the cost of control, portability and often unit price at scale.

Geography. A **region** is a cluster of data centres in one area; an **availability zone (AZ)** is an isolated data centre (or group) inside a region with separate power and networking. Spread replicas across at least two, preferably three, AZs for high availability. **Edge locations** (CDN points of presence) sit closer to users. Latency between AZs in a region is typically around one to a few milliseconds; between regions it is tens to hundreds.

Networking basics. A **VPC** is your private network; **subnets** divide it (public subnets have a route to the internet gateway, private ones do not); **security groups** are stateful instance-level firewalls and **network ACLs** are stateless subnet-level filters; **NAT gateways** let private instances make outbound calls (and are a notorious cost item); **private endpoints** keep traffic to managed services off the public internet; **VPNs, peering and transit gateways** connect networks. Put databases and internal services in private subnets and expose only load balancers.

Identity and access (IAM). Grant **least privilege** through roles, not long-lived keys; prefer short-lived credentials (instance roles, workload identity, OIDC federation from CI); require MFA for humans; separate accounts or projects for production, staging and development to limit the blast radius; log every administrative action.

Shared responsibility. The provider secures the cloud (hardware, hypervisor); you secure what you put in it (configuration, data, identities, patches for your own instances).

Lock-in and portability. Using managed databases and proprietary services speeds delivery and ties you to a vendor. Reduce painful lock-in with standard interfaces (PostgreSQL, S3 API, Kafka API, Kubernetes, OpenTelemetry, Terraform/OpenTofu), but do not contort your design for a hypothetical migration: true multi-cloud doubles operational complexity and is rarely worth it unless regulation or negotiation power demands it.

Low-cost and free paths. Every major provider has a free tier (limits and terms change, so verify the current ones). Oracle Cloud has offered an always-free allowance with Arm instances; Cloudflare's free plan covers DNS, CDN, DDoS protection and tunnels; budget VPS and bare-metal providers such as Hetzner or OVH give far more compute per rupee or dollar than hyperscalers at the cost of fewer managed services; platforms like Fly.io, Render, Railway, Vercel and Netlify have small free allowances; and your own hardware with Cloudflare Tunnel or Tailscale works for learning and small projects.

12.2 Containers

A container is a normal Linux process isolated by **namespaces** (own view of processes, network, mounts, users) and limited by **cgroups** (CPU, memory, I/O caps), started from an **image**: a stack of read-only layers plus metadata. It is not a virtual machine: containers share the host kernel, so they start in milliseconds and use little memory, but isolation is weaker than a VM's (hardened runtimes such as gVisor and Kata Containers add stronger isolation).

Image best practices.

- Small, pinned base images (distroless, Alpine or slim variants; pin versions or digests).
- **Multi-stage builds:** compile in a big builder stage, copy only the artifact into a minimal runtime stage.
- Order Dockerfile steps from least to most frequently changing so layer caching works (copy dependency manifests and install before copying source).
- Run as a **non-root** user, with a read-only root file system where possible; drop capabilities.
- One process per container, logs to stdout and stderr, configuration through environment variables or mounted files, handle SIGTERM for graceful shutdown.
- Scan images for vulnerabilities (Trivy, Grype), generate an SBOM, sign images (cosign).
- Never bake secrets into images.

Tooling: Docker Engine, Podman (daemonless, rootless), containerd and CRI-O (runtimes), Buildah and BuildKit, registries (Docker Hub, GHCR, ECR, GCR, Harbor, Zot).

12.3 Kubernetes

Kubernetes (K8s) is a **declarative** container orchestrator: you describe the desired state and **controllers** continuously reconcile reality toward it.

Architecture.

- **Control plane:** the **API server** (the only component that talks to storage and the front door for everything), **etcd** (the Raft-based key-value store holding cluster state), the **scheduler** (places pods on nodes by resources, affinity, taints), and **controller managers** (run the reconcile loops).
- **Nodes: kubelet** (starts pods and reports health), a container runtime, and **kube-proxy** or an eBPF dataplane (Cilium) for service networking.

Core objects.

Object	Purpose
Pod	One or more tightly coupled containers sharing network and storage; the unit of scheduling; ephemeral
Deployment / ReplicaSet	Run N identical stateless pods, rolling updates and rollback
StatefulSet	Pods with stable identities and per-pod storage (databases, Kafka, etcd)
DaemonSet	One pod per node (log shippers, node agents)
Job / CronJob	Run to completion, scheduled tasks
Service	Stable virtual IP and DNS name that load-balances to matching pods (ClusterIP, NodePort, LoadBalancer, headless)
Ingress / Gateway API	HTTP(S) routing from outside the cluster to Services (Nginx, Traefik, Envoy, Cilium implementations)
ConfigMap / Secret	Configuration and sensitive data injected as environment or files (Secrets are base64, not encrypted, unless you enable encryption at rest and use external secret managers)
PersistentVolume / Claim, StorageClass	Durable storage requests and provisioning
HPA, VPA, KEDA	Horizontal autoscaling on metrics, vertical right-sizing, event-driven scaling (queue depth, Kafka lag)
PodDisruptionBudget	Limit voluntary disruptions so upgrades do not take too many replicas down
NetworkPolicy	Allow-list pod-to-pod traffic
RBAC, ServiceAccount	Who may do what to which resources

Object	Purpose
CRD / Operator	Extend the API with your own resource types and controllers (databases, certificates, backups)

Production essentials.

- Set resource **requests** (scheduling guarantee) and **limits** (hard cap) on every container; requests equal to limits gives the Guaranteed QoS class. A memory limit breach kills the pod (OOMKilled); a CPU limit causes throttling.
- **Probes:** `livenessProbe` (restart if stuck), `readinessProbe` (receive traffic only when ready), `startupProbe` (protect slow starters). A bad liveness probe causes restart loops.
- **Graceful shutdown:** a preStop delay plus SIGTERM handling lets in-flight requests finish and load balancers deregister the pod.
- Spread replicas with **topology spread constraints** and anti-affinity across nodes and zones.
- Use **Helm** or **Kustomize** for templating, and **GitOps** (Argo CD, Flux) so Git is the source of truth and the cluster continuously syncs from it.
- Keep clusters upgraded, separate environments, restrict API access, apply Pod Security standards, and scan manifests (kube-linter, Kyverno, OPA Gatekeeper).

When *not* to use Kubernetes. For one or a few services run by a small team, a single VM with Docker Compose or systemd, or a PaaS, is simpler, cheaper and easier to debug. Kubernetes pays off with many services, multiple teams, need for self-healing and autoscaling, and platform standardization. Lighter options: **k3s** or **k0s** (small Kubernetes distributions), **HashiCorp Nomad**, **Docker Swarm**, **Podman with Quadlet and systemd**, Cloud Run or ECS Fargate (managed container runtimes with no cluster to run).

12.4 Deployment strategies

Strategy	How	Pros	Cons
Recreate	Stop old, start new	Simplest	Downtime
Rolling update	Replace instances gradually	No downtime, little extra capacity	Old and new versions run together; slow rollback

Strategy	How	Pros	Cons
Blue/green	Run full new environment, switch traffic atomically	Instant rollback, easy testing before switch	Double capacity during switch; database changes must be compatible with both
Canary	Send 1%, then 5%, 25%, 100% to the new version while watching metrics	Real-traffic validation, small blast radius	Needs good metrics and automation
Shadow (mirror)	Copy live traffic to the new version without returning its responses	Risk-free validation	Side effects must be suppressed; doubles load
Feature flags / dark launch	Deploy code turned off, enable per user or percentage	Decouples deploy from release; instant kill switch	Flag debt, combinatorial testing

Zero-downtime rules. Keep changes **backward and forward compatible** across at least one version (expand and contract for databases, tolerant readers for APIs and events); drain connections; use readiness probes; run migrations separately from the application rollout; always have a tested rollback path. **Progressive delivery** tools (Argo Rollouts, Flagger) automate canaries by analyzing metrics and rolling back on regressions.

12.5 CI/CD and software supply chain

- **Pipeline stages:** build, unit tests, static analysis and linting, dependency and secret scanning, build image, integration and contract tests, deploy to staging, smoke tests, promote to production. Fast feedback matters; keep the main pipeline under about ten minutes with caching and parallelism.
- **Trunk-based development** with short-lived branches, small frequent merges and feature flags beats long-lived branches.
- **Test pyramid:** many fast unit tests, fewer integration tests, few end-to-end tests; add **contract tests** (Pact) between services, **load tests** (k6, Locust, Gatling, wrk, vegeta) and chaos experiments for critical paths.
- **Promote artifacts, do not rebuild them:** the same image digest moves from staging to production.

- **Supply-chain security:** pin and review dependencies, SBOMs, signed images and provenance (Sigstore, cosign, SLSA levels), minimal CI permissions, short-lived credentials through OIDC, protected branches and required reviews.
- **DORA metrics** measure delivery health: deployment frequency, lead time for changes, change failure rate and time to restore service.
- **Free tools:** GitHub Actions (free tier), GitLab CI, Jenkins, Woodpecker CI, Drone, Tekton, Argo Workflows, Dagger.

12.6 Infrastructure as code

Describe infrastructure in version-controlled files so environments are reproducible, reviewable and recoverable.

- **Terraform** (BSL license) and its open-source fork **OpenTofu**, **Pulumi** (real programming languages), **AWS CDK**, **Crossplane** (Kubernetes-style control plane for cloud resources). **Ansible** configures machines (agentless, SSH, idempotent playbooks); Chef and Puppet are older alternatives; **cloud-init** and **Packer** build base images.
- **State:** Terraform tracks what it created in a state file; store it remotely with locking (object storage plus a lock table, or Terraform Cloud-style backends) and protect it, since it can contain secrets.
- **Practices:** small reusable modules, separate state per environment and per blast radius, plan-review-apply in CI, detect **drift** (manual console changes), policy as code (OPA, Sentinel, Checkov), and never edit production by hand.

12.7 Multi-AZ and multi-region design

Pattern	Description	Complexity	Typical RTO / RPO
Single region, multi-AZ	Replicas and load balancers across AZs	Low	Seconds to minutes / near zero
Backup region (cold or pilot light)	Data replicated (or backed up) to another region, minimal compute running	Medium	Hours / minutes
Warm standby	Scaled-down full stack in a second region	Medium-high	Minutes / seconds

Pattern	Description	Complexity	Typical RTO / RPO
Active-active multi-region	Both regions serve traffic and accept writes	Very high	Near zero

What makes multi-region hard is data. Writes in two regions need either a single-writer region per piece of data (user homing: each user's data lives in one region), conflict-free structures, or a globally consistent database (Spanner, CockroachDB, YugabyteDB) that pays cross-region latency on writes. Add global traffic management (anycast, GeoDNS, global load balancers), regional isolation of dependencies, data-residency rules, and failover that you **actually rehearse** (game days). Many outages in practice come from the failover machinery itself, or from hidden shared dependencies (a global control plane, a single identity provider). **Cell-based architecture** limits blast radius by running independent copies (cells) of the whole stack, each serving a subset of customers.

12.8 Autoscaling

- **Reactive scaling** adds capacity when a metric crosses a threshold. Choose a metric that reflects load and leads latency: CPU is a decent default for CPU-bound services; **requests per instance, queue depth or consumer lag** are better for I/O-bound or queue-based services.
- **Predictive and scheduled scaling** handles known patterns (morning peaks, campaigns) because instances take time to boot.
- **Mind the lags:** metric delay plus provisioning plus warm-up (JIT, cache fill) can exceed the spike you are chasing. Keep headroom and use **cooldowns** and hysteresis to avoid flapping.
- **Scale-to-zero** saves money for idle workloads (KEDA, Cloud Run, Lambda) and costs cold-start latency.
- **Limits matter:** the database, third-party APIs and downstream services do not scale as elastically; protect them with rate limits and queues. Autoscaling the app tier into an unscalable database simply moves the failure.
- **Cluster level:** node autoscalers (Cluster Autoscaler, Karpenter) provision machines for pending pods; mix on-demand and **spot or preemptible** instances for fault-tolerant work.

12.9 Cost engineering (FinOps)

Where money usually goes: compute that is idle or oversized; managed databases and their storage and IOPS; **data transfer out** (egress to the internet and across regions or AZs); NAT gateway processing; object storage requests and retrieval; observability data volume (logs and high-cardinality metrics can rival compute costs); and cross-AZ chatter in microservice meshes.

Levers.

- **Right-size** instances using real utilization; shut down dev environments outside working hours; delete orphaned disks, snapshots and load balancers.
- **Commitments:** reserved instances or savings plans for steady baseline load; **spot** for batch, CI and stateless workers.
- **Storage tiering and lifecycle rules**, compression, retention limits for logs.
- **Reduce egress:** CDN, compression, keep chatty services in the same zone, use private endpoints, consider providers with cheap or free egress (some CDNs and S3-compatible stores).
- **Architecture choices:** caching, batching, efficient serialization (Protobuf over verbose JSON), avoiding fan-out amplification, ARM-based instances for better price-performance.
- **Visibility:** tag every resource by team and service, set budgets and anomaly alerts, track **unit economics** (cost per 1,000 requests, per active user, per GB processed).
- **Build vs buy:** include engineering time and on-call burden in the comparison. A managed database at three times the instance price is often cheaper than a half-time engineer.

12.10 Stateful real-time fleets (game servers and similar)

Game servers, voice servers and simulation hosts behave differently from web services: they are **stateful, latency-sensitive, long-lived sessions**, often over UDP, with strict tick or frame budgets.

- **Allocation:** a **fleet manager** (Agones on Kubernetes, Nomad, custom daemons) keeps a pool of ready servers and hands one to a matchmaker or user; servers register, report health and player counts, and are drained rather than killed mid-session.

- **Placement:** put servers near players (regional fleets) because every 10 ms of latency is felt; use bare-metal or high-clock-speed instances for single-thread-bound simulations; avoid CPU throttling.
- **Control plane vs data plane:** a **panel or API** (web UI, auth, billing, scheduling) manages many **node daemons** that run server processes in containers or sandboxes, enforce resource limits, stream logs and console, manage files and backups. This split is how Pterodactyl-style panels and most hosting platforms work: the panel is a normal CRUD web app; the daemon is a small, reliable, privileged agent (a good fit for Go or Rust).
- **Networking:** allocate IP and port pairs per server, protect with DDoS mitigation (scrubbing, rate limits on the UDP edge, provider-level protection, Cloudflare Spectrum or similar for TCP and UDP proxying), keep control channels authenticated and encrypted.
- **Persistence:** world and player data in a database or object store with regular backups; local disks for the running session; snapshot before upgrades.
- **Scaling:** scale by whole servers, not by threads; autoscale the number of server instances on player demand and queue length, with warm spares to hide start-up time.
- **Security:** untrusted user-supplied code and mods require sandboxing (containers with seccomp, user namespaces, resource limits, no host mounts), per-tenant network isolation, and careful file-path handling.
- **Observability:** per-server tick time, player count, memory, network packet loss; alert on tick-time degradation, which is the early sign of overload.

Part 13: Reliability Engineering, Security and Evolvability

13.1 SLIs, SLOs and error budgets

- **SLI (service level indicator):** a measured number about user experience, expressed as a ratio of good events to total events. Examples: fraction of requests answered successfully; fraction served under 300 ms; fraction of messages delivered within 5 seconds; data freshness.
- **SLO (objective):** the target for the SLI over a window, for example 99.9% of requests succeed over 30 days.
- **SLA:** a contract with consequences (credits, penalties), always looser than the internal SLO.

- **Error budget:** 100% minus the SLO. A 99.9% monthly SLO allows 0.1% failure, which is **43.2 minutes** of full outage in a 30-day month (30 x 24 x 60 x 0.001). If you have budget left, ship faster; if you have burned it, freeze risky changes and spend effort on reliability. This turns the endless argument between features and stability into a shared number.

Alerting on burn rate. Instead of alerting on every blip, alert when the budget is being consumed too fast. A burn rate of 1 uses exactly the whole budget by the end of the window. Common multi-window rules for a 30-day SLO: page when the burn rate is 14.4 over one hour (2% of the monthly budget gone in an hour) and also over five minutes (confirms it is still happening); page at 6 over six hours (5% of the budget); ticket at 1 over three days. This catches fast outages quickly and slow leaks eventually, with few false alarms.

Choosing targets. Do not promise five nines; each extra nine costs roughly ten times more and your dependencies cap you anyway (the product of availabilities of everything in the request path). Pick the level users genuinely need, per user journey, and measure from the user's side (synthetic probes, client-side telemetry), not only from your servers.

13.2 Incident management

1. **Detect:** alerts based on symptoms; on-call acknowledgement within minutes.
2. **Triage and declare:** assign severity, an **incident commander** (coordinates, does not debug), a communications lead (status page, stakeholders) and technical responders.
3. **Mitigate first, diagnose later:** roll back the last change, shift traffic, disable a feature flag, scale up, fail over. Restoring service beats understanding the root cause during the outage.
4. **Communicate:** regular updates in one channel; a public status page for customer-facing impact.
5. **Resolve and verify.**
6. **Blameless postmortem:** a written timeline, impact, root causes (usually several contributing factors, never just one person's mistake), what went well, where luck helped, and concrete action items with owners and dates. The goal is to fix the system and the process.

On-call health: sustainable rotations, runbooks linked from every alert, alerts that are actionable (if an alert needs no action, delete it), and tracking of pages per shift. Burned-out on-call engineers make outages longer.

13.3 Testing for failure

- **Load testing** (expected traffic), **stress testing** (beyond limits to find the breaking point and failure mode), **soak testing** (hours or days to reveal leaks), **spike testing** (sudden surges). Use realistic data and traffic mixes, test with production-like infrastructure, and watch the saturated resource, not just average latency.
- **Chaos engineering:** form a steady-state hypothesis, inject a fault (kill instances, add latency, drop packets, fail a dependency, exhaust disk, skew a clock), observe, and fix the weakness. Start in staging with a small blast radius, then move to production with safeguards. Tools: Chaos Mesh, Litmus, Toxiproxy, tc and netem, Gremlin, AWS FIS.
- **Game days:** scheduled drills where the team practices a failover, a restore from backup, or a regional outage, with a script and a timer.
- **Backup restore drills:** prove RTO and RPO with real restores.
- **Resilience checklists per dependency:** What happens if it is down? Slow? Returns garbage? Returns an error halfway? Is the timeout shorter than my caller's? Is there a fallback? Is the retry safe?

13.4 Threat modeling

Security design starts with questions, not tools: **What are we protecting? Who might attack? How? What happens if they succeed? What is our defense and how do we know it works?**

STRIDE categories for each component and data flow: **S**poofing identity, **T**ampering with data, **R**epudiation (denying an action), **I**nformation disclosure, **D**enial of service, **E**levation of privilege. Draw a data-flow diagram with **trust boundaries** (internet to edge, edge to services, services to database, tenant to tenant), and ask the STRIDE questions at each boundary crossing. Rank threats by likelihood and impact and fix the top ones first.

Design principles: least privilege; defense in depth (several independent layers); secure defaults; fail closed for security decisions; minimize attack surface; separation of duties; economy of mechanism (simple designs are easier to secure); assume breach (limit lateral movement, log, detect).

13.5 Authentication and authorization in depth

Authentication flows.

- **Passwords** stored only as salted, slow hashes (**Argon2id**, scrypt, bcrypt) with per-user salts; enforce length over complexity rules; check against breached-password lists; rate limit and lock out gradually; never email passwords.
- **MFA:** TOTP apps, push, and best of all **passkeys (WebAuthn / FIDO2)**, which are phishing-resistant public-key credentials. SMS codes are better than nothing but vulnerable to SIM swapping.
- **Sessions:** a random, unguessable session ID in a cookie (`Secure`, `HttpOnly`, `SameSite=Lax` or `Strict`) mapped to server-side state in Redis or the database; easy to revoke. Rotate IDs on login and privilege change.
- **Tokens (JWT):** signed claims that stateless services can verify. Pitfalls: accepting `alg: none` or algorithm confusion (pin the expected algorithm), weak shared secrets, missing checks of `iss`, `aud`, `exp`, huge payloads, storing tokens in `localStorage` (readable by any XSS), and no revocation. Mitigate with **short-lived access tokens** (minutes) plus **rotating refresh tokens**, and a denylist or token-introspection for emergencies. JWTs are not automatically better than sessions; for a single web app, sessions are simpler and safer.
- **OAuth 2.0 and OpenID Connect:** OAuth 2.0 is delegated **authorization** (a user grants an app limited access to their resources); **OIDC** adds **authentication** (who the user is) through an ID token. Use the **authorization code flow with PKCE** for browsers, mobile and native apps; client credentials for service-to-service; avoid the deprecated implicit flow and the password grant. Do not write your own identity provider; use Keycloak, Authentik, Ory, Zitadel, or a managed service.
- **Service-to-service:** mTLS, signed service tokens (SPIFFE/SPIRE workload identities), short-lived credentials, never shared static secrets.

Authorization models.

Model	Idea	Fits
ACL	Per-resource list of who may do what	Files, sharing (Drive-style)
RBAC	Permissions attach to roles; users get roles	Most business apps
ABAC	Rules over attributes of user, resource and context (department, time, device)	Fine-grained, dynamic policies

Model	Idea	Fits
ReBAC (relationship-based)	Permissions derived from a graph of relationships (owner of folder implies edit of files inside; Google Zanzibar)	Collaboration products; tools: OpenFGA, SpiceDB, Ory Keto
Policy engines	Decouple decisions from code (OPA with Rego, Cedar)	Central, auditable policy

Always authorize on the server, on every request, against the specific object (prevent IDOR: insecure direct object reference, where changing an ID in a URL exposes someone else's data). Broken access control is the number-one category in the OWASP Top 10.

13.6 Common vulnerabilities and defenses

Risk	Defense
SQL / NoSQL / command injection	Parameterized queries and prepared statements, ORM query builders, never string-concatenate input, avoid shells
XSS (cross-site scripting)	Contextual output encoding, frameworks with auto-escaping, Content-Security-Policy, `HttpOnly` cookies
CSRF	SameSite cookies, anti-CSRF tokens, check Origin headers
SSRF (server tricked into calling internal URLs, such as cloud metadata endpoints)	Allow-list outbound destinations, block private ranges, enforce IMDSv2 on cloud metadata, network egress controls
Broken authentication / session fixation	Strong session management, MFA, rotation, rate limiting
Insecure deserialization	Never deserialize untrusted data with native object formats; use JSON with schemas
Path traversal, unrestricted upload	Canonicalize paths, store uploads outside the web root with random names, check type and size, scan

Risk	Defense
Mass assignment / over-posting	Explicit allow-lists of writable fields
Vulnerable dependencies	Dependency scanning (Dependabot, Renovate, OSV, Trivy), lock files, minimal dependencies
Secrets in repositories	Pre-commit and CI secret scanners (gitleaks, trufflehog), rotate on leak, vault-based injection
Misconfiguration (open buckets, default passwords, debug mode)	IaC scanning, baselines, regular audits
Logging sensitive data	Redact tokens, passwords, PII; control log access
Race conditions and business logic flaws (double spend, coupon reuse)	Atomic operations, unique constraints, idempotency, server-side validation of every rule
DoS	Rate limits, timeouts, request size limits, pagination caps, CDN and WAF, autoscaling with budgets, expensive-query limits (GraphQL depth and cost)

13.7 Cryptography and key management in practice

- **Never invent cryptography.** Use vetted libraries and high-level APIs (libsodium, Tink, platform TLS).
- **At rest:** disk or database encryption protects lost media; application-level or field-level encryption protects against a compromised database. **Envelope encryption** wraps per-object data keys with a master key held in a KMS or HSM (AWS KMS, Google Cloud KMS, HashiCorp Vault, a hardware module), so master keys never leave the secure boundary and rotation is cheap.
- **In transit:** TLS 1.2 or 1.3 everywhere, HSTS, mTLS internally for sensitive paths.
- **Hashing vs encryption vs signing:** hash for integrity and password storage (with the right slow function), encrypt for confidentiality (authenticated modes such as AES-GCM or ChaCha20-Poly1305), sign for authenticity (Ed25519, ECDSA, RSA-PSS), HMAC for symmetric message authentication.

- **Randomness:** use the operating system's cryptographic random source for tokens and keys, never a general-purpose pseudo-random generator.
- **Key rotation and compromise plans:** versioned keys, ability to re-encrypt, and a rehearsed response.

13.8 Network and infrastructure security

- **Zero trust:** authenticate and authorize every request regardless of network location; no implicit trust for "internal" traffic.
- **Segmentation:** separate networks or policies for public, application, data and management tiers; deny by default, allow specific flows (Kubernetes NetworkPolicy, security groups).
- **Edge protection:** CDN or WAF in front (Cloudflare, AWS WAF, ModSecurity with the OWASP Core Rule Set), DDoS mitigation, bot management, TLS termination with modern settings.
- **Administrative access:** no public SSH or databases; use bastions, VPNs, Tailscale-style mesh networks, or identity-aware proxies; short-lived certificates (SSH CA), session recording, MFA.
- **Patching:** automated OS and dependency updates, immutable infrastructure (replace rather than patch in place), minimal images.
- **Detection:** centralized immutable audit logs (cloud audit trails, Kubernetes audit logs), intrusion detection (Falco for runtime, Wazuh, Suricata), alerting on anomalies such as new admin users or unusual egress.
- **Backups and ransomware resilience:** offline or immutable copies (object lock), separate credentials, tested restores.

13.9 Multi-tenancy

SaaS products serve many customers (tenants) on shared infrastructure. Choose an isolation model per data and risk level:

Model	Description	Pros	Cons
Shared tables, tenant_id column	All tenants in the same tables, every query filtered by tenant	Cheapest, simplest operations, easy cross-tenant analytics	One missed filter leaks data (mitigate with PostgreSQL row-level security, query wrappers, tests), noisy neighbors
Schema per tenant	One schema per customer in a shared database	Better isolation, per-tenant backup and migration	Migration fan-out, connection and catalog overhead at thousands of tenants
Database per tenant	Separate databases (possibly on shared servers)	Strong isolation, per-tenant restore and placement	Operational cost, harder analytics
Stack per tenant (silo)	Fully separate deployments	Strongest isolation, custom SLAs and compliance	Most expensive, hardest to operate

Many products mix models (pooled for small customers, siloed for large or regulated ones). Also plan **noisy-neighbor controls** (per-tenant rate limits, quotas, fair queuing), tenant-aware observability, per-tenant encryption keys for sensitive customers, and data export and deletion workflows.

13.10 Privacy and compliance

- **Data minimization:** do not collect what you do not need; classify data (public, internal, confidential, regulated) and know where each class lives.
- **Regulations to know:** GDPR (EU), India's Digital Personal Data Protection Act, CCPA/CPRA (California), HIPAA (US health), PCI-DSS (cards), SOC 2 and ISO 27001 (organizational security attestations). Common requirements: lawful basis and consent, purpose limitation, access and deletion rights, breach notification, data-processing agreements, data residency.
- **Design for deletion:** if user data is copied into logs, caches, search indexes, analytics and backups, deletion becomes a project. Keep PII in few well-known places, key encrypted data per user so deleting the key erases the data (**crypto-shredding**), and

keep personal identifiers out of logs and event streams.

- **Audit trails:** who accessed or changed what and when, tamper-evident and retained according to policy.

13.11 Evolvability: designing for change

Systems live for years; the ability to change safely is a first-class requirement.

- **API and schema evolution:** only make backward-compatible changes (add optional fields; never reuse or repurpose a field; deprecate with a long runway); version explicitly when you must break. For events and RPC, use schema formats with compatibility rules (Protobuf, Avro with a registry) and apply the **robustness principle** (producers conservative, consumers tolerant of unknown fields).
- **Contracts between teams:** consumer-driven contract tests, published OpenAPI or Protobuf definitions, and changelogs.
- **Modularity:** strong boundaries and narrow interfaces (the modular monolith); hide data stores behind their owning module; avoid shared databases across services.
- **Conway's law:** systems mirror the communication structure of the organization that builds them. Design team boundaries and service boundaries together (the "inverse Conway maneuver").
- **Architecture decision records (ADRs):** short documents capturing the context, the decision, the alternatives rejected and the consequences, stored in the repository, so future engineers understand why. Write one for every significant trade-off.
- **Technical debt management:** make it visible, budget a fraction of capacity for repayment, and tie refactoring to concrete goals rather than aesthetics.
- **Strangler fig migrations:** route traffic gradually from an old system to a new one behind a facade; keep both running until the new one proves itself; avoid big-bang rewrites.
- **Documentation:** a context diagram, a container diagram and a handful of key sequence diagrams (the C4 model) are usually enough; runbooks for operations; keep them next to the code.

Part 14: Hands-On Configurations and Code (Infrastructure)

These are realistic starting points for the building blocks in the earlier parts. They are written to be correct and idiomatic, but they were **not executed in this environment**: treat them as templates, check versions and adjust names, ports and sizes to your setup.

14.1 Dockerfiles (multi-stage, non-root)

Go service (tiny static image):

```
# syntax=docker/dockerfile:1
FROM golang:1.23 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download                    # cached unless dependencies change
COPY . .
RUN CGO_ENABLED=0 go build -trimpath -ldflags='-s -w' -o /out/app ./cmd/app

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/app /app
USER nonroot:nonroot
EXPOSE 8080
ENTRYPOINT ["/app"]
```

Python (FastAPI) service:

```
FROM python:3.12-slim AS base
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd -r -u 10001 appuser
USER appuser
EXPOSE 8000
HEALTHCHECK CMD python -c "import
urllib.request;urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--
workers", "2"]
```

14.2 Docker Compose: a complete small production-like stack

Load balancer (Nginx), three app replicas, PostgreSQL, Redis, Prometheus and Grafana, all free software.

```
services:
  nginx:
    image: nginx:1.27-alpine
    ports: ["80:80"]
    volumes: ["./nginx.conf:/etc/nginx/nginx.conf:ro"]
    depends_on: [app]

  app:
    build: ./app
    environment:
      DATABASE_URL: postgres://app:secret@db:5432/app
      REDIS_URL: redis://cache:6379/0
    deploy:
      replicas: 3
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_started
    restart: unless-stopped

  db:
    image: postgres:16
    environment:
      POSTGRES_USER: app
      POSTGRES_PASSWORD: secret          # use Docker secrets or an env file
outside Git in real use
      POSTGRES_DB: app
    volumes: ["pgdata:/var/lib/postgresql/data"]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U app"]
      interval: 5s
      timeout: 3s
      retries: 10

  cache:
    image: redis:7-alpine
    command: ["redis-server", "--maxmemory", "256mb", "--maxmemory-policy",
```

```
"allkeys-lru"]

  prometheus:
    image: prom/prometheus:latest
    volumes: ["./prometheus.yml:/etc/prometheus/prometheus.yml:ro"]
    ports: ["9090:9090"]

  grafana:
    image: grafana/grafana:latest
    ports: ["3000:3000"]
    volumes: ["grafana:/var/lib/grafana"]

volumes:
  pgdata:
  grafana:
```

Run it with `docker compose up -d --build --scale app=3`. Pin real version tags instead of `latest` in anything you keep.

14.3 Nginx as load balancer, cache and TLS terminator

```
worker_processes auto;
events { worker_connections 10240; }

http {
  # per-client rate limit: 10 requests/sec with bursts of 20
  limit_req_zone $binary_remote_addr zone=perip:10m rate=10r/s;

  # small cache for GET responses
  proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=api:50m max_size=1g
inactive=10m use_temp_path=off;

  upstream app_backend {
    least_conn;
    server app:8000 max_fails=3 fail_timeout=10s;   # Compose DNS returns
every replica
    keepalive 64;                                     # reuse upstream
connections
  }

  gzip on;
  gzip_types text/plain application/json text/css application/javascript;
```

```
  map $http_upgrade $connection_upgrade { default upgrade; '' close; }

  server {
    listen 80;
    server_name example.com;

    location /api/ {
      limit_req zone=perip burst=20 nodelay;
      limit_req_status 429;

      proxy_http_version 1.1;
      proxy_set_header Connection "";
      proxy_set_header Host $host;
      proxy_set_header X-Real-IP $remote_addr;
      proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
      proxy_set_header X-Forwarded-Proto $scheme;

      proxy_connect_timeout 2s;
      proxy_read_timeout 10s;
      proxy_next_upstream error timeout http_502 http_503;   # retry
idempotent failures on another node
      proxy_next_upstream_tries 2;

      proxy_cache api;
      proxy_cache_valid 200 30s;
      proxy_cache_use_stale error timeout updating http_500 http_502
http_503;
      proxy_cache_lock on;                                   # stampede
protection
      proxy_pass http://app_backend;
    }

    location /ws/ {                                          # WebSocket
traffic
      proxy_pass http://app_backend;
      proxy_http_version 1.1;
      proxy_set_header Upgrade $http_upgrade;
      proxy_set_header Connection $connection_upgrade;
      proxy_read_timeout 3600s;                              # longer than
the heartbeat interval
    }
```

```
    location /static/ {
      root /var/www;
      expires 1y;
      add_header Cache-Control "public, immutable";
    }
  }
}
```

For HTTPS add a `listen 443 ssl http2;` server block with `ssl_certificate` and `ssl_certificate_key` (Let's Encrypt via certbot), or use Caddy, which obtains and renews certificates automatically with a two-line config.

14.4 HAProxy (Layer 4 and 7 in one file)

```
global
  maxconn 50000
defaults
  mode http
  timeout connect 3s
  timeout client  30s
  timeout server  30s
  option httpchk GET /health

frontend web
  bind *:80
  default_backend apps

backend apps
  balance leastconn
  server app1 10.0.0.11:8000 check inter 2s fall 3 rise 2
  server app2 10.0.0.12:8000 check inter 2s fall 3 rise 2
  server app3 10.0.0.13:8000 check inter 2s fall 3 rise 2 backup

frontend extra_listener
  # note: HAProxy balances TCP and HTTP only
  bind *:81
  default_backend apps
```

(The last block only illustrates a limit: HAProxy does not balance UDP. Nginx's `stream` module and Linux IPVS do.)

14.5 Kubernetes manifests for a stateless service

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 3
  strategy:
    rollingUpdate: { maxSurge: 1, maxUnavailable: 0 }
  selector:
    matchLabels: { app: api }
  template:
    metadata:
      labels: { app: api }
    spec:
      terminationGracePeriodSeconds: 30
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: topology.kubernetes.io/zone
          whenUnsatisfiable: ScheduleAnyway
          labelSelector: { matchLabels: { app: api } }
      containers:
        - name: api
          image: registry.example.com/api:1.4.2     # pin a version or digest
          ports: [{ containerPort: 8080 }]
          envFrom:
            - configMapRef: { name: api-config }
            - secretRef: { name: api-secrets }
          resources:
            requests: { cpu: 250m, memory: 256Mi }
            limits:   { memory: 512Mi }
          readinessProbe:
            httpGet: { path: /ready, port: 8080 }
            periodSeconds: 5
          livenessProbe:
            httpGet: { path: /health, port: 8080 }
            initialDelaySeconds: 10
            periodSeconds: 10
          lifecycle:
            preStop:
              exec: { command: ["sleep", "10"] }     # let load balancers
deregister first
```

```
          securityContext:
            runAsNonRoot: true
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
---
apiVersion: v1
kind: Service
metadata:
  name: api
spec:
  selector: { app: api }
  ports: [{ port: 80, targetPort: 8080 }]
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api
spec:
  scaleTargetRef: { apiVersion: apps/v1, kind: Deployment, name: api }
  minReplicas: 3
  maxReplicas: 30
  metrics:
    - type: Resource
      resource:
        name: cpu
        target: { type: Utilization, averageUtilization: 60 }
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: api
spec:
  minAvailable: 2
  selector:
    matchLabels: { app: api }
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-allow-ingress-only
spec:
  podSelector: { matchLabels: { app: api } }
  policyTypes: [Ingress]
```

```
  ingress:
    - from:
        - namespaceSelector:
            matchLabels: { kubernetes.io/metadata.name: ingress-nginx }
      ports: [{ port: 8080 }]
```

14.6 Terraform / OpenTofu: a small example

A remote state backend and an object-storage bucket (AWS syntax; the same shape applies to other providers).

```
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  backend "s3" {
    bucket         = "my-team-tfstate"
    key            = "prod/network.tfstate"
    region         = "ap-south-1"
    dynamodb_table = "tf-locks"      # state locking
    encrypt        = true
  }
}

provider "aws" { region = "ap-south-1" }

variable "env" { type = string }

resource "aws_s3_bucket" "uploads" {
  bucket = "myapp-${var.env}-uploads"
}

resource "aws_s3_bucket_public_access_block" "uploads" {
  bucket                  = aws_s3_bucket.uploads.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

resource "aws_s3_bucket_lifecycle_configuration" "uploads" {
```

```
  bucket = aws_s3_bucket.uploads.id
  rule {
    id     = "tier-old-objects"
    status = "Enabled"
    filter {}
    transition { days = 30  storage_class = "STANDARD_IA" }
    transition { days = 180 storage_class = "GLACIER" }
  }
}
```

Workflow: `tofu init`, `tofu plan -out=plan`, review, `tofu apply plan`, all in CI with a human approval for production.

14.7 Prometheus: scrape config and alerts

```
# prometheus.yml
global: { scrape_interval: 15s }
rule_files: ["alerts.yml"]
scrape_configs:
  - job_name: app
    metrics_path: /metrics
    static_configs: [{ targets: ["app:8000"] }]
```

```
# alerts.yml: symptom-based alerts and SLO burn rate
groups:
  - name: slo
    rules:
      - alert: HighErrorBurnRate
        # 99.9% SLO: budget is 0.1%. Burn rate 14.4 over 1h AND 5m
        expr: |
          (sum(rate(http_requests_total{code=~"5.."}[1h])) /
sum(rate(http_requests_total[1h])) > 14.4 * 0.001)
          and
          (sum(rate(http_requests_total{code=~"5.."}[5m])) /
sum(rate(http_requests_total[5m])) > 14.4 * 0.001)
        labels: { severity: page }
        annotations: { summary: "Error budget burning 14x too fast" }

      - alert: HighLatencyP99
        expr: histogram_quantile(0.99, sum by (le)
(rate(http_request_duration_seconds_bucket[5m]))) > 0.5
```

```
        for: 10m
        labels: { severity: ticket }

      - alert: InstanceDown
        expr: up == 0
        for: 2m
        labels: { severity: page }
```

14.8 Load test with k6

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '2m', target: 100 },    // ramp up
    { duration: '5m', target: 100 },    // steady
    { duration: '2m', target: 400 },    // spike
    { duration: '2m', target: 0 },      // ramp down
  ],
  thresholds: {
    http_req_failed:   ['rate<0.01'],   // under 1% errors
    http_req_duration: ['p(95)<300'],   // 95% under 300 ms
  },
};

export default function () {
  const res = http.get('https://staging.example.com/api/items?limit=20');
  check(res, { 'status is 200': (r) => r.status === 200 });
  sleep(1);
}
```

Run `k6 run script.js`. Watch the saturated resource (database connections, CPU, queue depth) while it runs, and always test against a production-like environment, never against production without safeguards.

14.9 GitHub Actions CI pipeline

```
name: ci
on:
  push: { branches: [main] }
  pull_request:
permissions: { contents: read }
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_PASSWORD: test }
        ports: ["5432:5432"]
        options: >-
          --health-cmd="pg_isready" --health-interval=5s --health-timeout=3s
--health-retries=10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12", cache: pip }
      - run: pip install -r requirements.txt
      - run: pytest -q
        env: { DATABASE_URL: postgres://postgres:test@localhost:5432/postgres
}
  image:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    permissions: { contents: read, packages: write, id-token: write }
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with: { registry: ghcr.io, username: ${{ github.actor }}, password:
${{ secrets.GITHUB_TOKEN }} }
      - uses: docker/build-push-action@v6
        with: { push: true, tags: "ghcr.io/${{ github.repository }}:${{
github.sha }}" }
```

14.10 SQL patterns used again and again

Idempotent writes with a unique key:

```
CREATE TABLE idempotency_key (
  key        TEXT PRIMARY KEY,
  request_hash TEXT NOT NULL,
  response   JSONB,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- first call inserts; a retry hits the conflict and reads the stored
response
INSERT INTO idempotency_key (key, request_hash) VALUES ($1, $2)
ON CONFLICT (key) DO NOTHING;
```

Transactional outbox:

```
BEGIN;
UPDATE orders SET status = 'PAID' WHERE id = 42;
INSERT INTO outbox (aggregate_id, type, payload)
VALUES (42, 'OrderPaid', '{"order_id":42,"amount":49900}');
COMMIT;   -- a relay (or Debezium) publishes outbox rows to Kafka and marks
them sent
```

Optimistic locking:

```
UPDATE document SET body = $1, version = version + 1
WHERE id = $2 AND version = $3;     -- 0 rows updated means someone else
changed it; reload and retry
```

Row-level security for multi-tenant isolation (PostgreSQL):

```
ALTER TABLE invoice ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON invoice
  USING (tenant_id = current_setting('app.tenant_id')::bigint);
-- per request, after acquiring a connection:  SET LOCAL app.tenant_id =
'123';
```

Time-range partitioning with cheap retention:

```
CREATE TABLE event (
  id BIGINT GENERATED ALWAYS AS IDENTITY,
  occurred_at TIMESTAMPTZ NOT NULL,
  payload JSONB,
  PRIMARY KEY (id, occurred_at)
) PARTITION BY RANGE (occurred_at);

CREATE TABLE event_2026_10 PARTITION OF event
  FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
-- retention: DROP TABLE event_2026_01;  (instant, no giant DELETE and no
vacuum debt)
```

Keyset pagination:

```
SELECT id, created_at, title
FROM post
WHERE (created_at, id) < ($1, $2)
ORDER BY created_at DESC, id DESC
LIMIT 50;      -- needs an index on (created_at DESC, id DESC)
```

Part 15: Hands-On Programs (Tested)

Unlike Part 14, every program in this part was **executed and its output checked** in a test environment (Node.js 18+ and Python 3). They are small, but they implement the real mechanisms behind systems described earlier.

15.1 A reverse tunnel in about 90 lines (Node.js)

The mechanism behind ngrok, frp, rathole and Cloudflare Tunnel. A client behind NAT dials **out** to a public server; when a visitor hits the server's public port, the server tells the client over the control connection, the client opens a fresh **data connection** back to the server, and the server pipes the visitor's socket into it. The client pipes that data connection to the local service.

```
visitor --> [public :8080] SERVER <--control (outbound from client)-- CLIENT
--> local app :3000
                         SERVER <--data conn (outbound, per visitor)-- CLIENT
```

server.js

```
const net = require('net');
const crypto = require('crypto');

const CONTROL_PORT = +process.env.CONTROL_PORT || 7000;
const PUBLIC_PORT  = +process.env.PUBLIC_PORT  || 8080;
const TOKEN        = process.env.TOKEN || 'change-me';

let control = null;                 // the client's control socket
const pending = new Map();          // id -> visitor socket waiting for a
data connection

// read the first line of a socket and hand back any extra bytes
function readLine(sock, cb) {
  let buf = Buffer.alloc(0);
  const onData = (chunk) => {
    buf = Buffer.concat([buf, chunk]);
    const i = buf.indexOf('\n');
    if (i === -1) { if (buf.length > 1024) sock.destroy(); return; }
    sock.off('data', onData);
    const rest = buf.subarray(i + 1);
    if (rest.length) sock.unshift(rest);
    cb(buf.subarray(0, i).toString().trim());
  };
  sock.on('data', onData);
}

net.createServer((sock) => {         // control and data connections from the
client
  sock.on('error', () => {});
  readLine(sock, (line) => {
    const [cmd, arg] = line.split(' ');
    if (cmd === 'HELLO') {
      const a = Buffer.from(arg || ''), b = Buffer.from(TOKEN);
      if (a.length !== b.length || !crypto.timingSafeEqual(a, b)) return
sock.destroy();
      if (control) control.destroy();           // newest client wins
      control = sock;
      sock.write('OK\n');
      sock.setKeepAlive(true, 15000);
      sock.on('close', () => { if (control === sock) control = null; });
    } else if (cmd === 'DATA' && pending.has(arg)) {
      const pub = pending.get(arg); pending.delete(arg);
      pub.pipe(sock); sock.pipe(pub);
```

```
      const end = () => { pub.destroy(); sock.destroy(); };
      pub.on('close', end); sock.on('close', end);
    } else sock.destroy();
  });
}).listen(CONTROL_PORT, () => console.log('control on', CONTROL_PORT));

net.createServer((pub) => {          // visitors
  pub.on('error', () => {});
  if (!control) return pub.end('no tunnel connected\n');
  const id = crypto.randomBytes(8).toString('hex');
  pending.set(id, pub);
  setTimeout(() => { if (pending.delete(id)) pub.destroy(); }, 10000);   //
client never answered
  control.write(`NEW ${id}\n`);
}).listen(PUBLIC_PORT, () => console.log('public on', PUBLIC_PORT));
```

client.js

```
const net = require('net');
const SERVER = process.env.SERVER_HOST || '127.0.0.1';
const PORT   = +process.env.CONTROL_PORT || 7000;
const LOCAL  = +process.env.LOCAL_PORT || 3000;
const TOKEN  = process.env.TOKEN || 'change-me';

function connect(attempt = 0) {
  const ctl = net.connect(PORT, SERVER, () => ctl.write(`HELLO ${TOKEN}\n`));
  let buf = '';
  ctl.setKeepAlive(true, 15000);
  ctl.on('data', (d) => {
    buf += d.toString();
    let i;
    while ((i = buf.indexOf('\n')) !== -1) {
      const line = buf.slice(0, i).trim(); buf = buf.slice(i + 1);
      if (line === 'OK') { attempt = 0; console.log('tunnel up'); }
      else if (line.startsWith('NEW ')) handle(line.slice(4));
    }
  });
  ctl.on('error', () => {});
  ctl.on('close', () => {                        // reconnect with
exponential backoff and jitter
    const wait = Math.min(30000, 500 * 2 ** attempt) * (0.5 + Math.random() /
2);
    console.log('control lost, retry in', Math.round(wait), 'ms');
```

```
    setTimeout(() => connect(attempt + 1), wait);
  });
}

function handle(id) {
  const data = net.connect(PORT, SERVER, () => data.write(`DATA ${id}\n`));
  const local = net.connect(LOCAL, '127.0.0.1');
  data.on('error', () => local.destroy());
  local.on('error', () => data.destroy());
  data.pipe(local); local.pipe(data);
}
connect();
```

Test result. With a local HTTP server on port 3000, the server and client running, and three concurrent requests to port 8080, all three returned the local server's response through the tunnel.

What this teaches, and what is missing for production:

Concept shown	Production gap
Outbound-initiated connection defeats NAT	**No TLS**: wrap both sockets in TLS (or run over WireGuard/SSH); the shared token travels in clear text here
Control channel plus per-visitor data connections	A new TCP handshake per visitor adds latency; real systems **multiplex streams** over one connection (yamux, smux, HTTP/2, QUIC)
Token authentication with constant-time comparison	Per-user tokens in a database, rotation, rate limits, quotas
Reconnect with backoff and jitter, keepalive	Application heartbeats to detect half-dead links, session resumption
Pending-connection timeout	Backpressure and bandwidth caps per tunnel
Single tunnel, single public port	Many tunnels: route by Host header (HTTP) or SNI (TLS), registry of subdomain to tunnel, port allocator for raw TCP, multiple edge nodes with a shared registry

Concept shown	Production gap
Node.js event loop	Excellent for prototypes; Go or Rust are the usual choice for the production data plane

15.2 Saga with compensation (Python)

A booking-style transaction across services without a distributed lock: reserve stock, charge the card, create the shipment; if any step fails, undo the earlier ones in reverse order.

```
class Saga:
    def __init__(self):
        self.steps = []                       # (name, action, compensation)
    def step(self, name, action, compensation):
        self.steps.append((name, action, compensation)); return self
    def run(self, ctx):
        done = []
        for name, action, comp in self.steps:
            try:
                action(ctx); done.append((name, comp))
            except Exception as exc:
                print(f'step {name} failed: {exc}; compensating')
                for n, c in reversed(done):
                    try: c(ctx)
                    except Exception as e2: print(f'  COMPENSATION {n} FAILED
(needs manual action): {e2}')
                    else: print(f'  compensated {n}')
                return False
        return True

def reserve_stock(c):   c['stock'] = 'reserved'
def release_stock(c):   c['stock'] = 'released'
def charge_card(c):     raise RuntimeError('card declined')
def refund_card(c):     c['payment'] = 'refunded'
def create_shipment(c): c['shipment'] = 'created'
def cancel_shipment(c): c['shipment'] = 'cancelled'

ctx = {}
```

```
ok = (Saga().step('stock', reserve_stock, release_stock)
            .step('payment', charge_card, refund_card)
            .step('shipping', create_shipment, cancel_shipment)).run(ctx)
print(ok, ctx)
```

Output: `step payment failed: card declined; compensating`, then `compensated stock`, then `False {'stock': 'released'}`.

Production additions: persist saga state after every step so a crash resumes where it stopped (this is what workflow engines like Temporal do for you); make every action and compensation **idempotent**; handle a compensation that itself fails (retry, then raise an alert for manual action); remember that intermediate states are visible to other users (a stock reservation exists briefly), which is the price of giving up atomic isolation.

15.3 Transactional outbox and idempotent consumer (Python and SQLite)

```
import sqlite3, json
db = sqlite3.connect(':memory:')
db.executescript('''
CREATE TABLE orders(id INTEGER PRIMARY KEY, status TEXT);
CREATE TABLE outbox(id INTEGER PRIMARY KEY AUTOINCREMENT, type TEXT, payload
TEXT, sent INTEGER DEFAULT 0);
CREATE TABLE processed(event_id INTEGER PRIMARY KEY);   -- consumer-side
dedupe table
''')

def pay_order(order_id):
    with db:                                  # one transaction: state change
and event, atomically
        db.execute('INSERT OR REPLACE INTO orders VALUES (?,?)', (order_id,
'PAID'))
        db.execute('INSERT INTO outbox(type,payload) VALUES (?,?)',
                   ('OrderPaid', json.dumps({'order_id': order_id})))

def relay_once(publish):                      # run in a loop; publishing is
at-least-once
    rows = db.execute('SELECT id,type,payload FROM outbox WHERE sent=0 ORDER
BY id LIMIT 100').fetchall()
    for id_, typ, payload in rows:
        publish(id_, typ, payload)            # may run twice if we crash
```

```
before the UPDATE below
        with db: db.execute('UPDATE outbox SET sent=1 WHERE id=?', (id_,))

def consume(event_id, typ, payload, handler): # idempotent consumer
    try:
        with db:
            db.execute('INSERT INTO processed(event_id) VALUES (?)',
(event_id,))   # fails if already seen
            handler(json.loads(payload))
    except sqlite3.IntegrityError:
        print('duplicate event', event_id, 'ignored')

pay_order(42)
published = []
relay_once(lambda i, t, p: published.append((i, t, p)))
effects = []
for ev in published + published:              # simulate duplicate delivery
    consume(*ev, handler=lambda d: effects.append(d['order_id']))
print('effects applied:', effects)
```

Output: `duplicate event 1 ignored` and `effects applied: [42]`: the event was delivered twice and the effect happened once. This pairing (outbox on the producer side, dedupe table on the consumer side) is how "exactly-once effects" are built from at-least-once delivery.

15.4 Worker pool with backpressure, and hedged requests (Python asyncio)

```
import asyncio, random, time

async def worker(name, q, results):
    while True:
        job = await q.get()
        try:
            await asyncio.sleep(random.uniform(0.01, 0.03))     # pretend
work
            results.append((name, job))
        finally:
            q.task_done()

async def main():
```

```
    q = asyncio.Queue(maxsize=10)          # bounded: producers feel
backpressure
    results = []
    workers = [asyncio.create_task(worker(f'w{i}', q, results)) for i in
range(4)]
    for job in range(50):
        await q.put(job)                   # blocks when the queue is full
    await q.join()                         # wait until every job is
processed
    for w in workers: w.cancel()           # graceful shutdown
    await asyncio.gather(*workers, return_exceptions=True)
    print('processed', len(results))

async def hedged(fn, delay=0.05):
    """Send a second copy if the first has not answered within delay; the
first result wins."""
    first = asyncio.create_task(fn())
    done, _ = await asyncio.wait({first}, timeout=delay)
    if done:
        return first.result()
    second = asyncio.create_task(fn())
    done, pending = await asyncio.wait({first, second},
return_when=asyncio.FIRST_COMPLETED)
    for p in pending: p.cancel()
    return done.pop().result()

async def flaky():
    await asyncio.sleep(0.5 if random.random() < 0.05 else 0.01)   # 5% slow
tail
    return 'ok'

async def demo():
    for label, call in (('hedged', lambda: hedged(flaky)), ('plain ',
flaky)):
        lat = []
        for _ in range(1000):
            t = time.perf_counter(); await call();
lat.append(time.perf_counter() - t)
        lat.sort(); print(label, 'p99 (s):', round(lat[int(.99 * len(lat))],
3))

asyncio.run(main()); asyncio.run(demo())
```

Measured output: 50 jobs processed; **hedged p99 about 0.061 s versus plain p99 about 0.501 s.** A duplicate request after a 50 ms delay cut the 99th percentile roughly eightfold when 5% of calls were slow, at the price of about 5% extra load. Two caveats worth remembering: hedge only **idempotent** calls, and when the slow fraction is large (a test with 25% slow calls showed no improvement at p99) hedging cannot save you because both copies are slow too, and the extra load makes things worse.

15.5 Quick experiments to run yourself

Each takes under an hour and teaches more than reading:

1. **Measure the cache.** Put an LRU in front of a slow function (`time.sleep(0.05)`), replay a Zipf-distributed key stream (a few keys are very popular) and plot hit rate against cache size. You will see why a small cache absorbs most traffic.
2. **Break consistent hashing.** Compare `hash(key) % N` with the ring when you change N from 4 to 5; count moved keys (about 80% vs about 20%).
3. **Create a retry storm.** Simulate 1,000 clients retrying without jitter against a server that fails for 2 seconds, then add jitter and a circuit breaker, and compare peak request rates.
4. **Watch queueing theory happen.** Load-test a single-worker service at 50%, 80%, 90% and 95% utilization and plot p50 and p99 latency; the hockey stick appears exactly as the table in 8.5 predicts.
5. **Kill a replica.** Run PostgreSQL with a streaming replica (two containers), write continuously, stop the primary, promote the replica and measure what was lost with asynchronous versus synchronous replication.
6. **Make Redis lie.** Use a Redis lock without fencing tokens, pause the holder (SIGSTOP) past the lock expiry, and watch two clients believe they hold the lock.
7. **Reproduce a lost update and a write skew** with two `psql` sessions at read committed and at serializable isolation.
8. **Extend the tunnel:** add TLS, then multiplex with a stream library, then add Host-header routing for multiple tunnels.

Part 16: Reference: Decisions, Numbers, Glossary, Q&A and Debugging

16.1 Decision cheat sheet: choose A when, choose B when

Decision	Choose A when	Choose B when
Monolith vs microservices	Small team, unclear domain, speed matters (modular monolith)	Many teams, clear bounded contexts, independent scaling or release needs
SQL vs NoSQL	Relational data, transactions, ad-hoc queries	Massive scale with known access patterns, flexible schema, multi-region writes
Vertical vs horizontal scaling	Early stage, stateful single node, simplicity	Beyond one machine's limits, need redundancy
Cache-aside vs write-through	Read-heavy, tolerate occasional stale read	Need cache always fresh, writes are fewer than reads
Sync vs async (queue)	Caller needs the result now, low latency, simple flow	Work is slow or spiky, caller can continue, need retries and buffering
Queue vs log (RabbitMQ vs Kafka)	Task distribution, per-message routing, simple consumers	Event streaming, replay, many independent consumers, huge throughput
REST vs gRPC vs GraphQL	Public, cacheable, simple CRUD (REST)	Internal low-latency typed calls (gRPC); varied client data needs (GraphQL)
Strong vs eventual consistency	Money, inventory, uniqueness, permissions	Feeds, counters, analytics, recommendations
Fan-out on write vs on read	Many reads per write, bounded followers	Huge fan-out (celebrities), many inactive consumers
Sessions vs JWT	One web app, need revocation and simplicity	Many independent services verifying identity without a lookup (use short expiry)

Decision	Choose A when	Choose B when
Polling vs WebSocket vs SSE	Infrequent updates (polling); one-way server push (SSE)	Bidirectional low-latency (WebSocket)
Push vs pull (CDN)	Known popular content, predictable launches (push)	Long tail, unpredictable access (pull)
Leader-based vs leaderless replication	Need simple consistency model, single write region	Always-writable, multi-region, tolerate conflict resolution
Orchestration vs choreography (sagas)	Complex flows, need central visibility and control	Simple flows, loose coupling, few steps
Managed vs self-hosted	Small team, non-core component, need reliability fast	Cost at scale, control, compliance, portability
Kubernetes vs simpler hosting	Many services, many teams, autoscaling, standard platform	One to a few services, small team (VM, Compose, PaaS)
Single region vs multi-region	Users concentrated, tolerance for rare regional outage	Strict availability targets, global users, residency laws
Batch vs streaming	Hours-old results fine, simplicity	Seconds-level freshness needed
Range vs hash sharding	Range queries and ordered scans important	Even load distribution matters most
Optimistic vs pessimistic locking	Conflicts rare, long operations	Conflicts frequent, short critical sections
Build vs buy	Differentiating capability, unusual requirements	Commodity capability (auth, payments, email, search hosting)

16.2 Numbers to remember

Orders of magnitude, useful for sanity checks. Real figures depend heavily on hardware, data and configuration, so measure your own.

Item	Rough magnitude
Single Redis node, simple commands	100,000+ operations/sec, sub-millisecond
Single PostgreSQL or MySQL node, simple indexed queries	Thousands to tens of thousands of queries/sec; writes fewer
Single Nginx node serving static or proxied traffic	Tens of thousands to 100,000+ requests/sec
Kafka cluster	Hundreds of thousands to millions of messages/sec in aggregate
Single app server (typical API, I/O-bound)	Hundreds to a few thousand requests/sec
Concurrent WebSockets per tuned server	50,000 to several hundred thousand
SSD sequential read	Several GB/sec (NVMe)
10 Gbit network link	About 1.2 GB/sec
Same-AZ round trip	0.2 to 1 ms
Cross-region round trip	50 to 250 ms
Seconds per day / month / year	86,400 / about 2.6 million / about 31.5 million
1 million requests/day	About 12 per second
1 KB x 1 million	1 GB
Text tweet-sized record (about 300 bytes) x 1 billion	300 GB
Photo (about 200 KB to 2 MB) x 1 million	200 GB to 2 TB
HD video, about 3 to 5 Mbit/sec	1 to 2 GB per hour
Powers of two	2^10 = 1 thousand, 2^20 = 1 million, 2^30 = 1 billion, 2^40 = 1 trillion

16.3 Glossary

Term	Short definition
ACID	Atomicity, consistency, isolation, durability: guarantees of transactional databases
Active-active	Multiple sites or nodes serve live traffic simultaneously
Anycast	One IP announced from many places; the network routes to the nearest
API gateway	Single entry point handling auth, routing and rate limiting for APIs
At-least-once	Delivery guarantee: never lost, possibly duplicated
Availability zone (AZ)	Isolated failure domain inside a cloud region
Backpressure	Slow consumers signaling producers to slow down
BASE	Basically available, soft state, eventual consistency
Blast radius	How much breaks when one thing fails
Bloom filter	Compact probabilistic set: no false negatives, some false positives
Blue/green deploy	Two full environments; switch traffic between them
Bulkhead	Isolating resources so one failure cannot sink everything
Cache stampede	Many requests rebuild the same expired entry at once
CAP theorem	During a partition, choose consistency or availability
Canary release	Sending a small share of traffic to a new version first
CDC	Change data capture: stream of database changes from its log

Term	Short definition
CDN	Network of edge caches near users
Circuit breaker	Stops calls to a failing dependency, probes for recovery
Consensus	Agreement among nodes on a value despite failures (Raft, Paxos)
Consistent hashing	Key placement on a ring so node changes move few keys
CQRS	Separate models for writing and reading
CRDT	Data type whose replicas always merge without conflicts
Dead-letter queue	Parking place for messages that repeatedly fail
Denormalization	Duplicating data to speed up reads
Distributed tracing	Following one request across many services
Eventual consistency	Replicas converge if updates stop
Event sourcing	Storing the sequence of events as the source of truth
Exponential backoff	Doubling wait time between retries, usually with jitter
Failover	Switching to a standby when the primary fails
Fan-out	One event or request triggering many downstream actions
Fencing token	Increasing number that lets a resource reject stale lock holders
Gossip protocol	Nodes spread information by talking to random peers
Graceful degradation	Reduced functionality instead of total failure
Health check	Probe that decides whether an instance can receive traffic

Term	Short definition
Heartbeat	Periodic signal showing a node is alive
Hot spot / hot key	Disproportionately loaded partition or key
Idempotency	Repeating an operation has the same effect as doing it once
Index	Auxiliary structure that speeds lookups at write cost
Jitter	Randomness added to timers to avoid synchronized behavior
Leader election	Choosing the single coordinator among nodes
Linearizability	Strongest consistency: behaves like one copy with instant operations
Load shedding	Rejecting low-priority work to protect critical work
LSM tree	Write-optimized storage with memtable, SSTables and compaction
Materialized view	Precomputed query result stored for fast reads
Merkle tree	Hash tree used to find differences between datasets
MVCC	Keeping row versions so readers do not block writers
Multi-tenancy	Serving many customers on shared infrastructure
mTLS	TLS where both client and server present certificates
Noisy neighbor	One tenant's heavy use degrading others
Outbox pattern	Writing events to a table in the same transaction as the data change
PACELC	CAP extended: else, trade latency against consistency
Partition (data)	A subset of data held by one node or shard

Term	Short definition
Partition (network)	Network split preventing nodes from communicating
p99 latency	Value below which 99% of requests complete
Quorum	Majority of nodes needed for a decision
Raft	Understandable consensus algorithm with leader and log replication
Rate limiting	Capping request rate per client
Read replica	Copy of a database serving reads
Replication lag	Delay before a replica reflects the primary's writes
RPO / RTO	Acceptable data loss / acceptable downtime
Saga	Long transaction as local steps with compensations
Service discovery	How services find each other's addresses
Service mesh	Proxy layer handling service-to-service networking concerns
Sharding	Splitting data across nodes by key
Sidecar	Helper container deployed alongside the main one
SLI / SLO / SLA	Measurement / target / contract of service quality
Split brain	Two nodes both believing they are the leader
Stateless	No per-client state kept between requests on the server
Strangler fig	Gradually replacing a legacy system behind a facade
Throughput	Work completed per unit time
Tombstone	Delete marker in LSM-style stores
Trie	Prefix tree for string lookups
TTL	Time to live: expiry time for data or cache entries

Term	Short definition
Two-phase commit	Prepare-then-commit protocol for atomic distributed transactions
Vector clock	Logical clock detecting causality and concurrency
WAL	Write-ahead log: changes logged before applying to data files
Write amplification	Disk bytes written per user byte
Zero downtime deploy	Releasing without user-visible outage
Zero trust	No implicit trust based on network location

16.4 Questions and model answers

1. How do you scale a read-heavy system? Add caching at every layer (browser, CDN, application, Redis), add read replicas, denormalize or precompute hot reads (materialized views, feed caches), index properly, and paginate. Accept that replicas and caches add staleness; decide how much is tolerable per data type.

2. How do you scale a write-heavy system? Reduce write cost (batch, group commits, append-only logs), buffer through a queue, use a write-optimized store (LSM-based), partition by a high-cardinality key, drop unnecessary indexes, move non-critical writes off the request path, and aggregate counters before writing. Sharding is the last step, not the first.

3. SQL or NoSQL? Start from access patterns and consistency needs, not fashion. Relational data with transactions and unknown future queries points to SQL (PostgreSQL by default). Enormous scale with known key-based access, flexible schemas or multi-region writes points to a NoSQL store, with the constraint that you design tables around queries.

3a. Why is exactly-once delivery hard? Because a sender cannot distinguish a lost request from a lost acknowledgement. You either risk duplicates (retry) or loss (do not retry). Practical systems use at-least-once delivery with idempotent processing, producing effectively-once results.

4. How do you handle cache invalidation? Prefer short TTLs plus explicit invalidation on writes; publish change events so all caches can evict; version keys; design data so stale reads are acceptable; protect against stampedes with coalescing and jittered expiry.

5. What happens if the cache goes down? Traffic floods the database. Defenses: replicate the cache and use failover, keep a small in-process fallback cache, rate-limit and shed load at the database, warm caches before returning to service, and use request coalescing. Design the database tier to survive degraded cache hit rates or fail gracefully.

6. How do you choose a shard key? High cardinality, even distribution, stable over time, aligned with the dominant query (so most queries hit one shard), avoids monotonic values (timestamps) that create hot shards. Think about cross-shard operations and resharding before committing.

7. How do you deal with a hot partition or hot key? Detect it from per-partition metrics; mitigate by replicating or caching the hot item, adding a random suffix to spread writes (and aggregating on read), splitting the partition, or rate limiting the offender. Redesign the key if the skew is structural.

8. How do you make an API idempotent? Require a client-generated idempotency key for state-changing operations, store the key with the request fingerprint and the response under a unique constraint, return the stored response for repeats, and expire keys after a window.

9. How do you migrate a database with zero downtime? Expand and contract: add new structures compatibly, dual-write, backfill in batches, verify, switch reads, stop old writes, drop later. For moving between databases, use change data capture to replicate continuously, compare data, cut over with a short write pause or dual-writing, and keep a rollback path.

10. Why not microservices everywhere? They trade code complexity for distributed-systems complexity: network failures, versioning, distributed transactions, observability, deployment pipelines, and organizational overhead. Unless independent scaling, team autonomy or fault isolation is genuinely needed, a modular monolith is faster and cheaper to build and run.

11. How do you design for 10x growth? Identify the first bottleneck with measurement (load test), make the stateless tier horizontally scalable, add caching and queues, plan the data layer's scaling path (replicas, partitioning) before it is urgent, automate capacity, and set alerts on leading indicators (utilization, queue depth) rather than on outages.

12. What is the difference between availability and reliability? Availability is the fraction of time the system responds; reliability is whether it behaves correctly over time. A system that is up but returns wrong answers is available and unreliable.

13. How do you ensure ordering? Only where needed, and only within a key: partition messages by the entity ID so one consumer handles them in order; use sequence numbers or versions; avoid global ordering, which needs a single sequencer and limits throughput.

14. How do you prevent cascading failures? Timeouts everywhere, bounded queues, retries with backoff and budgets, circuit breakers, bulkheads, load shedding, graceful degradation, and capacity headroom. Test with fault injection.

15. How would you secure a new public API? TLS, authentication (OAuth 2.0 or API keys with scopes), authorization on every object, input validation and output encoding, rate limiting and quotas, request size limits, logging and monitoring without sensitive data, dependency scanning, and a threat model of the main abuse cases.

16.5 Debugging playbook: "the system is slow"

1. **Define the symptom precisely:** which endpoint, which users, since when, p50 or p99, errors or just latency? Compare with the same time last week.
2. **Check what changed:** deploys, config or feature-flag changes, traffic shifts, data growth, dependency incidents, certificate or DNS changes, cron jobs and backups at that time.
3. **Look at the golden signals** (latency, traffic, errors, saturation) at each tier from the edge inward. Find the first tier where the numbers diverge.
4. **Find the saturated resource** using the USE method per resource: CPU, memory (swap, GC), disk I/O, network, file descriptors, connection pools, thread pools, locks, queue depths.
5. **Use traces** to see where time goes in a slow request: a downstream call, the database, a lock wait, a retry loop.
6. **Database specifics:** slow query log or `pg_stat_statements`, `EXPLAIN (ANALYZE, BUFFERS)`, lock waits, long-running transactions, replication lag, connection counts, autovacuum activity, cache hit ratio.
7. **Check for retries and amplification:** is traffic to a dependency higher than user traffic? Retry storms and fan-out multiply load.

8. **Mitigate first:** roll back, scale out, shed load, disable a heavy feature, fail over, restart a stuck component. Capture evidence (thread dumps, heap profiles, `perf`, flame graphs) before restarting if you can.
9. **Find the root cause and the contributing factors** afterwards, write the postmortem, and fix the system (alerts, limits, tests), not just the instance.

Closing note

If you read and understand Parts 1 to 3, you have the vocabulary and the mental models. Parts 7 and 8 explain why they behave as they do. Parts 9 to 11 show the models working on real products. Parts 12 and 13 cover running the result in production. Parts 14 and 15 give you things to run. The remaining skill is practice: pick a system, estimate it, design it, break it on paper, then build a small piece of it and measure. Do that repeatedly and the trade-offs stop being a list you memorized and become judgment.